

INDUSTRY PROJECT

ON

E-Commerce Web App

Submitted in partial fulfillment of requirements for the award of Degree of

MASTER OF COMPUTER APPLICATIONS
in
DEPARTMENT OF COMPUTER APPLICATION
Engineering College Bikaner

Session-2024-2026

Submitted by:

Guided by:

Submitted to:

(signature)

(signature)

(signature)

Name: Gagan Saini

Name: Kunal B. Ranga

Name: Sanjay Tejasvee

Father's Name:

Anand Prakash Saini

MCA Department

Engineering College

Bikaner Bikaner

Technical University,

Bikaner, Rajasthan.

HOD, MCA

Department

Engineering College

Bikaner Bikaner

Technical University,

Bikaner, Rajasthan

Roll No:

Semester: 4th



TABLE OF CONTENTS

1. Executive Technical Overview
2. System Topology
3. Frontend Architecture
4. Backend Architecture
5. Authentication & Authorization
6. Database Design
7. Domain Modeling
8. Cart System Architecture
9. Checkout & Order Architecture
10. Payment Processing Architecture
11. Product Catalog & Search
12. Inventory Design
13. Image Storage Architecture
14. Seller Platform Architecture
15. State Management Architecture
16. Component Architecture & Design System
17. API Design Architecture
18. Validation Architecture
19. Pricing & Discount Engine
20. Routing Architecture
21. Performance Engineering
22. Security Architecture
23. Reliability Engineering
24. Scalability Analysis
25. Deployment & Infrastructure
26. Observability & Monitoring
27. Testing Architecture
28. CI/CD Pipeline
29. Engineering Decision Records (EDRs)
30. Production Readiness Assessment
31. Scaling Roadmap
32. Glossary

1. EXECUTIVE TECHNICAL OVERVIEW

Closet Fashion is a full-stack fashion e-commerce platform built for individual buyers and independent sellers. The platform enables product discovery, cart management, direct purchase, seller onboarding, and order history. It is deployed entirely on Vercel -- the frontend as a static React SPA and the backend as a serverless Node.js function set.

The system is architecturally a two-tier monolith: a React single-page application communicating directly over HTTP with a single Express.js server backed by a MongoDB Atlas cluster. There are no microservices, no message queues, no caching layers, and no background workers. All business logic lives inside controller functions co-located in a single Express process.

PLATFORM SCALE CONTEXT

This platform is at an early-stage, MVP product maturity level. The architectural decisions -- embedded cart/order arrays in user documents, base64 image storage in MongoDB, no payment processing, single-tier Express -- reflect a build-and-ship philosophy prioritizing velocity over operational durability. Many production-grade concerns (rate limiting, observability, inventory control, real payment integration) are deliberately deferred or entirely absent.

This document does not judge that posture. It accurately describes what exists, why patterns were chosen, where the risk surface lies, and what must change at each growth threshold.

TECHNOLOGY STACK SUMMARY

The e-commerce platform follows a modern full-stack JavaScript architecture designed for scalability, maintainability, and rapid deployment. The frontend layer is built using React version 18.2.0, enabling a

component-driven UI architecture with efficient rendering and state-driven updates. The frontend application is deployed through the Vercel CDN infrastructure, allowing globally distributed static asset delivery with low-latency performance optimization. Application bundling and production build generation are handled through Create React App using `react-scripts` version 5.0.1, providing a standardized build pipeline, environment management, asset optimization, and production-ready static compilation.

Client-side routing is implemented using React Router DOM version 6.7.0, enabling single-page application navigation without full page reloads. This routing architecture improves user experience through faster transitions and reduced server dependency during navigation events. Styling and UI consistency are managed using Tailwind CSS version 3.4.4, integrated through a PostCSS-based build workflow. This utility-first CSS architecture improves maintainability, reduces custom stylesheet complexity, and enables responsive design implementation with minimal overhead. Communication between the frontend and backend services is performed using Axios version 1.2.4, operating within the browser environment to manage asynchronous HTTP requests, API integration, request interception, and centralized error handling workflows.

The backend infrastructure is built on Node.js with Express version 4.18.2 serving as the primary application framework. The backend is deployed using Vercel Serverless Functions, enabling automatic scaling, simplified deployment pipelines, and reduced infrastructure management overhead. The application database layer uses MongoDB Atlas as the managed cloud database provider, while Mongoose version 6.8.3 acts as the Object Data Modeling (ODM) layer responsible for schema validation, model abstraction, query management, and middleware integration. The database is hosted on the MongoDB Atlas M0 shared cluster tier, which provides a lightweight managed database environment suitable for small-to-medium scale workloads and development-stage deployments.

Authentication within the platform is implemented using JSON Web Tokens through the `jsonwebtoken` package version 9.0.0. The system follows a stateless authentication architecture where access tokens are generated and verified without requiring server-side session persistence. This approach improves horizontal scalability and simplifies distributed deployment handling. Password security is implemented using `bcrypt` version 5.1.0 for server-side password hashing and credential verification. The hashing mechanism ensures sensitive user credentials are never stored in plaintext and provides resistance against common credential compromise attacks.

For user interaction feedback and alert management, the application integrates SweetAlert version 2.1.2 within the browser environment. This library is used to provide modal-based notifications, confirmation dialogs, validation feedback, and transactional alerts across critical user workflows such as authentication, checkout actions, order placement, and administrative operations.

LIVE DEPLOYMENT ENDPOINTS

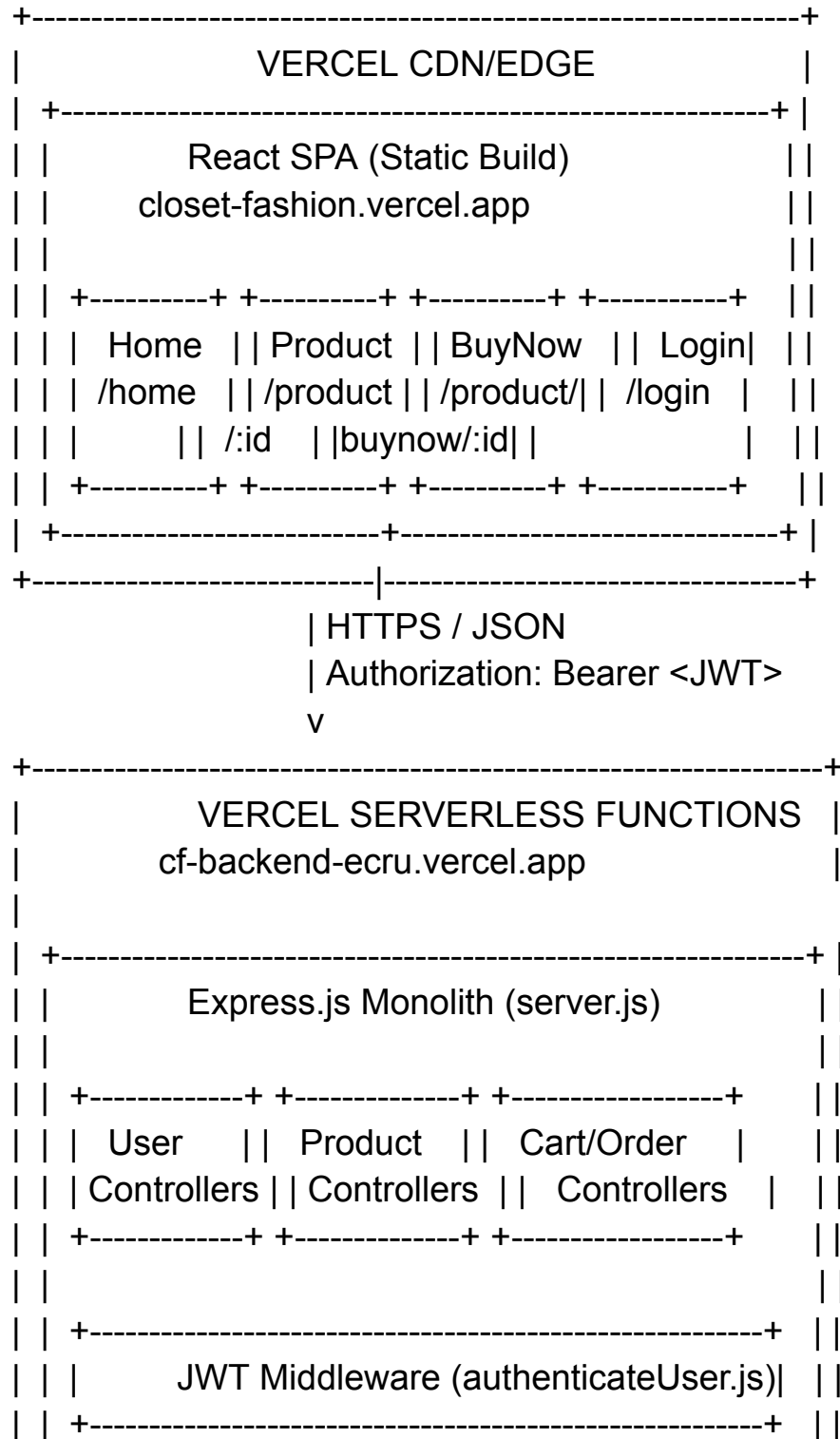
Frontend: <https://closet-fashion.vercel.app>

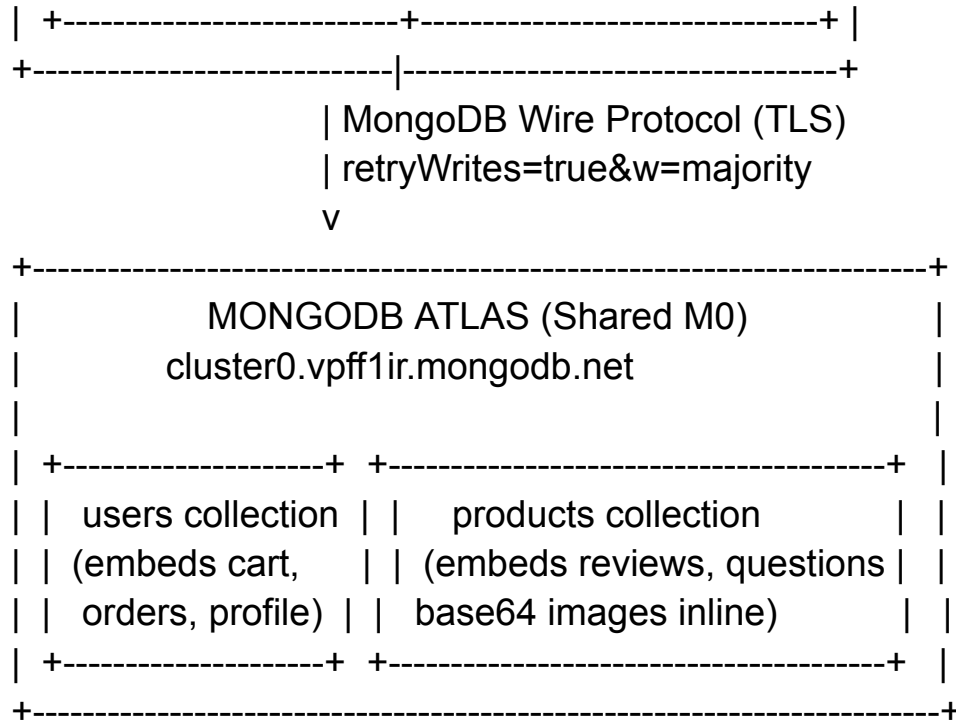
Backend API: <https://cf-backend-ecru.vercel.app>

Database: MongoDB Atlas cluster0.vpff1ir.mongodb.net

2. SYSTEM TOPOLOGY

MACRO ARCHITECTURE DIAGRAM





REQUEST LIFECYCLE - AUTHENTICATED API CALL

Browser (React)

- ```

|
| 1. Read authToken from localStorage
| 2. Axios POST with body + Authorization header
|
v

```

Vercel Edge Network

- ```

| 3. TLS termination, route to serverless function
v

```

Express server.js

- ```

| 4. cors() -- origin whitelist check
| 5. express.json() -- parse body (50MB limit)
| 6. authenticateJWT middleware
| |- read includeAuthToken from body
| |- extract Bearer token from Authorization header
| |- jwt.verify(token, AUTH_TOKEN_SECRET)
| |- User.findById(decoded._id) <- DB round-trip every request

```

- | +- attach user document to req.user
- | 7. Controller function
- | |- business logic
- | |- Mongoose queries
- | +- res.json(result)
- v

MongoDB Atlas

- | 8. Query execution
- | 9. Result returned
- v

Express Controller

- | 10. JSON serialization
- v

Browser (React)

- | 11. axios response -> state update -> re-render

## **SERVICE COMMUNICATION MODEL**

The system uses synchronous HTTP only. Every operation is a direct request-response cycle. There is no:

- Message queue (no RabbitMQ, SQS, Kafka)
- Background job processing
- Webhook infrastructure
- WebSocket / Server-Sent Events
- Cache invalidation bus

All data mutations are synchronous writes to MongoDB, and all reads are synchronous MongoDB queries. This is architecturally simple and correct for an MVP, but creates hard scalability and reliability ceilings discussed in Section 24.



## 3. FRONTEND ARCHITECTURE

### RENDERING MODEL

Closet Fashion uses Client-Side Rendering (CSR) exclusively, bootstrapped via Create React App (CRA). The public/index.html ships a bare `<div id="root"></div>` shell. React hydrates it on the client.

Why CSR only: CRA was chosen for its zero-configuration development setup. There is no Next.js or Remix SSR layer. This means:

1. First Contentful Paint (FCP) is delayed -- the browser must download the JS bundle, parse it, execute React, then fetch data from the API before anything meaningful renders.
2. SEO is non-existent -- product pages are not server-rendered, so web crawlers see empty HTML. Product catalog is invisible to search engines.
3. No static generation -- product listing pages are always fetched at runtime.

The tradeoff accepted here is simplicity. CRA produces a fully static artifact (HTML + JS + CSS) that Vercel can serve from CDN globally with zero server infrastructure for the frontend layer.

### APPLICATION ENTRY POINT

```
src/index.js
+-- ReactDOM.createRoot(document.getElementById('root'))
+-- <App />
+-- createBrowserRouter([...routes])
+-- RouterProvider
```

index.js is intentionally thin -- it creates the React root and calls `reportWebVitals()` (which logs to console but sends metrics nowhere). The router and context providers are configured inside `App.jsx`.

## ROUTING ARCHITECTURE

React Router DOM v6 with `createBrowserRouter` and the Outlet nested route pattern:

|                                  |                                          |                              |
|----------------------------------|------------------------------------------|------------------------------|
| <code>/</code>                   | <code>-&gt; &lt;AppLayout /&gt;</code>   | (persistent navbar + outlet) |
| <code>/login</code>              | <code>-&gt; &lt;Login /&gt;</code>       | (standalone, no layout)      |
| <code>/home</code>               | <code>-&gt; &lt;Home /&gt;</code>        | (product listing)            |
| <code>/product/:id</code>        | <code>-&gt; &lt;ProductPage /&gt;</code> | (product detail)             |
| <code>/product/buynow/:id</code> | <code>-&gt; &lt;BuyNow /&gt;</code>      | (checkout flow)              |
| <code>/</code>                   | <code>-&gt; redirect -&gt; /home</code>  |                              |
| <code>*</code>                   | <code>-&gt; redirect -&gt; /home</code>  |                              |

`AppLayout` is the persistent shell component. It renders `<Navbar>`, wraps everything in `<SearchContextProvider>`, and renders `<Outlet>` for child routes.

The layout is always mounted when the user is on any route except `/login`.

The routing decision to keep `/login` outside `AppLayout` is correct -- the login page needs a completely different visual layout (split-screen design) with no navbar or filter panel.

Deep-link behavior: `createBrowserRouter` produces history-mode URLs. Vercel handles the SPA routing correctly because its build output config rewrites all paths to `index.html`. No 404s on direct URL access.

## COMPONENT HIERARCHY

App

- +-- ThemeProvider (context)
  - +-- Login (standalone route)
  - +-- AppLayout
    - +-- SearchContextProvider (context)
    - +-- Navbar
      - | +-- NavbarRightOptions
      - | +-- ProfileSlider (slide-over panel)
        - | | +-- Profile tab -> Profile.jsx
        - | | +-- Orders tab -> MyOrders.jsx
        - | | +-- Seller tab -> SellerAccount.jsx OR UploadProduct.jsx
        - | | +-- Settings tab -> Settings.jsx -> ManageTheme.jsx
      - | +-- CartSlider (slide-over panel)
        - | +-- Cart.jsx
          - | +-- CartCard.jsx (per item)
          - | +-- CheckoutCard.jsx (per item, right panel)
- +-- Outlet
  - +-- Home
    - | +-- Filters
      - | | +-- FilterCard (per filter group)
      - | | +-- ColorFilter
      - | | +-- RangePicker (price range)
    - | +-- ProductCard1 (per product)
  - +-- ProductPage
    - | +-- ProductImages
    - | +-- ColorFilterCard (readonly)
    - | +-- Checkbox (readonly for sizes)
  - +-- BuyNow
    - +-- ProductImages
    - +-- QuantityInput
    - +-- Checkbox (size selection)
    - +-- ColorFilterCard (color selection)
    - +-- PaymentItem (per payment method)

## STATE ARCHITECTURE

No global state manager (no Redux, no Zustand, no Jotai). State is organized in three tiers:

Tier 1 - React Context (global cross-cutting):

- ThemeContext -- light/dark mode, persisted to localStorage["cf-theme"]
- SearchContext -- current search query string, shared between Navbar search input and Home component

Tier 2 - App-level state (prop-drilled):

- authToken -- stored in App state, initialized from localStorage["authToken"], prop-drilled four levels deep to Navbar, ProfileSlider, Home, ProductPage, CartSlider
- showFilterSection -- boolean, controls filter panel visibility on mobile
- showCartSlider, showProfileSlider -- boolean toggles for slide-over panels

Tier 3 - Component-local state:

- allProducts, page, totalPages -- pagination state in Home
- selectedFilters -- multi-dimensional filter state in Home
- cartProducts, productPriceDetails -- cart data in Cart.jsx
- product, sellerDetails -- product detail data in ProductPage
- Form state is managed entirely by the useForm custom hook

Architectural tension: authToken is simultaneously stored in App component state and localStorage. Two reads occur -- once during render and once inside useEffect. Components that need the token independently read from localStorage (e.g., Home.jsx reads it directly via localStorage.getItem("authToken")), bypassing the prop-drilled state. This creates an inconsistency where the React state and localStorage can briefly diverge on logout. The fix would be a dedicated AuthContext.

## CUSTOM HOOKS

### 1. useAPI (src/hooks/useAPI.js)

A generic data-fetching hook wrapping Axios:

```
useAPI(method, path, includeAuthToken, variables, headers,
dependencies) { data, loading, error }
```

Fires on mount and whenever dependencies change. Uses useCallback to memoize fetchData with JSON.stringify(variables) and JSON.stringify(headers) as dependencies -- a common React pattern to avoid deep-equality comparisons, but one that causes unnecessary refetches when object references change even if values are identical.

### 2. useDebounceAPI (src/hooks/useDebounceAPI.js)

A debounced variant of useAPI using useRef for the timeout handle.

Default debounce delay is 200ms;

Home uses 250ms. Used for:

- Product listing (triggers on page change and filter change)
- Any search-triggered API calls

The debounce implementation correctly uses a ref rather than state to store the timer ID, avoiding the stale-closure issue present in the naive useDebounce hook.

### 3. useDebounce (src/hooks/useDebounce.js)

A lower-level debounce hook that stores the timer ID in useState rather than useRef. This is a bug: setting timerId via setTimerId causes a re-render, which re-triggers the useEffect, creating potential for a cascade. This hook is not used in the main product listing but exists as an earlier implementation. The useDebounceAPI hook supersedes it.

### 4. useForm (src/hooks/useForm.js)

A generic controlled-form hook:

`useForm(initialValues, validationRules, onSubmit)`

`-> { formData, errors, isSubmitting, setFormData, setErrors,  
handleChange, handleSubmit }`

Validation rules are a map of `fieldName -> (value, formData) -> errorMessage` or undefined. Cross-field validation (e.g., card number required only when payment method is card) is supported via the second `formData` argument passed to each rule.

`handleSubmit` runs all validators synchronously, collects errors, and calls `onSubmit` only if validation passes. `isSubmitting` is set to true then immediately back to false -- it wraps no async operation directly, so it cannot be used as an async loading indicator.

## THEME SYSTEM

Two themes: light and dark. Persisted to `localStorage["cf-theme"]`, defaulting to light. Applied via conditional Tailwind class strings:  
`className={theme === "light" ? "bg-White text-Black" : "bg-Black text-White"}`

The custom Tailwind palette is:

- White: `#F0F3F6` (cool off-white)
- Light: `#DFE7F1` (light blue-gray)
- Black: `#1E1E1E` (near-black)
- Purple: `#7B68EE` (medium-slate-blue -- primary brand color)
- Gray: `#999999`
- Green: `#48D198`
- Red: `#FF0000`
- Blue: `#0F4392`
- Yellow: `#EFC863`

The theme toggle is buried inside the Profile Slider -> Settings tab. It is not accessible without being logged in (the slider is only shown when

authToken is present). Theme toggle for guest users is unavailable.

## INFINITE SCROLL IMPLEMENTATION

Home uses the Intersection Observer API for infinite scroll pagination:

```
const lastProductElementRef = useCallback((node) => {
 if (loading) return;
 if (observer.current) observer.current.disconnect();
 observer.current = new IntersectionObserver((entries) => {
 if (entries[0].isIntersecting && page < totalPages) {
 setPage(prev => prev + 1);
 }
 });
 if (node) observer.current.observe(node);
}, [loading, page, totalPages]);
```

This ref is attached to the last product card in the grid. When it enters the viewport, page increments, triggering a new API call which appends results to allProducts.

Critical behavior difference when filters are active: When any filter is applied, the backend ignores pagination and returns all matching results (offset is forced to 0). The frontend then replaces allProducts entirely rather than appending:

```
if (filtersActive || searchQuery?.length > 0) {
 setAllProducts([...data?.products]);
} else {
 setAllProducts(prev => [...prev, ...data?.products]);
}
```

This means filtered searches return unbounded result sets -- a filter for "Nike" products returns every Nike product in one response, not paginated. For large catalogs this becomes a scalability problem.

## 4. BACKEND ARCHITECTURE

### ARCHITECTURE PATTERN

The backend is a single-file Express monolith. All routes are registered in `server.js`. Controllers are organized into two directories:

- `controllers/userControls/` -- 6 user management controllers
- `controllers/productControls/` -- 11 product/cart/order controllers

There is no:

- Router-level modularization (`express.Router()`)
- Service layer (business logic lives directly in controllers)
- Repository/DAO pattern (Mongoose models called directly in controllers)
- Dependency injection
- Middleware pipeline beyond CORS + body parsing + JWT auth

This is a classic fat controller pattern. Business logic, data access, and HTTP handling are all co-located in each controller file.

### MODULE STRUCTURE

```
backend/
+-- server.js <- Express app, all route registration
+-- config/
| +-- db-config.js <- MongoDB connection setup
+-- middleware/
| +-- authenticateUser.js <- JWT verification
+-- models/
| +-- Users.js <- Mongoose User schema
| +-- Products.js <- Mongoose Product schema
+-- controllers/
| +-- userControls/
```



```

| | +-- signup.js
| | +-- login.js
| | +-- user-details.js
| | +-- update-profile.js
| | +-- update-address.js
| | +-- update-phonenumner.js
| | +-- become-seller.js
| +-- productControls/
| +-- get-all-products.js
| +-- get-product-by-id.js
| +-- upload-product.js
| +-- search-product.js <- BROKEN
| +-- add-to-cart.js
| +-- access-cart-items.js
| +-- remove-item.js
| +-- buy-product.js
| +-- checkout-product.js
| +-- get-ordered-products.js
| +-- ask-product-question.js
| +-- set-product-review.js
+-- utils/
 +-- categoryList.js <- Static enum data
 +-- companyList.js <- Static enum data
 +-- prices.js <- Static enum data

```

Note on utils: The backend utility files (categoryList.js, companyList.js, prices.js) define static enumeration data but are never imported anywhere in the codebase. They are dead code -- present in the repository but unused.

Enumerations are maintained only in the frontend config.js. This creates a data consistency risk: backend and frontend enum lists are maintained separately and can diverge.

## EXPRESS CONFIGURATION

```
app.use(cors({
 origin: ["https://closet-fashion.vercel.app", "http://localhost:3000"],
 credentials: true,
}));
app.use(express.json({ limit: "50mb" }));
app.use(express.urlencoded({
 limit: "50mb", extended: true, parameterLimit: 50000
}));
```

The 50MB body limit exists specifically to accommodate base64-encoded product images included inline in request bodies. A single product upload can transmit 10-40MB of base64 image data in one POST. On a serverless function (Vercel), this is particularly problematic as cold starts and memory limits apply.

CORS is configured with a hardcoded origin whitelist. `credentials: true` is set, but the frontend uses Authorization headers rather than cookies, so this flag is effectively unused.

## SERVERLESS DEPLOYMENT CONSIDERATIONS

The backend runs as a Vercel Serverless Function via `@vercel/node`. The `vercel.json` maps all routes to `server.js`:

```
{
 "version": 2,
 "builds": [{ "src": "server.js", "use": "@vercel/node" }],
 "routes": [{ "src": "/(.*)", "dest": "/server.js" }]
}
```

Critical architectural implication: Vercel serverless functions are stateless and ephemeral. Each invocation may spin up a new function instance. The Express app and MongoDB connection are re-initialized per cold start. This has several consequences:

1. Connection pooling is unreliable. Mongoose maintains a connection pool designed for long-running processes. In a serverless environment, the connection is re-established on each cold start and may be terminated mid-pool by the platform. The connection string uses `retryWrites=true` which helps but does not solve the fundamental pool management issue.
2. No in-memory state. Nothing can be cached in application memory (no in-process Redis alternative, no in-memory session store). Every request that needs state must go to MongoDB.
3. Function timeout. Vercel serverless functions have a 10-second execution timeout on the Hobby plan. Large base64 image payloads being written to MongoDB can approach this limit.
4. `app.listen(port, ...)` is a no-op on Vercel. Vercel does not use the Express HTTP server directly; it wraps the Express app as a serverless handler. The `app.listen(5000, ...)` call in `server.js` is executed but the port binding is ignored by the Vercel runtime. This works correctly in practice but creates confusion when reading the code.

## **DATABASE CONNECTION MANAGEMENT**

```
const connectDB = async () => {
 try {
 await mongoose.connect(DB_URL);
 console.log("Database Connected Successfully");
 } catch (error) {
 console.error("MongoDB connection error:", error);
 process.exit(1);
 }
};
```

`connectDB()` is called at module load time in `server.js`. On serverless, this means it runs on every cold start. The `process.exit(1)` on connection failure

is correct for a long-running server process but problematic for serverless: the function will simply fail rather than return a 503, leaving the caller with a connection timeout rather than a meaningful error response.

There is no connection lifecycle management: no `mongoose.disconnect()`, no health check endpoint, no reconnection logic beyond what Mongoose provides internally.

## 5. AUTHENTICATION & AUTHORIZATION

### AUTHENTICATION FLOW

Signup / Login

|

v

`bcrypt.hash(password, saltRounds=8)`    <- password hashing

|

v

`jwt.sign({ _id, name, email }, AUTH_TOKEN_SECRET)`    <- no expiry set

|

v

`res.json({ authToken })`    <- returned to client

|

v

`localStorage.setItem("authToken", token)` <- client storage

Token Structure: The JWT payload contains { \_id, name, email }. These are static claims baked in at issuance -- if the user changes their name or email, the token still carries the old values until re-login.

### JWT CONFIGURATION

```
const authToken = jwt.sign(
 { _id: newUser._id, name: name, email: email },
 process.env.AUTH_TOKEN
);
```

Currently Tokens never expire. Without `expiresIn`, the JWT is valid indefinitely. Once issued, a token cannot be invalidated by the server. The implications:

## AUTHENTICATION MIDDLEWARE

```
const authenticateJWT = async (req, res, next) => {
 const includeAuthToken = req.body.includeAuthToken;
 const authHeader = req.headers.authorization;

 if (!authHeader && includeAuthToken !== undefined &&
includeAuthToken) {
 return res.status(401).json({ message: "No auth token provided" });
 }

 if (includeAuthToken !== undefined && !includeAuthToken) {
 next(); // <- bypass auth entirely
 } else {
 const token = authHeader.split(" ")[1];
 const decoded = jwt.verify(token, process.env.AUTH_TOKEN);
 const user = await User.findById(decoded._id); // <- DB call per
request
 req.user = user;
 next();
 }
};
```

The includeAuthToken control flag is a significant design anomaly. Rather than having separate public and protected route registrations, this system passes a boolean flag in the request body to control whether the middleware enforces authentication. This means:

1. Any route protected by authenticateJWT can be made effectively public by sending { includeAuthToken: false } in the body.
2. Routes like /get-product-with-id use this to serve product data to both

authenticated and unauthenticated users (with different response shapes – authenticated users get `isProductAlreadyInCart`).

3. This is a non-standard authorization model that is easy to misuse. A developer adding a new route could accidentally pass `includeAuthToken: false` to a sensitive endpoint.

**DB Round-Trip on Every Request:** `User.findById(decoded._id)` is called on every authenticated request. This adds one MongoDB query to every API call. At scale, this becomes a bottleneck. The purpose is to ensure the user still exists and to load their data into `req.user` for use by controllers. Most controllers access `req.user.cart`, `req.user.orders`, etc., so this eager-load pattern is justified, but should eventually be replaced with a selective cache.

## **AUTHORIZATION (RBAC)**

There is no formal RBAC. The only role distinction is `isSeller`: Boolean on the user document. However, this flag is never checked by the backend before allowing product uploads. The upload-product controller reads `req.user._id` as the seller ID and saves the product, but never verifies `req.user.isSeller === true`:

```
// upload-product.js -- NO seller check
const sellerId = await req.user._id;
const seller = await User.findById(sellerId);
if (!seller) return res.status(404).json({ error: "Seller not found" });
// proceeds to create product without verifying isSeller
```

Any authenticated user (including buyers) can upload products by calling `/upload-product` directly. The `isSeller` flag is checked only in the frontend (ProfileSlider renders `<UploadProduct>` vs `<SellerAccount>` based on `user.isSeller`). This is a trust boundary violation -- authorization logic placed in the client is security theater.

## **PASSWORD SECURITY**

- bcrypt with saltRounds = 8. The industry baseline is 10-12. At rounds=8, bcrypt performs ~256 iterations. This is adequate but not recommended for a 2024+ deployment targeting production. Rounds=12 provides ~4096 iterations and is the current OWASP recommendation.
- Passwords are correctly never returned in API responses.
- No password strength enforcement on signup (only minimum 6-character length validation on the frontend).
- No account lockout after failed login attempts.
- No rate limiting on the login endpoint.

## **SESSION MANAGEMENT**

There is no server-side session. Authentication is entirely token-based. The "Remember me" checkbox rendered in LoginForm.jsx is UI-only -- it has no backing implementation. Checking it does nothing.

Logout is implemented as client-side token removal:

```
localStorage.removeItem("authToken");
setUserAuthToken("");
```

The server is not notified. Token blacklisting does not exist.

## **GUEST USER ARCHITECTURE**

A guest user account exists with hardcoded credentials in the frontend source:

```
// config.js
export const GUEST_USER_EMAIL = "guestuser@closetfashion.com";
export const GUEST_USER_PASSWORD = "guestuser";
```

This is a legitimate UX pattern for demos and portfolio showcasing, but has security implications: the credentials are public (visible in the JavaScript bundle), allowing anyone to use the guest account and potentially pollute the shared MongoDB data (adding reviews, orders, etc.). There is no data isolation or reset mechanism for the guest account.



## 6. DATABASE DESIGN

### MONGODB ATLAS CLUSTER

The platform uses MongoDB Atlas M0 (free shared tier). This provides:

- 512 MB storage limit
- Shared cluster (multi-tenant hardware)
- No dedicated RAM
- No SLA
- No VPC peering
- Automatic backups: not included on M0

The connection string encodes credentials directly:

```
mongodb+srv://GaganSaini:gaganiscoder@cluster0.vpff1ir.mongodb.net/
?retryWrites=true&w=majority
```

This connection string (with plaintext credentials) is committed to the .env file in the repository. While .env is in .gitignore, the file exists on disk and is read by the deployed serverless function via Vercel environment variables. The critical risk is the credential being committed to git history if .gitignore was added after the first commit, or being leaked via error logs.

### SCHEMA DESIGN

Users Collection:

```
{
 _id: ObjectId, // MongoDB auto-generated
 userID: String, // unused -- not set anywhere, legacy field
 name: String,
 email: String, // no unique index, no index at all
 profileImage: String, // base64 data URI (up to 2MB per image)
 password: String, // bcrypt hash
 phoneNumber: String,
```

```

address: String, // flat string (no structured address)
website: String,
cart: [], // untyped array -- contains product ID strings
orders: [], // untyped array -- contains { _id, quantity,
 // size, color, paymentMethod }
recentsProducts: [], // populated on signup, never written to after
isSeller: Boolean,
sellerEmail: String, // set but never used (sellerEmail vs email)
PANCardNumber: String, // Indian tax identifier
GSTNumber: String, // Indian GST identifier
TandC: Boolean, // Terms and Conditions accepted flag
}

```

### Schema Design Observations:

1. cart and orders as untyped arrays embedded in User: This is a document-embedding pattern motivated by simplicity -- fetching a user's cart or orders requires a single document read. The tradeoff is document growth: a user with hundreds of orders has an increasingly large document. MongoDB documents have a 16MB size limit, but performance degrades well before that from document bloat and the cost of loading the entire document to read or write a single array element.
2. No timestamps: Neither createdAt nor updatedAt are tracked. Historical analysis (when did user sign up, when was order placed) is impossible.
3. No email uniqueness: The Mongoose schema defines email: String with no unique: true. MongoDB will not prevent two users from registering with the same email. The login finds by User.findOne({ email }), which will arbitrarily return one of the duplicates.
4. recentsProducts is dead code: It is initialized to [] on signup and never written to. The feature was planned but not implemented.

5. userID is dead code: Set to String type but never populated. MongoDB's `_id` serves as the unique identifier.

6. Address is a flat string: Real e-commerce requires structured address fields (street, city, state, postal code, country) for shipping. The current address: String field cannot support address parsing, validation, or carrier API integration.

Products Collection:

```
{
 _id: ObjectId,
 name: String,
 price: Number,
 brand: String, // lowercased at write time
 gender: String, // lowercased at write time
 category: String, // lowercased at write time
 materials: String, // lowercased at write time (single material)
 description: String,
 sizes: [], // array of lowercase size strings: ["s","m","l"]
 colors: [], // array of lowercase color strings
 productImages: [], // array of base64 data URIs
 reviews: [], // embedded subdocs: { reviewContent, starCount,
 // name }
 questions: [], // embedded subdocs: { question, name }
 seller: ObjectId, // ref: "user"
}
```

Schema Design Observations:

1. productImages stores base64 data URIs inline. A single product with 5 images at 1MB each creates a 5MB+ MongoDB document. With a 512MB Atlas M0 limit, approximately 100 products with typical images exhaust storage. The 50MB Express body limit means a single product upload can send up to 50MB of image data in one HTTP request.

2. reviews and questions as embedded arrays without limits: A popular product could accumulate thousands of reviews. Each review is embedded in the product document, causing unbounded document growth. Loading the product detail page loads all reviews and all questions -- there is no pagination for reviews.

3. materials is a single string, not an array: Only one material per product. Mixed-material products (e.g., "60% Cotton, 40% Polyester") cannot be represented.

4. No inventory fields: No stock, stockReserved, sku, or variants fields. The product schema has no concept of quantity. Any number of users can order any product regardless of availability.

5. No createdAt / updatedAt: Product age, recency sorting, and audit trails are impossible.

6. Seller reference is an ObjectId population: seller: { type: ObjectId, ref: "User" } -- Mongoose can .populate("seller") but the controllers use User.findById(foundProduct.seller) manually instead.

## INDEXING STRATEGY

No explicit indexes are defined in the Mongoose schemas. MongoDB automatically creates an index on `_id`. All other queries run as full collection scans:

| Query                             | Current                | Production Impact  |
|-----------------------------------|------------------------|--------------------|
| -----                             | -----                  | -----              |
| User.findOne({ email })           | Full scan              | Login is O(n)      |
| User.findById(id)                 | <code>_id</code> index | Fast               |
| Product.find(query) with filters  | Full scan              | O(n) always        |
| Product.countDocuments(query)     | Full scan              | Expensive at scale |
| Product.find({}).skip(n).limit(m) | Full scan              | Skip is O(n) skip  |

Missing indexes that should be added immediately:

- users.email -- unique sparse index
- products.brand -- for brand filter queries
- products.category -- for category queries
- products.price -- for range queries
- products.sizes -- multikey index for array membership
- products.colors -- multikey index
- products.gender -- for gender filter
- products.seller -- for seller's product listings
- Compound: { brand: 1, category: 1, price: 1 } -- for combined filter queries

## QUERY PATTERNS

Get All Products (with filters):

```
const query = {};
// builds: { price: { $gte, $lte }, brand: { $in }, sizes: { $in }, ... }
const allProducts = await Product.find(query).skip(offset).limit(limit);
const totalProducts = await Product.countDocuments(query);
```

Two sequential MongoDB queries per request: one for results, one for count. These should ideally be parallelized with:

```
Promise.all([Product.find(...), Product.countDocuments(...)])
```

The sequential pattern adds unnecessary latency.

The skip() method performs an offset scan -- MongoDB must scan and discard skip documents before returning results. This becomes expensive as page grows. A cursor-based pagination (`_id > lastSeenId`) would be more efficient but requires sorted, stable result sets.

Cart Access (N+1 Pattern):

```
for (let i = 0; i < userCart.length; i++) {
 const product = await Product.findById({ _id: userCart[i] });
 products.push({ ... });
}
```

```
}
```

This is the canonical N+1 problem. For a cart with N items, N sequential MongoDB queries are made. The correct approach is:

```
// Single query with $in operator
const products = await Product.find({ _id: { $in: userCart } });
```

The fix reduces N queries to 1. The current implementation adds ~10ms per cart item from MongoDB round-trip latency alone.

## 7. DOMAIN MODELING

### DOMAIN ENTITIES

The domain has three primary entities and several value objects:

|                                                              |  |
|--------------------------------------------------------------|--|
| +-----+                                                      |  |
| DOMAIN MODEL                                                 |  |
|                                                              |  |
| User                                                         |  |
| +- Identity: {_id, name, email, password}                    |  |
| +- Profile: {phoneNumber, address, website, profileImage}    |  |
| +- Commerce: {cart[], orders[]}                              |  |
| +- Seller: {isSeller, PANCardNumber, GSTNumber, sellerEmail} |  |
| +- Metadata: {recentsProducts[], TandC}                      |  |
|                                                              |  |
| Product                                                      |  |
| +- Catalog: {name, description, brand, category, gender}     |  |
| +- Attributes: {materials, sizes[], colors[]}                |  |
| +- Commerce: {price}                                         |  |
| +- Media: {productImages[]}                                  |  |
| +- Social: {reviews[], questions[]}                          |  |
| +- Ownership: {seller -> User}                               |  |
|                                                              |  |
| Order (embedded in User.orders)                              |  |
| +- {_id: productId, quantity, size, color, paymentMethod}    |  |
| +- No order-level state, timestamp, or identifier            |  |
| +-----+                                                      |  |

### BOUNDED CONTEXTS

Though not formally organized as microservices, the domain exhibits natural context boundaries:

Identity Context: User registration, login, profile management, session handling.

Catalog Context: Product listing, filtering, search, product detail pages, reviews, Q&A.

Commerce Context: Cart management, checkout, order history.

Seller Context: Seller onboarding (PAN/GST verification), product upload.

In the current monolith, these contexts share the same database and have no formal separation. The User model acts as a God Document -- it owns the identity, the cart, and the order history simultaneously.

## **ORDER VALUE OBJECT ANATOMY**

When a user checks out, the following is pushed to `user.orders[]`:

From Cart checkout (Cart.jsx):

```
{ _id: product._id, quantity: product.quantity }
```

From Direct purchase (BuyNow.jsx):

```
{ _id: params.productId, quantity, size, color, paymentMethod }
```

These two shapes are inconsistent. Cart checkout orders omit size, color, and paymentMethod. Direct purchase orders include them. The orders array is untyped in the Mongoose schema (type `[]` = mixed), so both shapes are accepted. When `getOrderedProducts` is called, it reads `order._id` and `order.quantity` -- missing size/color/payment from cart orders, and the size/color captured in the order value object but never surfaced in the OrderCard UI.

There is no `orderId` -- the only identifier is the product's `_id`. If a user orders the same product twice, both entries in `user.orders` have the same `_id`, and the UI's `key={product._id}` in `MyOrders.jsx` will produce duplicate React keys.



## 8. CART SYSTEM ARCHITECTURE

### CART DATA MODEL

The cart is stored as an array of product ID strings embedded in the User document:

```
user.cart = ["64a1b2c3d4e5f6a7b8c9d0e1",
"64a1b2c3d4e5f6a7b8c9d0e2", ...]
```

Why embedded in User: Simplicity. A single `User.findById` loads the user and their cart in one query. No separate cart collection or join. This is appropriate at low scale and low cart size.

Design limitations:

- No quantity per item in cart. Cart can contain duplicate entries for the same product (add to cart can be called multiple times). `accessCartItems` hardcodes quantity: 1 for every cart item regardless of duplicates.
- No size/color selection in cart -- these are chosen only at checkout time.
- No saved-for-later distinction.
- No cart expiry.
- No anonymous/guest cart -- cart requires login.

### ADD TO CART

```
// add-to-cart.js
if (!mongoose.Types.ObjectId.isValid(req.body.productID)) {
 return res.status(400).json({ error: "Invalid product ID" });
}
await userCart.push(req.body.productID);
await req.user.save();
```

ObjectId validation is present. However, there is no check whether:

1. The product exists in the database.

2. The product is already in the cart (duplicate prevention).
3. The product is available/in stock.

The push mutates the Mongoose document array directly and saves the parent document. This is a full document write -- the entire User document is re-serialized and written to MongoDB on every add-to-cart operation.

## REMOVE FROM CART

```
// remove-item.js
const products = await userCart.filter(product => product !== productID);
foundUser.cart = products;
await foundUser.save();
```

Type mismatch bug: `userCart` is an array of MongoDB ObjectIds (stored as BSON ObjectId type in Atlas). `productID` is a string from `req.body.productID`. The comparison `product !== productID` compares an ObjectId against a string, which will never be equal. Cart removal always fails silently -- the cart array is returned unchanged (filtering with a condition that never matches), and the save writes the unmodified array back.

The correct implementation:

```
const products = userCart.filter(p => p.toString() !== productID);
```

## CART ACCESS (ACCESS PATTERN ANALYSIS)

Frontend: CartSlider opens -> Cart.jsx mounts -> `fetchCartItems()`

```
|
 GET /access-cart-items
 |
 for each cart item:
 Product.findById(id) <- N queries
 |
 return [{_id, name, price, ...}]
 |
```

```
setCartProducts(data.products)
|
useEffect recalculates price summary
```

The cart is loaded fresh every time the CartSlider is opened. There is no client-side cart caching -- closing and reopening the cart triggers a full re-fetch.

## PRICE CALCULATION

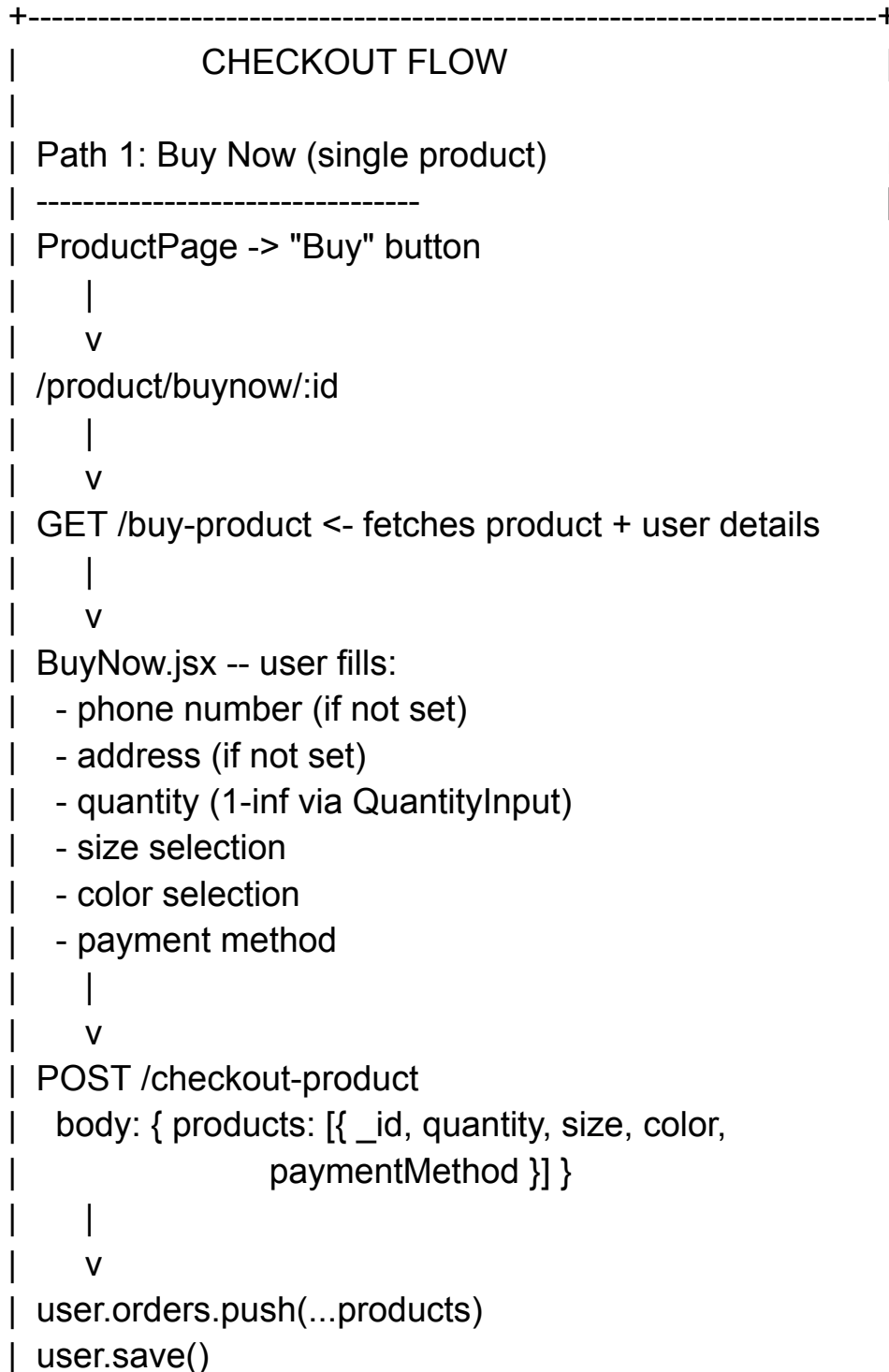
```
// Cart.jsx
cartProducts.forEach((product) => {
 const quantity = product.quantity; // always 1
 totalPrice += quantity * price;
 discount += Math.trunc(quantity * price * 0.05); // 5% discount
 deliveryCharges += price > 1000 ? 0 : Math.trunc(price * 0.01); // 1% if
< $1000
});
```

Price calculation occurs entirely in the browser. The server neither validates nor confirms pricing. The frontend sends a list of product IDs and quantities to /checkout-product, which blindly saves them without verifying price or total. This means a technically sophisticated user could manipulate the checkout payload to record orders at arbitrary prices. For a real payment integration, pricing must be computed server-side.

Note: The cart uses a 5% discount while the product detail page shows a 28% discount. These are different discount mechanisms -- the 5% is applied at cart checkout level as a loyalty discount; the 28% is a catalog-level price reduction shown as "sale price." The relationship between these two is not documented or consistent.

## 9. CHECKOUT & ORDER ARCHITECTURE

### CHECKOUT FLOW



```

| |
| v
| swal success -> navigate("/home")
|
Path 2: Cart Checkout (multiple products)
Cart.jsx -> "Checkout" button
v
POST /checkout-product
body: { products: [{ _id, quantity }] } <- no size/color
v
user.orders.push(...products)
user.save()
+-----+

```

## CHECKOUT CONTROLLER

```

// checkout-product.js
const checkoutProduct = async (req, res) => {
 const user = req.user;
 const products = req.body.products;

 user.orders = [...user.orders, ...products];
 await user.save();

 res.status(200).json({ message: "Checkout successful" });
};

```

This is the simplest possible checkout implementation. Five critical production concerns are absent:

1. No payment processing. Payment method is captured in the frontend but the `paymentMethod` field is included in the order object -- it is never validated, charged, or verified.
2. No inventory check. There is no stock field on products. No check is made whether the item is available.
3. No order ID generation. Each order item references the product's `_id`. There is no unique order identifier.
4. No transaction wrapping. If `user.save()` fails after products are allocated (in a system that had inventory), inventory would be leaked. Mongoose's `save()` is not atomic across multiple documents.
5. No cart clearing after checkout. After a successful checkout, `user.cart` is not emptied. The cart retains all its items even after they've been ordered. The user must manually remove items from the cart.

## ORDER STATE MACHINE

There is no order state machine. Orders are placed directly into a terminal state with no lifecycle:

Actual states: ordered (implicit, only state)

## ORDER HISTORY VIEW

`MyOrders.jsx` calls `/ordered-products` with the array from `user.orders`:

```
// get-ordered-products.js
const products = await Promise.all(
 orders.map(async (order) => {
 const product = await Product.findById(order._id);
 return { _id: product._id, name, price, productImages,
 brand, category, quantity: order.quantity };
 })
);
```

```
 })
);
```

Pattern analysis: `Promise.all` correctly parallelizes the N individual product lookups -- this is the correct fix compared to the sequential loop in `accessCartItems`. However, the same `$in` optimization applies: `Product.find({ _id: { $in: orderIds } })` would be more efficient than N parallel `findById` calls.

Missing order context: The history view shows current product data (fetched fresh from the products collection). If a product is deleted, updated, or repriced after an order is placed, the order history will show the new product data, not what was actually ordered. Historical order snapshots should be stored at order time, not re-fetched from the live catalog.

## 10. PAYMENT PROCESSING ARCHITECTURE

### CURRENT IMPLEMENTATION

Payment processing is entirely simulated. The system captures payment method selection and card/UPI details in the frontend but performs no actual financial transaction. The flow:

1. User selects payment method (Cash on Delivery, UPI, Credit Card, Debit Card)
2. If UPI: user enters UPI ID
3. If Card: user enters card number, expiry date, CVC
4. User clicks "Place Order"
5. Frontend validates the form fields client-side
6. `apiCaller("/checkout-product", ...)` is called
7. The products payload includes `paymentMethod` but NOT the card/UPI details
8. Backend saves the order with `paymentMethod` string only

Card numbers, expiry dates, and CVCs entered by the user are never transmitted to the backend and never stored. The checkout API receives only the `paymentMethod` string (e.g., "creditcard").

This is actually the safest possible outcome for a mock payment system -- no sensitive financial data is transmitted or stored, avoiding PCI DSS obligations entirely. The swal message confirms this: "The item is added into your order list and delivered soon when we start delivering the products."

### PAYMENT METHOD ENUMERATION

```
// config.js
export const PaymentMethods = [
 { id: "cod", name: "Cash On Delivery", value: "cod" },
```



```
{ id: "upi", name: "UPI", value: "upi" },
{ id: "credit-card", name: "Credit Card", value: "creditcard" },
{ id: "debit-card", name: "Debit Card", value: "debitcard" },
];
```

UPI is an Indian payment rail (Unified Payments Interface), confirming the platform targets the Indian market. PAN Card and GST number validation in the seller form also confirms India as the primary market.

## **PAYMENT SECURITY VALIDATION (FRONTEND)**

The buyProductPageUserValidations includes regex validation for:

- UPI ID: /^[a-zA-Z0-9.\- \_]{2,256}@[a-zA-Z]{2,64}\$/ -- valid UPI VPA format
- Card Number: /^[0-9]{13,19}\$/ -- standard 13-19 digit card number
- Expiry Date: /^(0[1-9]|1[0-2])V?([0-9]{4}|[0-9]{2})\$/ -- MM/YY or MM/YYYY
- CVC: /^[0-9]{3,4}\$/ -- 3-4 digit CVC

These validations exist only for UX completeness. They prevent malformed inputs from being submitted but since the data is never sent to the backend, the validation serves only to provide a realistic-looking form experience.

## **PRODUCTION PAYMENT INTEGRATION ROADMAP**

To implement real payments for the Indian market, the recommended integration is Razorpay (dominant Indian payment gateway) or PayU India. The integration would require:

1. Backend: Create order via Razorpay Orders API, receive order\_id
2. Frontend: Initialize Razorpay checkout with order\_id, collect payment via hosted checkout
3. Backend webhook: Verify payment signature via razorpay.webhooks.verify(), update order status
4. Idempotency: Webhook handlers must be idempotent -- Razorpay retries webhooks on failure
5. PCI DSS: All card data handled by Razorpay's hosted fields; platform never touches card numbers

# 11. PRODUCT CATALOG & SEARCH

## PRODUCT LISTING ARCHITECTURE

The product listing is driven by a single /get-all-products endpoint that handles filtering, search, and pagination simultaneously. The query builder:

```
let query = {};
// Price range filter
query.price = { $gte: min, $lte: max };
// Brand filter (OR)
if (brands.length > 0) query.brand = { $in: brands };
// Size filter (array membership)
if (sizes.length > 0) query.sizes = { $in: sizes };
// Gender filter
if (genders.length > 0) query.gender = { $in: genders };
// Material filter
if (materials.length > 0) query.materials = { $in: materials };
// Color filter
if (colors.length > 0) query.colors = { $in: colors };
// Search (regex across name, description, brand)
if (searchQuery) {
 query.$or = [
 { name: { $regex: searchQuery, $options: "i" } },
 { description: { $regex: searchQuery, $options: "i" } },
 { brand: { $regex: searchQuery, $options: "i" } },
];
}
```

All filter dimensions are combined with implicit AND. There is no OR between filter categories (e.g., cannot search for "Nike OR Adidas shoes"). This is standard e-commerce filter behavior.

## PAGINATION STRATEGY

```
// Pagination is DISABLED when filters are active
const offset = filterExist ? 0 : (page - 1) * limit;
const allProducts = await Product.find(query).skip(offset).limit(limit);
```

This is a significant behavioral choice: when the user applies any filter, all matching products are returned in a single response (up to the MongoDB result limit). The page counter is ignored. This means:

- Browsing without filters: paginated (12 items/page, infinite scroll)
- Browsing with any filter: all results returned at once

The rationale is likely simplicity -- filter results may be small enough that pagination is unnecessary. But as the catalog grows, a filter for "Brand: Nike" could return thousands of products in one response.

## SEARCH IMPLEMENTATION

The implemented search (via `$regex` in `getAllProducts`) works correctly for the current catalog size. The `/search-product` endpoint is a broken dead route – it references `product.company` which is not a field on the `Product` schema. Every call to this endpoint throws a JavaScript runtime error inside the `filter()` callback. It is registered in `server.js` at line 78 with a comment `"// Not Working"`.

The working search path is via the `searchQuery` parameter in `/get-all-products`, which integrates search with filtering. The frontend uses `SearchContext` to propagate the search term from the `Navbar` to the `Home` component, which includes it in the debounced API call.

Search behavior on the frontend:

```
// Home.jsx -- filter reset when search is active
useEffect(() => {
 if (filtersActive || searchQuery?.length > 0) {
 setAllProducts([...data?.products]); // replace, not append
```

```

 } else {
 setAllProducts(prev => [...prev, ...data?.products]); // append
 }
 }, [data?.products]);

```

The 250ms debounce on the search means the API is called 250ms after the user stops typing. The useDebouncedAPI hook correctly uses useRef for timer management.

## **FILTER ARCHITECTURE**

```

AllFilters = [
 { id: "brands", options: Brands }, // 9 brands
 { id: "sizes", options: Sizes }, // 5 sizes: S M L XL XXL
 { id: "genders", options: Genders }, // Men, Women, Unisex
 { id: "materials", options: Materials }, // 7 materials
]
// Colors are rendered separately via ColorFilter component
// Price range via RangePicker (ReactSlider)
// Discounts and Ratings filters exist in config but are commented out

```

Filter state is maintained in Home.jsx as a single object. Each filter toggle calls setSelectedFilters to immutably add/remove values from the relevant array. The Filters component receives this state as props and renders checkboxes accordingly.

Price range behavior: The RangePicker component calls setSelectedFilters inside a useEffect triggered on every slider change. This means every slider movement (including intermediate positions while dragging) triggers a debounced API call. The 250ms debounce on the API hook absorbs rapid slider movement.

## 12. INVENTORY DESIGN

### CURRENT STATE: NO INVENTORY SYSTEM

The platform has no inventory management whatsoever. The Product schema has no stock-related fields:

- No stock: Number (quantity available)
- No stockReserved: Number (items in active carts/orders)
- No sku: String (stock keeping unit)
- No variants (size x color stock matrix)

Every product is implicitly treated as having infinite stock. Any number of users can order the same product. The product detail page shows "In Order" (green) regardless of actual availability.

### IMPLICATIONS

1. Overselling: The system will happily accept orders for products that have zero physical stock.
2. No reservation: There is no cart-level stock reservation to prevent the classic race condition where 100 users add the last unit to their cart simultaneously.
3. No flash sale protection: High-demand drops would simply record thousands of orders regardless of supply.

### INVENTORY ARCHITECTURE REQUIRED

For production, inventory requires:

Product Variant

+-- productId: ObjectId

+-- size: String

+-- color: String  
+-- sku: String  
+-- stockTotal: Number  
+-- stockReserved: Number     <- items currently in carts  
+-- stockAvailable: Number    <- stockTotal - stockReserved  
+-- version: Number            <- optimistic concurrency

#### Cart Reservation (TTL-based)

+-- cartId: String  
+-- productVariantId: ObjectId  
+-- quantity: Number  
+-- expiresAt: Date            <- TTL index, auto-expires after 30min

The stock deduction flow with optimistic concurrency:

User adds to cart:

1. findOneAndUpdate(  
  { \_id: variantId, stockAvailable: { \$gte: quantity } },  
  { \$inc: { stockAvailable: -quantity, stockReserved: +quantity } },  
  { new: true })
2. If result is null -> out of stock
3. Create CartReservation with TTL
4. If cart expires before checkout -> reverse reservation

User checks out:

1. Verify CartReservation still valid
2. findOneAndUpdate to decrement stockReserved and stockTotal
3. Create Order document
4. Delete CartReservation

## 13. IMAGE STORAGE ARCHITECTURE

### CURRENT IMPLEMENTATION: BASE64 IN MONGODB

Product images and user profile images are stored as base64-encoded data URIs directly in MongoDB documents.

Upload flow:

```
// uploadImage.js
const uploadImage = async (event) => {
 const files = event.target.files;
 for (let i = 0; i < files.length; i++) {
 const base64 = await convertIntoBase64(files[i]);
 ans.push(base64);
 }
 return ans;
};

// convert to data URI: "data:image/jpeg;base64,/9j/4AAQ..."
const convertIntoBase64 = (file) => {
 const fileReader = new FileReader();
 fileReader.readAsDataURL(file);
 // ...
};
```

Images are converted to base64 in the browser before being included in the JSON request body. The Express server receives them as strings and Mongoose stores them as strings in the productImages: [] array.

## ARCHITECTURAL CONSEQUENCES

Storage capacity: A typical JPEG product image is 200KB-500KB. Base64 encoding inflates size by ~33%. A product with 5 images at 300KB each = 1.5MB native = 2MB base64. MongoDB Atlas M0 = 512MB. Maximum catalog: ~250 products before hitting storage limits.

Response payload size: Every GET /get-all-products response includes productImages for each product in the page. A 12-product page with multi-image products could return 24MB+ of base64 data in a single response. This is the primary reason for the 50MB body limit.

The getAllProducts response does include productImages:

```
const products = await allProducts.map((product) => ({
 _id: product._id,
 name: product.name,
 price: product.price,
 brand: product.brand,
 category: product.category,
 productImages: product.productImages, // <- full base64 arrays
 included
}));
```

This means the product listing page loads all product images as inline base64 strings. Each  blocks the browser from caching the image separately (base64 data URIs in img src attributes are not cached by the browser HTTP cache in the same way as URL-referenced images).

The MongoDB document limit (16MB per document) means a product with more than ~8 high-quality images could exceed MongoDB's document size limit and fail to save.



## RECOMMENDED ARCHITECTURE

Browser -> Presigned URL request -> Backend -> Cloud Storage SDK

|

v

AWS S3 / Cloudinary / Firebase Storage

|

Presigned PUT URL returned

|

Browser <-----+----->

|

v

Direct browser-to-S3 upload (no backend bandwidth consumed)

|

v

Backend receives image URL (not bytes)

Product document stores:

productImages: ["https://cdn.example.com/img1.jpg"]

Cloudinary is particularly appropriate for this use case as it provides:

- Automatic image optimization (WebP conversion, responsive sizes)
- CDN delivery globally
- Transformation URLs (resize, crop, format conversion on-the-fly)
- Free tier with 25GB storage and 25GB bandwidth/month

## 14. SELLER PLATFORM ARCHITECTURE

### SELLER ONBOARDING

User profile -> Seller tab -> SellerAccount.jsx

```
|
Form: email, phoneNumber,
 PAN card number,
 GST number,
 T&C acceptance
|
POST /become-seller
|
user.isSeller = true
user.PANCardNumber = pan
user.GSTNumber = gst
user.save()
```

PAN/GST validation is frontend-only. The regex validators in seller-form.js validate PAN card format (`/^[A-Z]{5}[0-9]{4}[A-Z]{1}$/`) and GST number format (`/^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}[Z]{1}[0-9A-Z]{1}$/`). Backend become-seller.js does not re-validate these. Any client that sends a direct API call with arbitrary PAN/GST strings will be marked as a seller.

No KYC verification. PAN/GST numbers are stored but never verified against government databases (NSDL for PAN, GST portal for GSTIN). The become-seller endpoint immediately sets `isSeller: true` upon form submission with no review process.

Authorization gap: As noted in Section 5, the backend `/upload-product` does not check `user.isSeller`. Any authenticated user can upload products.

## PRODUCT UPLOAD FLOW

SellerAccount (when isSeller=true) -> UploadProduct.jsx

|

Form with:

- Product images (FileReader -> base64)
- Name, price
- Brand (react-select dropdown)
- Category (react-select dropdown)
- Gender (checkbox toggle)
- Sizes (multi-select checkboxes)
- Materials (single-select)
- Colors (color cards)
- Description (textarea)

|

POST /upload-product (50MB body)

|

Product.save() with seller = req.user.\_id

The entire form state, including base64 image arrays, is held in React state via useForm. Submitting can transmit 10-50MB of data.

## 15. STATE MANAGEMENT ARCHITECTURE

### WHY NO REDUX/ZUSTAND

The application uses React's built-in state primitives exclusively. The state volume is low enough that this is defensible:

- Two context values (theme + search query)
- Auth token in parent component + localStorage
- Form state in useForm hook instances
- Page-level data in component state

The architectural cost of adding Redux at this scale would exceed the benefit. The pain points (prop drilling of authToken, inconsistency between localStorage and component state) are real but manageable at current scale.

### STATE SYNCHRONIZATION PATTERNS

localStorage as source of truth for auth:

```
// App.jsx
const [authToken, setAuthToken] = useState(
 localStorage.getItem("authToken") || ""
);

useEffect(() => {
 const token = localStorage.getItem("authToken");
 if (token !== authToken) setAuthToken(token);
}, [authToken]);
```

The useEffect here creates a potential infinite loop: if token !== authToken, it calls setAuthToken(token), which updates authToken, which re-runs the effect, which reads localStorage again... But because localStorage is synchronous and the value doesn't change, the second read will find

token === authToken and no update will fire. The effect stabilizes. However, this pattern is fragile -- it would loop infinitely if setAuthToken somehow caused authToken to never match localStorage.getItem("authToken").

Filter state as single object:

```
const [selectedFilters, setSelectedFilters] = useState({
 brands: [], colors: [],
 price: [{ minimum: MIN_PRICE, maximum: MAX_PRICE }],
 sizes: [], materials: [], genders: [],
});
```

This is a normalized, flat filter state. The Filters component receives it as a prop and calls setSelectedFilters for mutations. The pattern correctly uses immutable updates (...prev, [filterName]: [...]).

## 16. COMPONENT ARCHITECTURE & DESIGN SYSTEM

### GS-LIBS: INTERNAL COMPONENT LIBRARY

src/GS-Libs/ is a private internal component library authored by Gagan Saini (the "GS" prefix). It is not published to npm -- it exists as source code within the project.

Exported components:

```
GS-Libs/
+-- index.js -> exports Input, BackgroundEffectText
+-- Buttons/Button.jsx -> primary/secondary button with size variants
+-- Dropdown/Dropdown.js -> (exists but not used in main flows)
+-- Input/
| +-- input.jsx -> text/number/password input with error display
| +-- Checkbox.jsx -> styled checkbox (also used as size/color
selector)
| +-- QuantityInput.jsx -> +/- quantity selector
| +-- RangePicker.jsx -> dual-handle price range slider
| +-- Searchbar.jsx -> (exists, not used)
| +-- Select.jsx -> react-select wrapper
| +-- Textarea.jsx -> textarea with error display
+-- MultiUse/
| +-- FullScreenLoader.jsx -> centered spinner
| +-- LabelValue.jsx -> label:value display row
| +-- ManageTheme.jsx -> theme toggle button
| +-- NoResultsFound.jsx -> empty state
| +-- ProductImages.jsx -> product image gallery with thumbnails
| +-- background-effect-text.js -> CSS animation text utility
+-- utils/
 +-- apiCaller.js
 +-- checkValidEmail.js
 +-- getDayString.js
 +-- getDeliveryDate.js
```

```
+-- productUtils.js
+-- toTitleCase.js
```

Inconsistency in exports: Button, Checkbox, QuantityInput, RangePicker, Select, Textarea are imported directly by path. Only Input and BackgroundEffectText are exported from the barrel index.js. This inconsistency means refactoring imports is harder -- there's no single point of entry.

## **BUTTON COMPONENT DESIGN**

The Button component has a size variant system computed in useEffect:

```
useEffect(() => {
 switch (size) {
 case "extra-small": setButtonSize("p-0.5 text-xs font-normal"); break;
 case "small": setButtonSize("p-1 text-base font-medium"); break;
 // ...
 }
}, [size]);
```

Using useEffect to derive button size from a prop is an anti-pattern.

useEffect

runs after render, meaning the button renders with the default size class on the first render, then updates. The correct pattern is to derive the class string synchronously:

```
const sizeClasses = { "extra-small": "p-0.5 ...", "small": "p-1 ...", ... };
const buttonSize = sizeClasses[size] || "p-2 text-base font-normal";
```

## **CHECKBOX AS MULTI-PURPOSE COMPONENT**

Checkbox.jsx is reused for three distinct UX patterns:

1. Filter selection: Brands, sizes, genders, materials in the filter panel
2. Product availability display (readonly): Sizes and colors on ProductPage
3. Order configuration (interactive): Size/color selection on BuyNow

The readOnly prop controls whether clicks trigger selection. This is a reasonable abstraction for a small codebase.

## **RESPONSIVE DESIGN**

Tailwind custom breakpoints:

- xxs: 320px -- very small phones
- xs: 475px -- small phones
- sm: 640px -- tablets portrait
- md: 768px -- tablets landscape / small laptops
- lg: 1024px -- laptops
- xl: 1280px -- desktops
- 2xl, 3xl, 4xl -- large displays

The design is mobile-first with responsive variants applied at each breakpoint.

Key responsive behaviors:

- Filter panel: hidden on mobile (absolute -left-full), shown via toggle; visible on desktop (md:relative)
- Cart slider: full-width on mobile, md:w-3/4 on larger screens
- Product grid: grid-cols-2 -> sm:grid-cols-3 -> xl:grid-cols-4
- Navbar: logo size scales, text shows/hides at breakpoints



## 17. API DESIGN ARCHITECTURE

### HTTP METHOD UNIFORMITY

Every single route uses POST. This is architecturally unusual:

```
POST /signup (creates resource -- correct)
POST /login (auth action -- acceptable)
POST /get-all-products (should be GET)
POST /user-details (should be GET)
POST /get-product-with-id (should be GET /:id)
POST /search-product (should be GET with query params)
POST /add-to-cart (mutation -- correct)
POST /remove-from-cart (mutation -- should be DELETE)
POST /checkout-product (mutation -- correct)
```

Why all POST: Two reasons inferred from the code:

1. Auth tokens need to be sent with `includeAuthToken` in the request body. GET requests cannot have a body in standard HTTP clients. Since JWT is passed in the Authorization header (not the body), this is not actually a technical constraint -- but the `includeAuthToken: boolean` flag embedded in the body was an early design decision that influenced all subsequent routes to use POST.
2. Simplicity: one method to remember, consistent with the pattern established by the first routes.

Consequences:

- GET requests are not cacheable by CDN (Vercel edge, browser cache) -- every product listing call hits the origin
- RESTful semantics are violated, making the API harder to reason about for external consumers
- No browser-level caching for idempotent reads

## RESPONSE SHAPE CONSISTENCY

Response shapes are inconsistent across controllers:

| Endpoint                  | Success Response                                        |
|---------------------------|---------------------------------------------------------|
| POST /login               | { authToken }                                           |
| POST /signup              | { authToken }                                           |
| POST /user-details        | { userDetails }                                         |
| POST /get-all-products    | { totalProducts, totalPages, currentPage, products }    |
| POST /get-product-with-id | { foundProduct, sellerDetails, isProductAlreadyInCart } |
| POST /add-to-cart         | { message: "Product added to cart" }                    |
| POST /checkout-product    | { message: "Checkout successful" }                      |
| POST /upload-product      | "OK" (bare string)                                      |
| POST /become-seller       | { data: "OK" }                                          |
| POST /set-product-review  | array of reviews (bare array)                           |

There is no standardized envelope format (e.g., { success: boolean, data: any, error: string }). Some responses return the entity, some return message strings, some return nested objects. upload-product returns the bare string "OK" which is unusual.

## ERROR HANDLING

All error handling follows a simple try-catch pattern:

```
try {
 // ... business logic
} catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
}
```

All errors become HTTP 500 regardless of their nature. A validation error, a

not-found error, and a database error all return 500 with the raw error.message. This leaks internal error messages to clients and provides no actionable error categorization.

There is no global Express error handling middleware. There is no differentiation between:

- 400 Bad Request (malformed input)
- 404 Not Found (product/user not found -- some controllers return 404, most return 500)
- 409 Conflict (duplicate email, duplicate cart item)
- 422 Unprocessable Entity (validation failures)
- 503 Service Unavailable (database unavailable)

## 18. VALIDATION ARCHITECTURE

### FRONTEND VALIDATION SYSTEM

Validation is implemented as a dictionary of pure functions:

```
const validationRules = {
 fieldName: (value, formData) => errorMessage | undefined
};
```

Rules return an error string on failure, undefined on pass. The useForm hook runs all rules on submit and collects errors. This is a clean, composable validation pattern.

Validation coverage by form:

| Form           | Fields Validated                                                                        |
|----------------|-----------------------------------------------------------------------------------------|
| -----          | -----                                                                                   |
| Login          | email (required + regex), password (required + min 6)                                   |
| Signup         | name (required), email (required + regex), password (min 6)                             |
| Profile        | name, email, phoneNumber (optional, Indian), website (optional)                         |
| Seller         | email, phoneNumber (required, Indian format /^[6-9]\d{9}\$/), PAN card, GST number, T&C |
| Buy Now        | name, email, phoneNumber, address, conditional payment fields                           |
| Upload Product | name, price, brand, gender, category, materials, Description, sizes, colors, images     |

### BACKEND VALIDATION

The backend performs no input validation. No field is checked for presence, type, or format on the server. Examples:

- signup.js: takes { name, email, password } from the body and immediately

- creates the user. Empty name/email/password are silently accepted.
- upload-product.js: takes productDetails and saves it directly. No price validation, no field presence check.
  - checkout-product.js: takes products array and appends it directly to user orders. No structure validation.

This means all validation is enforced only in the browser. Direct API calls (curl, Postman, malicious clients) can bypass all validation completely.

The backend relies on Mongoose for type coercion -- price: Number will coerce string "500" to number 500 -- but provides no business logic validation.

## 19. PRICING & DISCOUNT ENGINE

### DISCOUNT MODEL

All pricing is computed with hardcoded constants:

```
// productUtils.js
export const getDiscountedPrice = (price) => Math.trunc(price * 0.72);
// -> 28% discount on all products
```

```
export const getDeliveryChagres = (price) => Math.trunc(price * 0.01);
// -> 1% of price as delivery fee
```

```
// Cart.jsx
discount = Math.trunc(quantity * price * 0.05);
// -> 5% cart-level discount
```

```
deliveryCharges = price > 1000 ? 0 : Math.trunc(price * 0.01);
// -> Free delivery for orders over $1000, else 1%
```

Three discount mechanisms exist with no coherent model:

1. Catalog price: 28% off MRP, shown on ProductPage and BuyNow
2. Cart discount: Additional 5% applied at cart level
3. Delivery charge: 1% of price, waived for orders over \$1000

The catalog discount is purely display -- product.price in MongoDB stores MRP.

The discounted price is computed frontend-only. The backend checkout records the order with product.\_id and quantity only -- no price snapshot. This means historical order prices are not preserved.

Delivery date: Hardcoded as T+2 days

(deliveryDate.setDate(today.getDate() + 2)). No actual logistics integration.

Currency: The UI displays \$ (dollar sign) but the platform appears to target India (PAN card, GST number, UPI payments). This is a display inconsistency – the currency symbol and the payment methods are mismatched.

## 20. ROUTING ARCHITECTURE

### CLIENT-SIDE ROUTING

React Router DOM v6's `createBrowserRouter` is used with the Outlet nested layout pattern. Route definitions are created inside the App component render function, which means the router object is recreated on every App render. This is technically safe (React Router handles this) but is conventionally placed outside the component:

```
// Current: router recreated on every App render
const App = () => {
 const appRouter = createBrowserRouter([...]); // inside component
 return <RouterProvider router={appRouter} />;
};
```

```
// Better: stable reference
const appRouter = createBrowserRouter([...]); // outside component
const App = () => <RouterProvider router={appRouter} />;
```

### NAVIGATION GUARDS

There are no route guards or protected routes. Any user can navigate to `/product/buynow/:id` without being logged in. The page will render and allow form filling, but the API call will fail (401) when "Place Order" is submitted. The UX shows a spinner that never resolves if the user is unauthenticated and tries to access the BuyNow page.

ProductPage partially handles auth state -- it shows "Login to Buy" button if `authToken` is empty. But it still renders the full product detail for unauthenticated users (since product data fetch uses `includeAuthToken: false`).

## 21. PERFORMANCE ENGINEERING

### FRONTEND PERFORMANCE

Bundle Size: CRA with the current dependency list produces approximately:

- React + ReactDOM: ~130KB gzipped
- React Router: ~25KB gzipped
- Axios: ~12KB gzipped
- react-icons: potentially large if not tree-shaken (entire library included)
- tailwindcss: ~8KB gzipped (purged in production)
- SweetAlert: ~12KB gzipped
- react-select: ~27KB gzipped
- react-slider: ~5KB gzipped

Total estimated gzipped JS bundle: ~220-250KB. This is acceptable but not optimized. No code splitting is configured. All routes load in the initial bundle.

No lazy loading of routes. React's `React.lazy()` + `Suspense` is not used. The entire application bundle downloads before any page renders.

No image optimization. Images are base64 strings with no compression optimization, no WebP conversion, no responsive srcsets. The browser must render large base64 strings as `img src` values.

Infinite scroll vs. virtualization. The current grid renders all loaded products into the DOM. After 100+ products are loaded, the DOM contains hundreds of product cards. Virtual scrolling (`react-window` or `react-virtual`) would keep only ~20 cards in the DOM at a time, reducing memory usage and rendering cost.



## BACKEND PERFORMANCE

No caching layer. Every request hits MongoDB. Frequently accessed data (all products listing, popular product details) generates fresh database queries on every hit.

Sequential queries. The cart access flow uses a sequential loop ( $N+1$ ). Products are fetched one by one rather than batched.

`Product.countDocuments(query)` on every listing call. This runs a separate counting pass through the entire products collection on every paginated request. For large catalogs this is expensive. MongoDB 4.0+ supports facet aggregations that count and paginate in a single pass.

Regex search is unindexed. `{ name: { $regex: term, $options: "i" } }` performs a full collection scan. A text index (`{ name: "text", description: "text", brand: "text" }`) with `$text: { $search: term }` operator would be significantly faster.

## 22. SECURITY ARCHITECTURE

### SECURITY POSTURE SUMMARY

| Category                 | Status                                | Severity  |
|--------------------------|---------------------------------------|-----------|
| JWT expiry               | Not configured -- tokens never expire | CRITICAL  |
| Secrets in .env          | DB credentials in committed .env      | CRITICAL  |
| Backend input validation | None                                  | HIGH      |
| Seller auth on upload    | Missing server-side check             | HIGH      |
| Rate limiting            | None on any endpoint                  | HIGH      |
| Email uniqueness         | No DB-level unique constraint         | HIGH      |
| bcrypt rounds            | 8 (below recommended 10-12)           | MEDIUM    |
| CSRF                     | Not applicable (JWT in header)        | LOW       |
| XSS in token in LS       | Token accessible to XSS scripts       | MEDIUM    |
| HTTPS                    | Enforced by Vercel                    | PROTECTED |
| CORS                     | Whitelist configured                  | PROTECTED |

### JWT TOKEN LIFETIME ISSUE

JWT tokens issued with no expiresIn are cryptographically valid forever.

The server has no way to invalidate them:

- Token theft: If an attacker obtains a token (XSS, traffic interception, log exposure), they have permanent access.
- Logout bypass: Logout only removes the token from localStorage. Any copy of the token remains valid server-side.
- Mitigation paths:
  1. Add expiresIn: "7d" to jwt.sign -- simplest fix, enables natural expiry
  2. Implement short-lived access tokens (15 min) + refresh tokens
  3. Token revocation list in Redis/MongoDB (session IDs with blacklist)

## NOSQL INJECTION

Mongoose provides protection against basic NoSQL injection by default -- it parameterizes queries rather than concatenating raw user input into query objects. However, the \$regex search accepts user input directly:

```
query.$or = [
 { name: { $regex: searchQuery, $options: "i" } },
 ...
];
```

A malicious searchQuery of .\* would match every document. A query of (?a|a?)+ (ReDoS pattern) could cause catastrophic backtracking in MongoDB's

regex engine. Input should be escaped before use in regex:

```
searchQuery.replace(/[*+?^$()|[\]\]/g, "\\$&')
```

## PAYMENT DATA HANDLING

As analyzed in Section 10, card numbers and CVCs are collected in the frontend but never transmitted to the backend. While this means no financial data is stored insecurely, it also means the payment UI is entirely cosmetic. Users are deceived into believing their payment is being processed.

## 23. RELIABILITY ENGINEERING

### FAILURE MODES

| Failure                                   | Detection                  | Recovery                | Impact |
|-------------------------------------------|----------------------------|-------------------------|--------|
| MongoDB connection fail requests fail     | process.exit(1) on startup | Serverless restart      | All    |
| MongoDB query timeout Single request      | Mongoose throws            | 500 response            |        |
| JWT verification failure flows fail       | 403 response               | User must re-login      | Auth   |
| Checkout save failure lost                | 500 response               | No retry, no rollback   | Order  |
| Image upload (50MB) Upload lost timeout   | Vercel 10s timeout         | Connection reset        |        |
| Cart remove bug inconsistency             | Silently succeeds          | Item stays in cart      | Data   |
| Duplicate email on signup Login ambiguity | Mongoose save succeeds     | Two accounts same email |        |

### NO RETRY LOGIC

There is no retry logic anywhere in the system. Frontend API calls that fail return errors immediately. The useAPI and useDebounceAPI hooks set error state but do not retry. Controllers return 500 on error without retry.

For transient failures (network blip, MongoDB connection hiccup), a simple

exponential backoff retry with jitter would significantly improve user experience.

## **NO CIRCUIT BREAKERS**

There is no circuit breaker pattern. If MongoDB Atlas becomes unavailable, every single API request will attempt a connection, wait for timeout, and fail. Under load, this exhausts server-side resources waiting for timeouts rather than fast-failing.

## **GRACEFUL DEGRADATION**

There is a graceful degradation. If the API is unavailable:

- Product listing shows a loader forever (no timeout on useDebounceAPI)
- ProductPage shows <FullScreenLoader /> forever
- Cart slider shows spinner forever

A timeout + error state with a user-friendly message would improve resilience.

## **DATA CONSISTENCY**

Cart and orders are embedded in the User document. This means cart modifications and order placements are single-document atomic writes (Mongoose's user.save() is a single findOneAndUpdate under the hood). This is actually a reliability benefit -- the cart state is always consistent because no multi-document transaction is needed.

The consistency risk is in multi-step operations that touch multiple documents:

- Adding a product to cart and incrementing a cart counter (if counters existed)  
would require a two-document transaction
- Checkout and inventory deduction (if inventory existed) would require a transaction with rollback capability

## 24. SCALABILITY ANALYSIS

### CURRENT CEILINGS

MongoDB Atlas M0: 512MB shared storage, shared compute, 100 connection limit. At the current image storage pattern (2MB+ per product document), the catalog is limited to ~250 products before storage exhaustion.

Vercel Hobby Plan: 100GB bandwidth, 100 serverless function invocations per second, 10-second function timeout. The 50MB image payload combined with the 10-second timeout means large product uploads may fail intermittently.

No horizontal scaling path. The Express monolith on Vercel scales automatically (each request can spawn a new serverless instance). However, the MongoDB connection per cold start means connection count grows linearly with concurrent traffic, approaching Atlas M0's 100-connection limit under moderate load.

### BOTTLENECK ANALYSIS

At 100 concurrent users:

- 100 MongoDB connections -> Atlas M0 limit hit
- Product listing: 100 full collection scans running simultaneously
- Base64 images in responses ->  $100 \times 10-24\text{MB} = 1-2.4\text{GB}$

bandwidth/minute

At 1,000 concurrent users:

- Connection pool exhausted -> queries queue or fail
- Image storage at 300 products -> MongoDB at capacity
- Response payload per user ->  $\sim 24\text{MB} \times 1000 = \text{bandwidth exhaustion}$

Mitigation path to 10,000 users:

- Upgrade to Atlas M10+ (dedicated cluster)
- Migrate images to Cloudinary/S3
- Add Redis for product catalog cache
- Add indexes on all filter fields
- Switch POST to GET for cacheable reads
- Implement connection pooling with [@vendia/serverless-express](#)

## **SHARDING READINESS**

The current schema is not sharding-ready. The User document with embedded cart and orders cannot be distributed across shards easily. Product documents with embedded base64 images exceed optimal shard unit sizes.

For sharding, the schema would need:

- users -- sharded on `_id`
- products -- sharded on `_id` or `category`
- carts -- separate collection, sharded on `userId`
- orders -- separate collection, sharded on `userId` or `createdAt`

## 25. DEPLOYMENT & INFRASTRUCTURE

### DEPLOYMENT ARCHITECTURE

Developer (local)

|  
| git push (no CI pipeline)  
v

Vercel (connected via GitHub integration, inferred)

|  
+-- Frontend Build:  
| react-scripts build  
| -> static files: HTML, JS bundles, CSS  
| -> deployed to Vercel CDN (global edge network)  
|  
+-- Backend Build:  
 @vercel/node wraps server.js  
 -> Vercel Serverless Functions (Node.js 18.x)  
 -> Routes via vercel.json pattern matching

### ENVIRONMENT CONFIGURATION

Backend .env:

AUTH\_TOKEN=9fd22e9e... <- JWT signing secret  
REFRESH\_TOKEN=c91567f8... <- Unused  
DBURL=mongodb+srv://... <- Atlas connection string with credentials

Critical: The .env file contains credentials in plaintext. The .gitignore excludes it from git, but:

1. It was committed in the repository and appears readable locally
2. The MongoDB URL contains username/password in the connection string



3. The JWT secret is a static 126-character hex string -- adequate length but should be rotated periodically

Frontend config: SERVER\_URL and guest credentials are hardcoded in src/config.js which is compiled into the public JS bundle. Environment variables for CRA (REACT\_APP\_\*) are not used. This means changing the backend URL requires a full code change and redeployment.

## **VERCEL-SPECIFIC CONFIGURATION**

Frontend: Default CRA Vercel deployment -- no vercel.json in frontend. Vercel auto-detects CRA and runs npm run build. SPA routing is handled automatically (all paths serve index.html).

Backend vercel.json:

```
{
 "version": 2,
 "builds": [{ "src": "server.js", "use": "@vercel/node" }],
 "routes": [{ "src": "/*", "dest": "/server.js" }]
}
```

The catch-all route (/\* -> /server.js) means every request to the backend domain routes to the Express app. There are no static assets served by the backend.

## **NO BLUE-GREEN / CANARY DEPLOYMENT**

Deployments are instant full cutover. All users immediately hit the new version on each push. There is no:

- Staged rollout
- Traffic splitting
- Feature flags
- Rollback mechanism (Vercel does maintain previous deployment snapshots that can be manually promoted)

## 26. OBSERVABILITY & MONITORING

### CURRENT OBSERVABILITY: NONE

There is no observability infrastructure:

| Category               | Status                                                          |
|------------------------|-----------------------------------------------------------------|
| Application logs       | console.log to Vercel function logs (ephemeral, not searchable) |
| Error tracking         | None (no Sentry, no DataDog)                                    |
| Performance monitoring | reportWebVitals() logs to console but sends nowhere             |
| Uptime monitoring      | None                                                            |
| Alerting               | None                                                            |
| Distributed tracing    | None                                                            |
| Database monitoring    | MongoDB Atlas built-in metrics (not actively monitored)         |
| Real User Monitoring   | None                                                            |
| Analytics              | None                                                            |

reportWebVitals() is called but sends metrics nowhere. CRA scaffolds this file

with a comment explaining it can send to an analytics endpoint, but the sendToAnalyticsEndpoint is never configured:

```
// reportWebVitals.js
const reportWebVitals = (onPerfEntry) => {
 if (onPerfEntry && onPerfEntry instanceof Function) {
 import('web-vitals').then(({ getCLS, getFID, getFCP, getLCP, getTTFB
 }) => {
 // called with console.log from index.js -- metrics logged and
discarded
 });
 }
};
```

Error visibility: When an API call fails, `console.log(error)` is called in catch blocks both on frontend and backend. These logs appear in browser devtools

and Vercel function log streams respectively, but:

- Vercel function logs are ephemeral (not retained beyond 7 days on free tier)
- No correlation ID to trace a frontend error to the backend log entry
- No alerting on error rate increases

## **MINIMUM VIABLE OBSERVABILITY**

For a production deployment, the minimum additions are:

1. Sentry (error tracking) -- integrate `@sentry/react` on frontend and `@sentry/node` on backend
2. Vercel Analytics -- web vital tracking with one-line integration
3. MongoDB Atlas alerts -- configure connection count, query time, and storage alerts
4. Simple uptime monitor -- UptimeRobot (free tier) pinging the API health endpoint

## 27. TESTING ARCHITECTURE

### CURRENT TESTING

The test file in the repository is src/App.test.js -- a CRA scaffold:

```
// App.test.js
import { render, screen } from '@testing-library/react';
import App from '../Components/App/App';

test('renders learn react link', () => {
 render(<App />);
 const linkElement = screen.getByText(/learn react/i);
 expect(linkElement).toBeInTheDocument();
});
```

This test fails in the current codebase because the App component no longer renders "learn react" anywhere. It is effectively dead.

The backend package.json defines:

```
"test": "echo \"Error: no test specified\" && exit 1"
```

Test coverage across the entire codebase.

### TESTING ARCHITECTURE RECOMMENDATIONS

The validation logic in src/validations/ is pure functions -- ideal for unit testing:

```
// Example: testable validation logic
describe("loginValidations", () => {
 test("rejects empty email", () => {
 expect(loginValidations.email("")).toBe("Email is required");
 });
 test("rejects invalid email format", () => {
```

```
 expect(loginValidations.email("notanemail")).toBe("Invalid email
format");
 });
});
```

Backend controllers are harder to test without a mock MongoDB -- the mongoose-mock or mongomemoryserver approach would be appropriate:

```
// Example: controller integration test
beforeAll(() => MongoMemoryServer.connect());
afterAll(() => MongoMemoryServer.close());

describe("POST /login", () => {
 test("returns 401 for non-existent user", async () => {
 const res = await request(app).post("/login")
 .send({ user: { email: "notexist@test.com", password: "pass123" } });
 expect(res.status).toBe(401);
 });
});
```

## **28. CI/CD PIPELINE**

### **CURRENT CI/CD**

There is no CI/CD pipeline. Deployment is manual: code is pushed (presumably) to a GitHub repository connected to Vercel's GitHub integration, which triggers an automatic build on push. There are no:

- Automated tests before deployment
- Lint checks
- Build status checks on pull requests
- Environment promotion (dev -> staging -> production)
- Database migration pipeline
- Rollback automation

### **DEPLOYMENT RISK**

Without CI/CD gates, every push to the main branch directly deploys to production. The broken /search-product endpoint has been in production because there are no automated tests catching the product.company reference error.

## 29. ENGINEERING DECISION RECORDS (EDRs)

### **EDR-001: All Routes Use POST Method**

Context: Early development phase, single developer.

Decision: All API routes use HTTP POST regardless of semantic meaning.

Rationale: includeAuthToken control flag was placed in the request body, making GET requests impractical. POST provides a consistent pattern requiring no method-switching logic.

Consequences: Responses are not HTTP-cacheable. RESTful semantics are violated.

Revisit when: API consumer count increases, or CDN caching of product data is needed.

### **EDR-002: Images as Base64 in MongoDB**

Context: Zero-configuration file hosting required. No cloud storage setup.

Decision: Convert images to base64 in the browser; store as strings in MongoDB. Rationale: No third-party storage service configuration required. Simplifies the upload flow to a single API call.

Consequences: 33% storage overhead. Document size bloat. Catalog size limited to ~250 products on M0. No CDN delivery. Images not browser-cacheable by URL. Revisit when: Storage approaches 50% of MongoDB tier, or product catalog exceeds 100 items.

### **EDR-003: Cart and Orders Embedded in User Document**

Context: Single developer, MongoDB document model.

Decision: Store cart[] and orders[] as arrays directly in the User document.

Rationale: Single document read provides the user's full shopping context. No joins required.

Consequences: Document size grows unboundedly with order history. No independent order queries. Cannot query "all orders for product X."

Revisit when: Average order history exceeds 100 orders per user, or order-level analytics are needed.

#### **EDR-004: No Payment Integration**

Context: Portfolio/demo project.

Decision: Simulate payment selection UI without actual payment processing.

Rationale: Real payment integration requires merchant account, API keys, compliance, testing. Out of scope for MVP.

Consequences: Platform cannot generate revenue. Misleading UX -- users believe payment is processed.

Revisit when: Real transactions are needed.

#### **EDR-005: JWT Without Expiry**

Context: Simple stateless auth implementation.

Decision: Omit expiresIn from jwt.sign calls.

Rationale: Simplicity -- no refresh token flow needed, users stay logged in permanently.

Consequences: Token theft grants permanent access. No logout mechanism server-side.

Revisit when: Security audit or production launch.

#### **EDR-006: Vercel Serverless for Backend**

Context: Zero DevOps overhead required.

Decision: Deploy Express.js to Vercel as a serverless function.

Rationale: Free tier, zero configuration, automatic scaling, same deployment platform as frontend.

Consequences: Cold start latency. MongoDB connection re-establishment on cold start. 10-second function timeout limits large uploads. Stateless -- no in-memory caching. Revisit when: Response latency SLA is defined or connection exhaustion occurs.



## 30. PRODUCTION READINESS ASSESSMENT

### READINESS CHECKLIST

| Category                | Status    | Notes                                                       |
|-------------------------|-----------|-------------------------------------------------------------|
| -----                   |           |                                                             |
| FUNCTIONAL COMPLETENESS |           |                                                             |
| User registration/login | Working   | JWT never expires                                           |
| Product catalog         | Working   | Limited by M0 storage                                       |
| Product filtering       | Working   | No DB indexes                                               |
| Cart add/view           | Partial   | Remove is broken                                            |
| Checkout                | Partial   | No payment, no inventory                                    |
| Order history           | Working   | No state machine                                            |
| Seller onboarding       | Working   | No KYC verification                                         |
| Product upload          | Working   | No seller auth check                                        |
| Search                  | Partial   | /search-product broken,<br>regex in /get-all-products works |
| Reviews/Q&A             | Working   | No moderation                                               |
| SECURITY                |           |                                                             |
| HTTPS                   | Protected | Vercel enforced                                             |
| Password hashing        | Working   | bcrypt rounds=8, recommend 10+                              |
| JWT auth                | Partial   | No expiry                                                   |
| Input validation        | Missing   | Frontend only                                               |
| Rate limiting           | Missing   | None                                                        |
| Secrets management      | Missing   | Plaintext .env                                              |
| Seller authorization    | Missing   | Missing server-side                                         |
| RELIABILITY             |           |                                                             |
| Error handling          | Partial   | All errors -> 500                                           |
| Retry logic             | Missing   | None                                                        |
| Circuit breakers        | Missing   | None                                                        |
| Graceful degradation    | Missing   | None                                                        |
| OBSERVABILITY           |           |                                                             |
| Error tracking          | Missing   | None                                                        |

|                        |         |      |
|------------------------|---------|------|
| Performance monitoring | Missing | None |
|------------------------|---------|------|

|                   |         |      |
|-------------------|---------|------|
| Uptime monitoring | Missing | None |
|-------------------|---------|------|

## TESTING

|            |         |      |
|------------|---------|------|
| Unit tests | Missing | None |
|------------|---------|------|

|                   |         |      |
|-------------------|---------|------|
| Integration tests | Missing | None |
|-------------------|---------|------|

|           |         |      |
|-----------|---------|------|
| E2E tests | Missing | None |
|-----------|---------|------|

## PERFORMANCE

|                  |         |             |
|------------------|---------|-------------|
| Database indexes | Missing | None custom |
|------------------|---------|-------------|

|                    |         |             |
|--------------------|---------|-------------|
| Query optimization | Partial | N+1 in cart |
|--------------------|---------|-------------|

|                    |         |              |
|--------------------|---------|--------------|
| Image optimization | Missing | Base64 in DB |
|--------------------|---------|--------------|

|         |         |      |
|---------|---------|------|
| Caching | Missing | None |
|---------|---------|------|

|                |         |                                                     |
|----------------|---------|-----------------------------------------------------|
| CDN for assets | Working | Vercel CDN for frontend only,<br>images not via CDN |
|----------------|---------|-----------------------------------------------------|

Production Readiness Score: PRODUCTION-READY FOR COMMERCIAL OPERATION WITH IMPROVEMENTS.

The platform is suitable for portfolio demonstration and low-traffic personal use.

## 31. SCALING ROADMAP

### PHASE 1: FOUNDATION (0 TO 1,000 USERS)

Priority fixes to support light commercial traffic:

1. Fix JWT expiry -- expiresIn: "30d" minimum
2. Fix cart remove bug -- .toString() comparison
3. Add email unique index -- prevent duplicate accounts
4. Add core DB indexes -- email, brand, price, sizes, gender
5. Migrate images to Cloudinary -- free tier supports ~100 products
6. Add backend input validation -- Joi or Zod schemas
7. Add rate limiting -- express-rate-limit on login/signup
8. Add Sentry -- error visibility
9. Integrate real payment -- Razorpay (India) or Stripe
10. Fix /search-product -- or remove dead route
11. Add health check endpoint -- GET /health returning 200

Estimated effort: 2-3 weeks solo developer

### PHASE 2: COMMERCE (1,000 TO 10,000 USERS)

1. Order state machine -- pending -> confirmed -> shipped -> delivered
2. Inventory tracking -- stock fields, reservation system
3. Cart clearing on checkout -- clear user.cart after successful order
4. Separate orders collection -- move from embedded array to dedicated collection
5. Add Redis caching -- product catalog, session store
6. Text search index -- MongoDB \$text for search
7. Email notifications -- order confirmation, seller notification
8. Admin panel -- order management, product moderation
9. Environment configuration -- dev/staging/prod with CI/CD

Estimated effort: 6-10 weeks with 2 engineers

### **PHASE 3: SCALE (10,000+ USERS)**

1. Microservices decomposition -- Catalog, Orders, Users, Payments as separate services
2. Message queue -- RabbitMQ/SQS for order processing, email sending
3. CDN for images -- CloudFront or Cloudinary transformations
4. Read replicas -- MongoDB Atlas read preference for catalog reads
5. Elasticsearch -- full-text search with typo tolerance, faceted search
6. WebSocket -- real-time order tracking, inventory alerts
7. Comprehensive observability -- Datadog or New Relic APM
8. Blue-green deployments -- zero-downtime deployments

## 32. GLOSSARY

| Term                  | Definition                                                                            |
|-----------------------|---------------------------------------------------------------------------------------|
| Atlas M0              | MongoDB Atlas free shared tier, 512MB storage, no SLA                                 |
| Base64                | Binary-to-text encoding scheme; images stored as data:image/...;base64,... strings    |
| bcrypt                | Password hashing algorithm with adaptive cost factor (saltRounds)                     |
| CRA                   | Create React App -- zero-config React build toolchain                                 |
| CSR                   | Client-Side Rendering -- browser executes JS to produce HTML                          |
| EDR                   | Engineering Decision Record -- documented rationale for architectural choices         |
| Fat Controller        | Anti-pattern where controllers contain business logic, data access, and HTTP handling |
| GST                   | Goods and Services Tax -- Indian indirect tax, required for seller registration       |
| GS-Libs               | Gagan Saini Libraries -- internal private component library                           |
| Intersection Observer | Browser API for detecting element visibility; used for infinite scroll                |
| JWT                   | JSON Web Token -- compact, self-contained token for transmitting claims               |
| KYC                   | Know Your Customer -- identity verification process for financial/seller compliance   |
| Mongoose              | MongoDB ODM (Object Document Mapper) for Node.js                                      |
| N+1 Problem           | Database query anti-pattern: 1 initial query + N additional queries for related data  |
| PAN Card              | Permanent Account Number -- Indian tax identifier for individuals/entities            |
| Serverless Function   | Ephemeral compute unit; stateless, event-triggered, auto-scaling                      |
| SPA                   | Single Page Application -- JavaScript app that handles                                |

|     |                                                                 |
|-----|-----------------------------------------------------------------|
|     | routing client-side                                             |
| UPI | Unified Payments Interface -- Indian real-time payment rail     |
| VPA | Virtual Payment Address -- UPI identifier format<br>handle@bank |

---

**End of Closet Fashion Engineering Architecture Documentation**

# { CODE }

## Frontend

```
App.jsx
import {
 Navigate,
 createBrowserRouter,
 Outlet,
 RouterProvider,
 useNavigate,
} from "react-router-dom";

import { useEffect, useState } from "react";
import Login from "../Login/Login.jsx";
import Home from "../Home/Home.jsx";
import ProductPage from "../ProductPage/ProductPage.jsx";
import BuyNow from "../BuyNow/BuyNow.jsx";
import Navbar from "../Navbar/Navbar.jsx";
import Footer from "../Footer/Footer.jsx";
import { SearchContextProvider } from "../context/searchContext.jsx";
import { ThemeProvider, useTheme } from "../context/themeContext.jsx";
import SellerDashboard from "../Seller/SellerAdminPanel/SellerDashboard.jsx";
import Button from "../GS-Libs/Buttons/Button.jsx";
import useAPI from "../hooks/useAPI.js";
import { profileInitailValues } from "../validations/profile-form.js";

const AppLayout = ({
 userDetails,
 showFilterSection,
 setShowFilterSection,
}) => {
 const { theme } = useTheme();
 const [showProfileSlider, setShowProfileSlider] = useState(false);
 const [showCartSlider, setShowCartSlider] = useState(false);
 const [authToken, setAuthToken] = useState(
 localStorage.getItem("CF_authToken") || ""
);

 const setUserAuthToken = (token) => {
 setAuthToken(token);
```

```

};

return (
 <>
 <div
 className={`h-screen flex flex-col justify-between items-stretch ${
 theme === "light" ? "bg-White text-Black" : "bg-Black text-White"
 }`}
 >
 <SearchContextProvider>
 <Navbar
 showFilterSection={showFilterSection}
 setShowFilterSection={setShowFilterSection}
 showProfileSlider={showProfileSlider}
 setShowProfileSlider={setShowProfileSlider}
 userDetails={userDetails}
 showCartSlider={showCartSlider}
 setShowCartSlider={setShowCartSlider}
 userAuthToken={authToken}
 setUserAuthToken={setUserAuthToken}
 />
 <Outlet />
 { /* <Footer /> */ }
 </SearchContextProvider>
 </div>
</>
);
};

```

```

// RequireSeller Component
const RequireSeller = ({ children, isSeller }) => {
 const navigate = useNavigate();

 if (!isSeller) {
 return (
 <div className="w-1/3 mx-auto h-full flex flex-col items-center justify-center">
 <div className="text-xl font-medium pb-4 text-center">
 Sorry, you are not seller.

 Become seller to access admin panel.
 </div>
 <div className="w-1/3">
 <Button
 onClick={() => navigate("/")}
 text={"Go to Home Page"}
 />
 </div>
 </div>
);
 }
 return children;
};

```



```

 primaryColor={true}
 />
 </div>
 </div>
);
}

return children;
};

const App = () => {
 const [openCart, setOpenCart] = useState(false);
 const [authToken, setAuthToken] = useState(
 localStorage.getItem("CF_authToken") || ""
);

 useEffect(() => {
 const token = localStorage.getItem("CF_authToken");
 if (token !== authToken) {
 setAuthToken(token);
 }
 }, [authToken]);

 const setUserAuthToken = (token) => {
 setAuthToken(token);
 };

 const handleOpenCart = () => {
 setOpenCart(true);
 };

 const [userDetails, setUserDetails] = useState(profileInitailValues);
 const [showFilterSection, setShowFilterSection] = useState(false);
 const [showCartSlider, setShowCartSlider] = useState(false);

 const { data } = useAPI(
 "post",
 "/user-details",
 authToken ? true : false,
 {},
 {
 "Content-Type": "application/json",
 authorization: `Bearer ${authToken}`,
 }
);

```

$$);$$

```
useEffect(() => {
 if (data) {
 setUserDetails(data.userDetails);
 }
}, [data]);
```

```
const appRouter = createBrowserRouter([
 {
 path: "/login",
 element: (
 <ThemeProvider>
 <Login setUserAuthToken={setUserAuthToken} />
 </ThemeProvider>
),
 },
 {
 path: "/",
 element: (
 <ThemeProvider>
 <AppLayout
 userDetails={userDetails}
 showFilterSection={showFilterSection}
 setShowFilterSection={setShowFilterSection}
 />
 </ThemeProvider>
),
 children: [
 {
 path: "/home",
 element: (
 <Home
 showFilterSection={showFilterSection}
 setShowFilterSection={setShowFilterSection}
 openCart={openCart}
 setOpenCart={setOpenCart}
 userAuthToken={authToken}
 setUserAuthToken={setUserAuthToken}
 setShowCartSlider={setShowCartSlider}
 />
),
 },
],
 },
])
```

```

 path: "/product/:productID",
 element: <ProductPage handleOpenCart={handleOpenCart} />,
 },
 {
 path: "/product/buynow/:productID",
 element: <BuyNow />,
 },
 {
 path: "/seller-admin-panel",
 element: (
 <RequireSeller isSeller={userDetails.isSeller}>
 <SellerDashboard />
 </RequireSeller>
),
 },
 {
 path: "/",
 element: <Navigate to="/home" />,
 },
 {
 path: "**",
 element: <Navigate to="/home" />,
 },
],
},
]);

```

```

return <RouterProvider router={appRouter} />;
};

```

```

export default App;

```

BuyNow.jsx

```

import React, { useEffect, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";
import swal from "sweetalert";
import { Colors, PaymentMethods, Sizes } from "../../config";
import { Input } from "../../GS-Libs";
import Button from "../../GS-Libs/Buttons/Button";
import { Textarea } from "../../GS-Libs/Input/Textarea";
import Checkbox from "../../GS-Libs/Input/Checkbox";
import ColorFilterCard from "../../Filters/ColorFilterCard";
import { toTitleCase } from "../../GS-Libs/utils/toTitleCase";

```

```

import QuantityInput from "../../GS-Libs/Input/QuantityInput";
import useForm from "../../hooks/useForm";
import {
 buyProductPageUserInitailValues,
 buyProductPageUserValidations,
} from "../../validations/buy-product-page-validations";
import { apiCaller } from "../../GS-Libs/utlis/apiCaller";
import ProductImages from "../../GS-Libs/MultiUse/ProductImages";
import {
 getDiscountedPrice,
 limitText,
} from "../../GS-Libs/utlis/productUtils";
import { useTheme } from "../../context/themeContext";

const PaymentItem = ({ name, value, isSelected, setUser }) => {
 return (
 <div
 className={`border border-dashed hover:border-solid p-2 rounded ${
 isSelected
 ? "bg-Purple text-White border-Purple"
 : "border-Gray hover:border-Purple hover:text-Purple"
 }}
 onClick={() => setUser((prev) => ({ ...prev, paymentMethod: value })))}
 >
 <div>{name}</div>
 </div>
);
};

```

```

export default function BuyNow() {
 const { theme } = useTheme();
 const navigate = useNavigate();
 const params = useParams();

 const [userPhoneNumber, setUserPhoneNumber] = useState("");
 const [userAddress, setUserAddress] = useState("");

 const [showTextLimit, setShowTextLimit] = useState(16);
 const [product, setProduct] = useState();
 const [selectedSize, setSelectSize] = useState("");
 const [selectedColor, setSelectedColor] = useState("");
 const [isSubmitted, setIsSubmitted] = useState(false);
 const [quantity, setQuantity] = useState(1);

```

```

const placeOrder = async () => {
 try {
 await apiCaller(
 "/checkout-product",
 "post",
 {
 products: [
 {
 _id: params.productID,
 quantity: quantity,
 size: selectedSize,
 color: selectedColor,
 paymentMethod: user.paymentMethod,
 },
],
 },
 true
);

 await swal(
 "Congrats!",
 "The item is added into your order list and delivered soon when we start delivering the products.",
 "success"
).then(() => {
 setIsSubmitted(false);
 navigate("/home");
 });
 } catch (error) {
 swal("Sorry!", "We don't start delivering products yet!", "info").then(
 () => setIsSubmitted(false)
);
 console.log(error);
 }
};

const {
 formData: user,
 setFormData: setUser,
 errors,
 handleChange,
 handleSubmit,
} = useForm(

```

```

 buyProductPageUserInitailValues,
 buyProductPageUserValidations,
 placeOrder
);

const buyProductDetails = async () => {
 try {
 const data = await apiCaller(
 "/buy-product",
 "post",
 {
 productID: params.productID,
 },
 {},
 true
);

 setUser((prev) => ({
 ...prev,
 name: data.userDetails.name,
 email: data.userDetails.email,
 phoneNumber: data.userDetails.phoneNumber,
 address: data.userDetails.address,
 }));
 setProduct(data.productDetails);
 } catch (error) {
 console.log(error);
 }
};

useEffect(() => {
 if (params.productID) buyProductDetails();
}, [params.productID]);

useEffect(() => {
 if (user) {
 setUserPhoneNumber(user.phoneNumber);
 setUserAddress(user.address);
 }
}, [user]);

if (!product || !user) {
 return (
 <div className="fullscreen-loader">

```

```

 <div className="spinner"></div>
 </div>
);
}

const submitForm = (e) => {
 setIsSubmitted(true);
 handleSubmit(e);

 if (selectedSize === "" || selectedColor === "") {
 return;
 }
};

const updateUserAddress = () => {
 setUser((prev) => ({ ...prev, address: userAddress }));
 apiCaller("/update-address", "post", { address: userAddress });
};

const updateUserPhoneNumber = () => {
 setUser((prev) => ({ ...prev, phoneNumber: userPhoneNumber }));
 apiCaller("/update-phonenum", "post", {
 phoneNumber: userPhoneNumber,
 });
};

return (
 <div
 className={`lg:h-full flex flex-col md:flex-row justify-center ${
 theme === "light" ? "bg-White" : "bg-Black"
 }`}
 >
 <div className="w-full md:w-3/4 flex flex-col md:flex-row gap-4 px-4 py-5">
 <div className="flex flex-col md:flex-row gap-4 w-full">
 <ProductImages product={product} />
 <div className="w-full max-w-[500px]">
 <div className="text-xl font-semibold">{product.name}</div>
 <div className="font-medium w-full">
 <p className="hidden xs:block">{product?.description}</p>
 <p className="xs:hidden">
 {limitText(product?.description, showTextLimit)}
 </p>
 <span
 className="text-Purple/80"
 onClick={

```

```

 showTextLimit <= 16
 ? () => setShowTextLimit(1000)
 : () => setShowTextLimit(16)
 }
 >
 {showTextLimit <= 16 ? "...show more" : "...hide"}

</p>
</div>
<div className="flex items-center gap-2 text-lg mt-4">
 <div className="font-medium inline-block text-center border border-dashed
border-Gray px-2 py-1 rounded hover:border-Purple hover:border-solid hover:bg-Purple
hover:text-White transition-all duration-200">
 {toTitleCase(product.brand)}
 </div>
 <div className="font-medium inline-block text-center border border-dashed
border-Gray px-2 py-1 rounded hover:border-Purple hover:border-solid hover:bg-Purple
hover:text-White transition-all duration-200">
 {toTitleCase(product.category)}
 </div>
</div>
<div className="text-3xl font-semibold text-Purple mt-4">
 -28% ₹
 {getDiscountedPrice(product?.price)}
</div>
<div className="text-lg text-Gray">
 MRP{" "}
 ₹{product?.price}
</div>
<div className="mt-4">
 <div className="text-xl font-semibold">Available Sizes</div>
 <div className="flex gap-2">
 {Sizes.map((size) => {
 return (
 <Checkbox
 label={size.name}
 key={size.id}
 isSelected={
 product?.sizes?.includes(size.name.toLowerCase()) ||
 false
 }
 >
 {size.name}
)
 })}
 </div>
</div>
);

```



```

 }}
 </div>
</div>
<div className="mt-4">
 <div className="text-xl font-semibold">Available Colors</div>
 <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-5 gap-1">
 {Colors.map((color) => {
 return (
 <ColorFilterCard
 color={color.color}
 key={color}
 isSelected={product.colors.includes(
 color.color.toLowerCase()
)}
 readOnly={true}
 />
);
 })}
 </div>
</div>
</div>
</div>
<div
 className={`w-full md:w-1/4 flex flex-col overflow-y-scroll justify-between gap-4 p-4
 shadow ${
 theme === "light" ? "bg-Light" : "bg-Light/10"
 }}
>
 <div className="w-full flex flex-col gap-2">
 <div className="w-full flex flex-col gap-1">
 <div className="text-lg font-medium">
 <p className="text-xs">Name</p> {user.name}
 </div>
 <div className="text-lg font-medium">
 <p className="text-xs">Email</p> {user.email}
 </div>
 <div className="text-lg font-semibold">
 <p className="text-xs">Phone Number</p>
 {user.phoneNumber ? (
 <>{user.phoneNumber}</>
) : (
 <>
 <Input

```

```

 type="number"
 name="phoneNumber"
 placeholder="Enter Phone Number"
 value={userPhoneNumber}
 onChange={(e) => setUserPhoneNumber(e.target.value)}
 errorMessage={errors.phoneNumber}
 className="p-2 border-2 border-Black/20 rounded w-full"
 />
 <div className="ml-auto w-fit mt-2">
 <Button
 text="Save Phone Number"
 disabled={userPhoneNumber === "" ? true : false}
 onClick={() => updateUserPhoneNumber()}
 />
 </div>
 </>
)}
</div>
<div className="text-lg font-semibold">
 <p className="text-xs">Address</p>
 {user.address ? (
 <>{user.address}</>
) : (
 <>
 <Textarea
 name="address"
 placeholder="Enter Address"
 value={userAddress}
 onChange={(e) => setUserAddress(e.target.value)}
 errorMessage={errors.address}
 className="p-2 border-2 border-Black/20 rounded w-full"
 />
 <div className="ml-auto mt-2 w-fit">
 <Button
 text="Save Address"
 disabled={userAddress === "" ? true : false}
 onClick={() => updateUserAddress()}
 />
 </div>
 </>
)}
</div>
</div>
<div className="w-full">

```

```

<div className="text-xs font-semibold">Quantity</div>
<div className="mt-2">
 <QuantityInput quantity={quantity} setQuantity={setQuantity} />
</div>
</div>
<div className="w-full">
 <div className="text-xs font-semibold">Select Size</div>

 <div className="flex gap-2 mt-2">
 {product.sizes.map((size) => {
 return (
 <div onClick={() => setSelectSize(size)}>
 <Checkbox
 label={size.toUpperCase()}
 key={size}
 isSelected={selectedSize === size}
 />
 </div>
);
 })}
 </div>
 {isSubmitted && selectedSize === "" && (
 Please Select Size
)}
</div>
<div className="w-full">
 <div className="text-xs font-semibold">Select Color</div>

 <div className="grid grid-cols-3 gap-2 mt-2">
 {product.colors.map((color) => {
 return (
 <div onClick={() => setSelectedColor(color)} key={color}>
 <ColorFilterCard
 color={color}
 isSelected={selectedColor === color}
 />
 </div>
);
 })}
 </div>
 {isSubmitted && selectedColor === "" && (
 Please Select Color
)}
</div>

```



```

<div className="grid grid-cols-2 gap-2 mt-2">
 <div className="col-span-2">
 <Input
 type="number"
 name="cardNumber"
 placeholder="Enter card number"
 value={user.cardNumber}
 onChange={handleChange}
 errorMessage={errors.cardNumber}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 </div>
 <Input
 type="text"
 name="validDate"
 placeholder="Enter Valid Date"
 value={user.validDate}
 onChange={handleChange}
 errorMessage={errors.validDate}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 <Input
 type="number"
 name="cvc"
 placeholder="Enter CVC"
 value={user.cvc}
 onChange={handleChange}
 errorMessage={errors.cvc}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
</div>
</div>
)}
</div>
</div>
<div className="w-full mt-4">
 <Button text="Place Order" size="medium" onClick={submitForm} />
</div>
</div>
</div>
);
}

```

Cart.jsx

```
import React, { useEffect, useState } from "react";
import CartCard from "../CartCard";
import Button from "../../GS-Libs/Buttons/Button";
import LabelValue from "../../GS-Libs/MultiUse/LabelValue";
import CheckoutCard from "../ProductCards/CheckoutCard";
import swal from "sweetalert";
import NotFoundImage from "../../Assets/images/not found.jpg";
import { apiCaller } from "../../GS-Libs/Utils/apiCaller";
import FullScreenLoader from "../../GS-Libs/MultiUse/FullScreenLoader";
```

```
const Cart = ({ showCartSlider }) => {
 const [cartProducts, setCartProducts] = useState([]);
 const [productPriceDetails, setProductPriceDetails] = useState({
 discount: 0,
 deliveryCharges: 0,
 totalPrice: 0,
 });
 const [isLoading, setIsLoading] = useState(false);
```

```
 const fetchCartItems = async () => {
 const data = await apiCaller("/access-cart-items", "post", {});

 setCartProducts(data.products);
 setIsLoading(false);
 };
};
```

```
useEffect(() => {
 if (showCartSlider) {
 setIsLoading(true);
 fetchCartItems();
 }
}, [showCartSlider]);
```

```
useEffect(() => {
 let totalPrice = 0,
 discount = 0,
 deliveryCharges = 0;
```

```
 cartProducts?.forEach((product) => {
 const quantity = product.quantity;
 const price = product.price;

 totalPrice += quantity * price;
 discount += Math.trunc(quantity * price * 0.05);
```

```

 deliveryCharges +=
 quantity * price > 1000 ? 0 : Math.trunc(quantity * price * 0.01);
 });

 setProductPriceDetails({
 discount: discount,
 deliveryCharges: deliveryCharges,
 totalPrice: totalPrice,
 });
}, [cartProducts]);

const checkout = async () => {
 if (cartProducts.length === 0) return;

 await apiCaller("/checkout-product", "post", {
 products: cartProducts.map((product) => ({
 _id: product._id,
 quantity: product.quantity,
 })),
 })
 .then(() => swal("Done", "Order placed successfully", "success"))
 .catch(() => swal("Oops", "Something went wrong", "error"));
};

const removeFromCart = async (productId) => {
 setIsLoading(true);
 await apiCaller("/remove-from-cart", "post", {
 productId: productId,
 });
 await fetchCartItems();
};

if (isLoading) {
 return (
 <div className="w-full h-full text-xl font-medium text-Gray text-center col-span-full flex
flex-col justify-center">
 <div className="relative w-full h-full rounded-md">
 <FullScreenLoader message="Loading cart products..." />
 </div>
 </div>
);
}

return (

```

```

<div className="w-full h-full flex flex-col md:flex-row justify-between overflow-y-scroll">
 <div className="w-full md:w-2/3 md:h-full p-2 md:px-4 md:pt-8 md:pb-4">
 {cartProducts?.length === 0 ? (
 <div className="w-full h-full pt-10">
 <div className="text-lg font-medium text-Gray">Empty Cart</div>
 <div className="">
 <img
 src={NotFoundImage}
 alt=""
 className="rounded-md max-w-64 max-h-64 md:max-w-96 md:max-h-96 "
 />
 </div>
 </div>
) : (
 <div className="grid grid-cols-1 xs:grid-cols-2 sm:grid-cols-3 gap-2 md:gap-4 md:h-full
overflow-y-scroll">
 {cartProducts?.reverse().map((product) => {
 return (
 <CartCard
 product={product}
 key={product._id}
 removeFromCart={removeFromCart}
 />
);
 })}
 </div>
)}
 </div>
 <div className="w-full md:w-1/3 md:h-full p-2 md:px-4 md:pt-8 md:pb-4 bg-Gray/20 relative
flex flex-col justify-between">
 <div className="w-full">
 <div className="pb-5 uppercase text-lg font-medium">Checkout</div>

 <div className="grid grid-cols-1 gap-2 overflow-y-scroll">
 {cartProducts?.reverse().map((product) => {
 return (
 <CheckoutCard
 key={product._id}
 product={product}
 cartProducts={cartProducts}
 setCartProducts={setCartProducts}
 />
);
 })}
 </div>
 </div>
 </div>

```



```

 </div>
 </div>
 <div className="w-full">
 <div className="flex flex-col gap-1 font-medium pb-4">
 <LabelValue
 label="Discount"
 value={` ₹${productPriceDetails.discount}`}
 />
 <LabelValue
 label="Delivery Charges"
 value={
 productPriceDetails.deliveryCharges > 0
 ? ` ₹${productPriceDetails.deliveryCharges}`
 : "Free Delivery"
 }
 />
 <hr className="bg-Black/40 text-Black/40 h-0.5 rounded" />
 <LabelValue
 label="Total Price"
 value={` ₹${productPriceDetails.totalPrice}`}
 />
 </div>
 <Button
 text="Checkout"
 onClick={checkout}
 disabled={cartProducts.length === 0}
 />
 </div>
</div>
</div>
);
};

```

export default Cart;

CartCard.jsx

```

import React from "react";
import Button from "../../GS-Libs/Buttons/Button";
import { Link } from "react-router-dom";
import { getDiscountedPrice } from "../../GS-Libs/utils/productUtils";

```

```

const CartCard = ({ product, removeFromCart }) => {
 return (

```

```

<div className="w-full h-fit p-2 rounded-md shadow-md flex flex-col justify-between border
border-dashed border-Gray cursor-pointer hover:border-solid hover:border-Purple
hover:shadow-lg">
 <div className="h-64">
 <img
 src={product.productImages[0]}
 alt=""
 className="w-full h-full object-fill rounded-md"
 />
 </div>
 <div className="text-Black pt-1 flex flex-col gap-1">
 <div className="text-xl font-semibold text-ellipsis overflow-hidden whitespace-nowrap">
 {product.name}
 </div>
 <div className="w-full flex items-center justify-between">
 <p className="text-Black">Price</p>
 <div className="flex items-center justify-between">
 <div className="flex items-center gap-1">
 <p className="font-semibold">
 ₹{getDiscountedPrice(product.price)}
 </p>
 <p className="line-through text-Gray">₹{product.price}</p>
 </div>
 </div>
 </div>
 </div>
 <div className="grid grid-cols-2 gap-2 mt-2">
 <Button
 text="Remove"
 size="small"
 primaryColor={false}
 onClick={() => removeFromCart(product._id)}
 />
 <Link to={"/product/buynow/" + product._id}>
 <Button
 text="Buy Now"
 size="small"
 onClick={() => {
 console.log("Go to Buy now page");
 }}
 />
 </Link>
 </div>
</div>

```

```
);
};
```

```
export default CartCard;
```

```
CartSlider.jsx
```

```
import React from "react";
import Cart from "../Cart";
import { ImCross } from "react-icons/im";
import { useTheme } from "../../context/themeContext";
```

```
const CartSlider = ({ showCartSlider, setShowCartSlider }) => {
 const { theme } = useTheme();
```

```
 return (
 <div
 className={`w-full md:w-3/4 h-full shadow shadow-Gray flex text-Black fixed
overflow-hidden transition-all duration-300 ease-in-out top-0 ${
 showCartSlider ? "right-0" : "-right-full"
 } ${theme === "light" ? "bg-White text-Black" : "bg-Black text-White"}}`
 >
 <div className="bg-Purple w-12 flex flex-col items-center justify-between pt-12 pb-4
px-2"></div>
 <div className="w-full flex relative">
 <div
 className="w-7 h-7 bg-White p-2 rounded-full flex justify-center items-center
cursor-pointer absolute top-2 -left-[15px] shadow-lg"
 onClick={() => setShowCartSlider(false)}
 >
 <ImCross className="w-5 h-5 text-Purple" />
 </div>
 <Cart showCartSlider={showCartSlider} />
 </div>
 </div>
);
};
```

```
export default CartSlider;
```

```
ColorFilter.jsx
```

```
import React from "react";
import { Colors } from "../../config";
import ColorFilterCard from "../ColorFilterCard";
```

```

const ColorFilter = ({ selectedFilters, onClick }) => {
 return (
 <div className="p-2 rounded-md shadow-md w-full max-w-md mx-auto">
 <div className="text-xl font-semibold pb-2">Colors</div>
 <div className="grid grid-cols-2 md:grid-cols-1 lg:grid-cols-2 gap-2">
 {Colors.map((color) => (
 <ColorFilterCard
 key={color.id}
 color={color.color}
 onClick={onClick}
 isSelected={selectedFilters["colors"].includes(
 color.color.toLowerCase()
)}
 >
))}
 </div>
 </div>
);
};

```

```
export default ColorFilter;
```

```
ColorFilterCard.jsx
```

```
import React, { useEffect, useState } from "react";
```

```

const ColorFilterCard = ({ color, onClick, isSelected, readOnly }) => {
 const [isCheckedSelected, setIsCheckboxSelected] = useState(false);

```

```

 useEffect(() => {
 setIsCheckboxSelected(isSelected);
 }, [isSelected]);
 return (
 <div
 className="flex items-center w-full"
 onClick={() => {
 onClick("colors", color);
 }}
 >
 <div
 className={`w-full flex items-center border ${
 isCheckedSelected
 ? "border-solid border-Purple bg-Purple text-White font-semibold"
 : "border-dashed border-Gray font-medium ${
 !readOnly && "hover:text-Purple"
 }`
 }

```

```

 }
 } p-1 rounded text-Gray font-medium min-w-8 text-center transition-all duration-150 ${
 !readOnly && "hover:border-solid hover:border-Purple"
 } ${readOnly ? "cursor-default" : "cursor-pointer"}}
 onClick={() => setIsCheckboxSelected(isCheckboxSelected ? false : true)}
>
 <div
 className={` ${color} min-w-6 min-h-6 mr-1 rounded border border-Black/20`}
 ></div>
 <div>
 {color.slice(0, 1).toUpperCase()}
 {color.slice(1)}
 </div>
</div>
</div>
);
};

```

export default ColorFilterCard;

FilterCard.jsx

import React from "react";

import Checkbox from "../GS-Libs/Input/Checkbox";

```

const FilterCard = ({ filter, selectedFilters, onClick }) => {
 return (
 <div className="p-2 rounded-md shadow-md w-full max-w-md mx-auto border border-Gray
border-dashed">
 <div className="text-xl font-semibold pb-2">{filter.name}</div>
 <div className="flex flex-wrap gap-2">
 {filter.options.map((option) => {
 return (
 <div
 key={option.id}
 onClick={() => onClick(filter.id, option.name)}
 >
 <Checkbox
 label={option.name}
 isSelected={selectedFilters[filter.id].includes(
 option.name.toLowerCase()
)}
 />
 </div>
);
 })}
 </div>
 </div>
);
};

```

```

 }}
 </div>
</div>
);
};

export default FilterCard;

```

Filters.jsx

```

import React, { useEffect, useState } from "react";
import { IoFilter } from "react-icons/io5";
import { AllFilters, MAX_PRICE, MIN_PRICE } from "../config";
import FilterCard from "../FilterCard";
import RangePicker from "../GS-Libs/Input/RangePicker";
import ColorFilter from "../ColorFilter";
import { useTheme } from "../context/themeContext";

const Filters = ({ selectedFilters, setSelectedFilters }) => {
 const [numberOfFilters, setNumberOfFilters] = useState(0);
 const { theme } = useTheme();
 const selectFilters = (filterName, optionName) => {
 const alreadySelected = selectedFilters[filterName].includes(
 optionName.toLowerCase()
);
 if (alreadySelected) {
 // Remove from selectedFilters
 setSelectedFilters((prev) => ({
 ...prev,
 [filterName]: prev[filterName].filter(
 (option) => option.toLowerCase() !== optionName.toLowerCase()
),
 }));
 } else {
 // Add into selectedFilters
 setSelectedFilters((prev) => ({
 ...prev,
 [filterName]: [...prev[filterName], optionName.toLowerCase()],
 }));
 }
 };

 useEffect(() => {
 if (selectedFilters) {

```

```

 let numberOfFilters = 0;
 Object.values(selectedFilters).forEach((filter) => {
 numberOfFilters += filter.length;
 });
 setNumberOfFilters(numberOfFilters);
 }
}, [selectedFilters]);

return (
 <div
 className={`w-full px-4 py-2 border-r ${
 theme === "light"
 ? "border-Black/40 bg-White text-Black"
 : "border-White/40 bg-Black text-White"
 }`}
 >
 <div
 className={`flex items-center justify-between pb-1 border-b ${
 theme === "light" ? "border-Black/40" : "border-White/40"
 }`}
 >
 <div className="flex gap-1 items-center">
 <IoFilter className="w-4 h-4" />
 <div className="text-lg font-semibold">Filters</div>
 </div>
 <div className="bg-Purple text-White text-lg font-semibold p-2 rounded w-7 h-7 flex
justify-center items-center">
 {numberOfFilters}
 </div>
 </div>
 <div className="flex flex-col gap-4 py-2">
 <RangePicker
 label="Price Range"
 minRangeLabel="Min Price"
 minRangeValue={MIN_PRICE}
 maxRangeLabel="Max Price"
 maxRangeValue={MAX_PRICE}
 setSelectedFilters={setSelectedFilters}
 />
 <div className="flex flex-col gap-4">
 {AllFilters.map((filter) => (
 <FilterCard
 key={filter.id}
 filter={filter}

```

```

 selectedFilters={selectedFilters}
 onClick={selectFilters}
 />
)))
 </div>
 <ColorFilter
 selectedFilters={selectedFilters}
 onClick={selectFilters}
 />
</div>
</div>
);
};

```

export default Filters;

Footer.jsx

```

import React from "react";
import { MdCopyright } from "react-icons/md";
import { SocialMedia } from "../../config";

```

```

const Footer = () => {
 const today = new Date().getFullYear();
 return (
 <div className="w-full px-4 py-2 bg-Black text-White/75 flex justify-between items-center relative">
 <div className="flex items-center gap-1">
 <div>All right reserved</div>
 <MdCopyright className="relative top-0.5" />
 <div>{today} - Closet Fashion</div>
 </div>
 <div className="flex items-center gap-4">
 <div className="hover:text-Purple font-medium text-lg">Gagan Saini</div>
 {SocialMedia.map((item) => {
 return (
 <div key={item.id} className="w-6 h-6">

 <img
 src={item.imgSrc}
 alt={item.name}
 className="w-full h-full"
 />

 </div>
);
 })}
 </div>
 </div>
);
};

```



```

);
 }}
</div>
</div>
);
};

```

export default Footer;

Home.jsx

```

import React, { useCallback, useEffect, useRef, useState } from "react";
import { MAX_PRICE, MIN_PRICE } from "../../config";
import Filters from "../Filters/Filters";
import { Link } from "react-router-dom";
import ProductCard1 from "../ProductCards/ProductCard1";
import NoResultsFound from "../../GS-Libs/MultiUse/NoResultsFound";
import { useSearchContext } from "../../context/searchContext";
import useDebounceAPI from "../../hooks/useDebounceAPI";
import FullScreenLoader from "../../GS-Libs/MultiUse/FullScreenLoader";
import { useTheme } from "../../context/themeContext";

```

```

const Home = ({ openCart, setShowCartSlider, showFilterSection }) => {
 const { theme } = useTheme();
 const [totalPages, setTotalPages] = useState(1);
 const observer = useRef();

```

```

 const limit = 12;
 const { searchQuery } = useSearchContext();
 const [allProducts, setAllProducts] = useState([]);
 const [page, setPage] = useState(1);
 const [selectedFilters, setSelectedFilters] = useState({
 brands: [],
 colors: [],
 price: [
 {
 minimum: MIN_PRICE,
 maximum: MAX_PRICE,
 },
],
 sizes: [],
 materials: [],
 genders: [],
 });

```

```
const authToken = localStorage.getItem("CF_authToken");
```

```
const { data, loading } = useDebounceAPI(
 "post",
 "/get-all-products",
 {
 selectedFilters,
 searchQuery,
 page,
 limit,
 },
 {
 "Content-Type": "application/json",
 authorization: `Bearer ${authToken}`,
 },
 [page],
 250
);
```

```
useEffect(() => {
 if (data?.products) {
 // If filter exist then save all products.
 if (
 selectedFilters.brands.length > 0 ||
 selectedFilters.colors.length > 0 ||
 selectedFilters.sizes.length > 0 ||
 selectedFilters.materials.length > 0 ||
 selectedFilters.genders.length > 0 ||
 searchQuery?.length > 0
) {
 setAllProducts([...data?.products]);
 } else {
 // For Pagination Records.
 setAllProducts((prev) => [...prev, ...data?.products]);
 }
 setTotalPages(data?.totalPages);
 }
}, [data?.products]);
```

```
useEffect(() => {
 if (
 selectedFilters.brands.length === 0 &&
 selectedFilters.colors.length === 0 &&
 selectedFilters.sizes.length === 0 &&
```

```

 selectedFilters.materials.length === 0 &&
 selectedFilters.genders.length === 0 &&
 searchQuery?.length === 0
) {
 setAllProducts([]);
 setPage(1);
 }
}, [selectedFilters]);

useEffect(() => {
 if (openCart) {
 setShowCartSlider(true);
 }
}, [openCart]);

// Intersection Observer callback to detect when the sentinel div is visible
const lastProductElementRef = useCallback(
 (node) => {
 if (loading) return;
 if (observer.current) observer.current.disconnect();
 observer.current = new IntersectionObserver((entries) => {
 if (entries[0].isIntersecting && page < totalPages) {
 setPage((prev) => prev + 1);
 }
 });
 if (node) observer.current.observe(node);
 },
 [loading, page, totalPages]
);

return (
 <div
 className={`flex relative h-full ${
 theme === "light" ? "text-Black/80" : "bg-Black text-White/80"
 }`}
 >
 <div
 className={`w-full xs:w-3/5 md:w-1/4 xl:w-1/5 h-full transition-all duration-300 absolute
md:relative z-10 ${
 showFilterSection ? "left-0" : "-left-full md:left-0"
 }`}
 >
 <div className="w-full h-full z-50">
 <Filters

```

```

 selectedFilters={selectedFilters}
 setSelectedFilters={setSelectedFilters}
 />
</div>
</div>
<div className="w-full md:w-3/4 xl:w-4/5 h-full">
 <div className="flex flex-col px-4 py-2 w-full h-full fixed md:w-3/4 xl:w-4/5">
 <div className="w-full h-full overflow-y-scroll">
 {allProducts.length > 0 ? (
 <>
 <div className="text-2xl pb-2 font-normal">All products</div>
 <div className="w-full min-h-fit h-fit grid grid-cols-2 sm:grid-cols-3 md:grid-cols-3
xl:grid-cols-4 gap-2 pb-28">
 {allProducts.map((product, index) => {
 return (
 <div
 className="w-full"
 key={product._id}
 ref={
 allProducts.length === index + 1
 ? lastProductElementRef
 : null
 }
 >
 <Link className="" to={"/product/" + product._id}>
 <ProductCard1 product={product} />
 </Link>
 </div>
);
 })}
 {loading && (
 <div className="pt-4 w-full text-xl font-medium text-Gray text-center
col-span-full">
 More Products Loading...
 </div>
)}
 </div>
 </>
) : (
 <>
 {loading ? (
 <div className="pt-4 w-full text-xl font-medium text-Gray text-center col-span-full">
 <FullScreenLoader message="Loading Products..." />
 </div>
) : (

```

```

) : (
 <div className="w-full h-full">
 <NoResultsFound
 actionText="Please Adjust Filters"
 infoText="Sorry! No Results Found"
 />
 </div>
)}
 </>
)}
</div>
</div>
</div>
);
};

```

export default Home;

Login.css

```

.login-container {
 height: 100vh;
 width: 100%;
 display: flex;
 overflow: hidden;
}

```

```

.login-item {
 width: 50%;
 overflow: hidden;
}

```

/\* Logo Container \*/

```

.circle {
 width: 700px;
 height: 700px;
 border-radius: 100%;
 /* background-color: var(--Blue); */
 position: absolute;
 left: -12%;
 top: -20%;
}

```

```
.shopping-logo-img {
 width: 50%;
 position: absolute;
 left: 250px;
 top: 200px;
}
```

```
.shopping-logo-img img {
 width: 100%;
}
```

```
.delivery-truck-img {
 width: 10%;
 position: absolute;
 left: 315px;
 top: 635px;
}
```

```
.delivery-truck-img img {
 width: 100%;
}
```

```
/* Form Container */
```

```
.form-container {
 padding-top: 50px;
 width: 85%;
 margin: 0px 0px auto 0px;
 cursor: pointer;
}
```

```
.greet-container {
 display: flex;
 justify-content: space-between;
 align-items: center;
}
```

```
.user-img {
 width: 15%;
}
```

```
.greet-message h1 {
 margin: 0px;
 font-size: 2.5rem;
```

```
}
```

```
.form-box {
 padding-top: 20px;
}
```

```
.extra-form-items {
 display: flex;
 justify-content: space-between;
 align-items: center;
 font-weight: 500;
}
```

```
.form-btn button {
 width: 100%;
 padding: 10px 0px;
 margin: 50px 0px 10px;
 font-size: 2rem;
 font-weight: 700;
 border: none;
 border-radius: 5px;
}
```

```
.signup-info {
 margin: 20px 0px;
}
```

```
.signup-info span {
 font-weight: 600;
}
```

```
.social-media-icon-box {
 width: 40%;
 margin: 30px auto;
 display: flex;
 justify-content: space-between;
 align-items: center;
}
```

```
.social-media-icon {
 width: 30%;
}
```

```
.social-media-icon img {
```

```
width: 70%;
}
```

```
.bottom-logo-circle {
 /* background-color: var(--Yellow); */
 border-radius: 100% 0px 0px 0px;
 width: 125px;
 height: 100px;
 display: flex;
 justify-content: center;
 align-items: center;
 position: absolute;
 right: 0%;
 bottom: 0%;
}
```

```
.bottom-logo-circle div {
 text-align: center;
 position: relative;
 left: 7%;
 top: 5%;
}
```

```
.company-name {
 margin: 0px;
 font-size: 2rem;
 font-weight: 700;
}
```

```
.company-full-name {
 margin: 0px;
 font-size: 1rem;
 font-weight: 500;
}
```

```
.guest-user {
 font-size: 1.1rem;
 font-weight: 600;
 text-align: center;
 padding: 4px 8px;
 border-radius: 4px;
 width: 40%;
 margin: 20px auto;
 cursor: pointer;
```



```
}
```

```
/* Tab View */
```

```
@media (min-width: 700px) and (max-width: 1200px) {
```

```
 .logo-container {
 display: none;
 }
```

```
 .login-item {
 width: 50%;
 }
```

```
 .login-item:nth-child(1) {
 display: none;
 }
```

```
 .login-item:nth-child(2) {
 width: 70%;
 margin: 10px auto;
 }
```

```
 .form-container {
 width: 90%;
 margin: auto;
 }
```

```
 .bottom-logo-circle {
 width: 100%;
 height: 70px;
 border-radius: 50% 50% 0px 0px;
 }
```

```
 .bottom-logo-circle div {
 position: relative;
 left: 0%;
 top: 0%;
 }
}
```

```
/* Mobile View */
```

```
@media (min-width: 300px) and (max-width: 700px) {
```

```
 .logo-container {
 display: none;
```

```

}

.login-item:nth-child(1) {
 display: none;
}

.login-item:nth-child(2) {
 width: 100%;
}

.greet-message h1 {
 margin: 0px;
 font-size: 1.5rem;
}

.form-container {
 width: 90%;
 margin: auto;
}

.bottom-logo-circle {
 width: 100%;
 height: 70px;
 border-radius: 50% 50% 0px 0px;
}

.bottom-logo-circle div {
 position: relative;
 left: 0%;
 top: 0%;
}
}

```

```

Login.jsx
import React, { useEffect, useState } from "react";
import SignupForm from "../SignupForm";
import LoginForm from "../LoginForm";
import {
 GUEST_USER_EMAIL,
 GUEST_USER_PASSWORD,
 SERVER_URL,
} from "../../config";
import { useNavigate } from "react-router-dom";
import swal from "sweetalert";

```

```
import OnlineShoppingImage from "../../Assets/images/online-shopping.png";
import DeliveryTruckImage from "../../Assets/images/delivery-truck.png";
import ManProfileImage from "../../Assets/images/man.png";
import { useTheme } from "../../context/themeContext";
```

```
export default function Login(props) {
 const [showLogin, setShowLogin] = useState(true);
 const navigate = useNavigate();

 const guestUserLogin = () => {
 fetch(`${SERVER_URL}/login`, {
 method: "POST",
 body: JSON.stringify({
 user: {
 email: GUEST_USER_EMAIL,
 password: GUEST_USER_PASSWORD,
 },
 }),
 headers: {
 "Content-type": "application/json; charset=UTF-8",
 },
 })
 .then((response) => {
 if (response.status === 401) {
 swal("Oops!", "User not found", "error");
 throw new Error("Unauthorized");
 }
 return response.json();
 })
 .then((data) => {
 const authToken = data.authToken;
 localStorage.setItem("CF_authToken", authToken);
 props.setUserAuthToken(authToken);
 navigate("/");
 })
 .catch((err) => {
 console.log(err);
 swal("Oops!", "Something went wrong", "error");
 });
 };
};
```

```
useEffect(() => {
 const authToken = localStorage.getItem("CF_authToken");
 if (authToken) {
```

```

 navigate("/");
 }
});

const { theme } = useTheme();

return (
 <div
 className={`login-container ${
 theme === "light" ? "bg-White text-Black" : "bg-Black text-White"
 }`}
 >
 <div className="login-item">
 <div className="logo-container">
 <div className="circle bg-Blue">
 <div className="shopping-logo-img">

 </div>
 <div className="delivery-truck-img">

 </div>
 </div>
 </div>
 <div className="login-item flex justify-center items-center">
 <div className="max-w-96">
 <div className="flex items-center justify-between">
 <div className="text-2xl font-semibold">
 <div>Hello,</div>
 {showLogin ? (
 <p>Welcome back!</p>
) : (
 <p>Welcome at Closet Fashion</p>
)}
 </div>

 </div>
 {showLogin ? (
 <LoginForm
 setUserAuthToken={props.setUserAuthToken}
 setShowLogin={setShowLogin}
 />
) : (

```

```

 <SignupForm
 setUserAuthToken={props.setUserAuthToken}
 setShowLogin={setShowLogin}
 />
)}
 <div
 onClick={guestUserLogin}
 className={`w-full rounded p-2 text-center cursor-pointer border-2 ${
 theme === "light"
 ? "bg-Light border-Black/20"
 : "bg-White/10 border-Light/50"
 }`}
 >
 Login as a Guest User
 </div>
</div>

<div className="bottom-logo-circle bg-Yellow text-Black/80">
 <div>
 <p className="company-name">CF</p>
 <p className="company-full-name">Closet Fashion</p>
 </div>
</div>
</div>
</div>
);
}

```

```

LoginForm.jsx
import React, { useState } from "react";
import "./Login.css";
import { useNavigate } from "react-router-dom";
import { SERVER_URL } from "../../config";
import { Input } from "../../GS-Libs";
import swal from "sweetalert";
import useForm from "../../hooks/useForm";
import {
 loginInitailValues,
 loginValidations,
} from "../../validations/login-validations";
import { IoMdEye, IoMdEyeOff } from "react-icons/io";

const LoginForm = ({ setUserAuthToken, setShowLogin }) => {
 const [showPassword, setShowPassword] = useState(false);

```

```

const navigate = useNavigate();

const handleLogin = () => {
 fetch(`${SERVER_URL}/login`, {
 method: "POST",
 body: JSON.stringify({
 user: user,
 }),
 headers: {
 "Content-type": "application/json; charset=UTF-8",
 },
 })
 .then((response) => {
 if (response.status === 401) {
 swal("Oops!", "User not found", "error");
 setUser(loginInitailValues);
 throw new Error("Unauthorized");
 }
 return response.json();
 })
 .then((data) => {
 const authToken = data.authToken;
 localStorage.setItem("CF_authToken", authToken);
 setUserAuthToken(authToken);
 navigate("/home");
 })
 .catch((err) => {
 console.log(err);
 swal("Oops!", "Something went wrong", "error");
 });
};

const {
 formData: user,
 setFormData: setUser,
 errors,
 handleChange,
 handleSubmit,
} = useForm(loginInitailValues, loginValidations, handleLogin);

return (
 <div className="w-full">
 <form className="form-box" onSubmit={handleSubmit}>
 <div className="flex flex-col gap-2">

```

```

<Input
 type="text"
 name="email"
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 placeholder="Email"
 value={user.email}
 onChange={handleChange}
 errorMessage={errors.email}
/>
<div className="relative">
 <Input
 type={showPassword ? "text" : "password"}
 name="password"
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 placeholder="Password"
 value={user.password}
 onChange={handleChange}
 errorMessage={errors.password}
 />
 <div
 className="absolute right-2 top-1/2 -translate-y-1/2 cursor-pointer"
 onClick={() => setShowPassword((prev) => !prev)}
 >
 {showPassword ? <IoMdEye /> : <IoMdEyeOff />}
 </div>
</div>
<div className="extra-form-items">
 <div className="flex items-center gap-2">
 <input type="checkbox" name="" />
 Remember me
 </div>
 <div>Forgot Password?</div>
</div>
</div>

<button
 type="submit"
 className="w-full bg-Purple p-2 rounded text-white text-center cursor-pointer mt-10"
>
 Login
</button>
</form>
<div className="signup-info">
 Don't have an account {" "}

```

```

 {
 setShowLogin(false);
 }}
 className="text-Purple"
 >
 Sign-up here!

 </div>
</div>
);
};

```

export default LoginForm;

SignupForm.jsx

```

import React, { useState } from "react";
import swal from "sweetalert";
import { useNavigate } from "react-router-dom";
import { SERVER_URL } from "../../config";
import { Input } from "../../GS-Libs";
import useForm from "../../hooks/useForm";
import {
 signupInitailValues,
 signupValidations,
} from "../../validations/signup-validations";
import { IoMdEye, IoMdEyeOff } from "react-icons/io";

const SignupForm = ({ setUserAuthToken, setShowLogin }) => {
 const [showPassword, setShowPassword] = useState(false);
 const navigate = useNavigate();
 const handleSignup = (event) => {
 fetch(`${SERVER_URL}/signup`, {
 method: "POST",
 body: JSON.stringify({
 user: user,
 }),
 headers: {
 "Content-type": "application/json; charset=UTF-8",
 },
 })
 .then((response) => response.json())
 .then((data) => {
 const authToken = data.authToken;

```



```

 localStorage.setItem("CF_authToken", authToken);
 navigate("/home");
 setUserAuthToken(authToken);
 setUser(signupInitailValues);
 })
 .catch((err) => {
 console.log(err);
 swal("Oops!", "Something went wrong", "error");
 });
};

const {
 formData: user,
 setFormData: setUser,
 errors,
 handleChange,
 handleSubmit,
} = useForm(signupInitailValues, signupValidations, handleSignup);

return (
 <div>
 <div>
 <form className="form-box" onSubmit={handleSubmit}>
 <div className="flex flex-col gap-2">
 <Input
 type="text"
 name="name"
 placeholder="Name"
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 value={user.name}
 errorMessage={errors.name}
 onChange={handleChange}
 />
 <Input
 type="email"
 name="email"
 placeholder="Email"
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 value={user.email}
 errorMessage={errors.email}
 onChange={handleChange}
 />
 <div className="relative">
 <Input

```

```

 type={showPassword ? "text" : "password"}
 name="password"
 placeholder="Password"
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 value={user.password}
 onChange={handleChange}
 errorMessage={errors.password}
 />
 <div
 className="absolute right-2 top-1/2 -translate-y-1/2 cursor-pointer"
 onClick={() => setShowPassword((prev) => !prev)}
 >
 {showPassword ? <IoMdEye /> : <IoMdEyeOff />}
 </div>
 </div>
</div>
<button
 type="submit"
 className="w-full bg-Purple p-2 rounded text-white text-center cursor-pointer mt-10"
>
 Create Account
</button>
</form>
<div className="signup-info">
 Already have an account {" "}
 {
 setShowLogin(true);
 }}
 className="text-Purple"
 >
 Sign-in here!

</div>
</div>
</div>
);
};

```

export default SignupForm;

Navbar.jsx

```

import React, { useEffect, useState } from "react";
import { Link } from "react-router-dom";

```

```

import NavbarRightOptions from "../NavbarRightOptions";
import ProfileSlider from "../Profile/ProfileSlider";
import CartSlider from "../Cart/CartSlider";
import { profileInitailValues } from "../validations/profile-form";
import CFLogoImage from "../Assets/images/closet fashion-logos.jpeg";
import useAPI from "../hooks/useAPI";
import { FiMenu } from "react-icons/fi";
import { RxCross1 } from "react-icons/rx";
import { useTheme } from "../context/themeContext";

export default function Navbar({
 showProfileSlider,
 setShowProfileSlider,
 showCartSlider,
 setShowCartSlider,
 userDetails,
 userAuthToken,
 setUserAuthToken,
 showFilterSection,
 setShowFilterSection,
}) {
 const { theme } = useTheme();
 const [searchFlag, setSearchFlag] = useState(false);

 return (
 <div
 className={`flex items-center shadow px-4 py-2 md:py-4 sticky top-0 z-50 ${
 theme === "light"
 ? "bg-White text-Black shadow-Black/10"
 : "bg-Black text-White shadow-Light/30"
 }`}
 >
 {searchFlag !== undefined && !searchFlag && (
 <div className="flex items-center justify-between w-full">
 <div className="flex items-center gap-2">
 <div
 className="md:hidden"
 onClick={() => setShowFilterSection((prev) => !prev)}
 >
 {showFilterSection ? (
 <RxCross1 className="w-6 h-6 font-semibold" />
) : (
 <FiMenu className="w-6 h-6 font-semibold" />
)}
 </div>
 </div>
 </div>
)}
 </div>
);
}

```

```

</div>
<Link to="/home">
 <img
 src={CFLogoImage}
 alt=""
 className="w-6 h-6 md:w-8 md:h-8 rounded"
 />
</Link>
<div className="font-medium md:font-semibold text-sm sm:text-lg md:text-2xl">
 Closet Fashion
</div>
</div>

<div className="flex items-center gap-2">
 <NavbarRightOptions
 userAuthToken={userAuthToken}
 profileImage={userDetails?.profileImage}
 name={userDetails?.name}
 setShowCartSlider={setShowCartSlider}
 setShowProfileSlider={setShowProfileSlider}
 />

 {userAuthToken !== undefined &&
 userAuthToken !== null &&
 userAuthToken !== "" ? (
 <>
 <ProfileSlider
 userDetails={userDetails}
 setUserAuthToken={setUserAuthToken}
 showProfileSlider={showProfileSlider}
 setShowProfileSlider={setShowProfileSlider}
 />
 <CartSlider
 showCartSlider={showCartSlider}
 setShowCartSlider={setShowCartSlider}
 />
 </>
) : (
 <div className="flex items-center gap-2">
 <Link to="/login">
 <div className="w-full bg-Purple px-2 py-1 rounded text-white text-center
cursor-pointer">
 Login
 </div>
 </div>
 </div>
)
 }
</div>

```

```

 </Link>
 </div>
)}
 </div>
</div>
)}
</div>
);
}

```

NavbarRightOptions.jsx

```

import React from "react";
import Searchbar from "../../GS-Libs/Input/Searchbar";
import { PiShoppingCart } from "react-icons/pi";
import ManProfileImage from "../../Assets/images/man.png";

const NavbarRightOptions = ({
 userAuthToken,
 profileImage,
 name,
 setShowCartSlider,
 setShowProfileSlider,
}) => {
 return (
 <div className="flex items-center gap-2 md:gap-4">
 <Searchbar />
 {userAuthToken !== undefined &&
 userAuthToken !== null &&
 userAuthToken !== "" && (
 <>
 <div
 className="flex items-center gap-1 md:gap-2 cursor-pointer"
 onClick={() => setShowCartSlider(true)}
 >
 <PiShoppingCart className="w-6 h-6 md:w-8 md:h-8" />
 <div className="text-md font-medium hidden md:block">Cart</div>
 </div>
 <div
 className="flex items-center gap-1 md:gap-2 cursor-pointer"
 onClick={() => setShowProfileSlider(true)}
 >
 <img
 className="w-6 h-6 md:w-8 md:h-8 rounded"
 src={profileImage || ManProfileImage}
 />
 </div>
 </>
)}
 </div>
);
}

```

```

 alt="Profile Image"
 />
 <div className="text-md font-medium hidden md:block">
 {name || "Profile"}
 </div>
 </div>
 </>
)}
</div>
);
};

```

```
export default NavbarRightOptions;
```

CheckoutCard.jsx

```

import React, { useEffect, useState } from "react";
import QuantityInput from "../../GS-Libs/Input/QuantityInput";
import { getDiscountedPrice } from "../../GS-Libs/utils/productUtils";

const CheckoutCard = ({ product, cartProducts, setCartProducts }) => {
 const [quantity, setQuantity] = useState(1);

 useEffect(() => {
 const quantityUpdatedCartProducts = cartProducts.map((item) => {
 if (item._id === product._id) {
 item.quantity = quantity;
 }

 return item;
 });

 setCartProducts([...quantityUpdatedCartProducts], [quantity]);
 });

 return (
 <div
 className="border border-dashed border-Gray rounded p-1 w-full"
 key={product._id}
 >
 <div className="flex justify-between gap-2">
 <div className="flex gap-2">
 <div className="w-20 h-20">
 <img
 src={product.productImages[0]}

```

```

 className="w-full h-full rounded"
 alt=""
 />
 </div>
 <div>
 <div className="font-medium text-xl">{product.name}</div>
 <div className="text-Purple text-2xl">
 ₹{getDiscountedPrice(product.price)}
 </div>
 </div>
 </div>
 <div>
 <QuantityInput quantity={quantity} setQuantity={setQuantity} />
 </div>
</div>
);
};

```

```

export default CheckoutCard;
OrderCard.jsx
import React from "react";
import { getDiscountedPrice } from "../../GS-Libs/utils/productUtils";

```

```

const OrderCard = ({ product }) => {
 return (
 <div className="w-full h-fit p-2 rounded-md shadow-md flex flex-col justify-between border
border-dashed border-Gray cursor-pointer hover:border-solid hover:border-Purple
hover:shadow-lg">
 <div className="h-64">
 <img
 src={product.productImages[0]}
 alt=""
 className="w-full h-full object-fill rounded-md"
 />
 </div>
 <div className="pt-1 flex flex-col gap-1">
 <div className="text-xl font-semibold text-ellipsis overflow-hidden whitespace-nowrap">
 {product.name}
 </div>
 <div className="w-full flex items-center justify-between pt-2">
 <p className="">Price</p>
 <div className="flex items-center justify-between">
 <div className="flex items-center gap-1">
 <p className="font-semibold">
 ₹{getDiscountedPrice(product.price)}
 </p>

```

```

 </p>
 <p className="line-through text-Gray">₹{product.price}</p>
 </div>
 </div>
 </div>
</div>
</div>
);
};

```

export default OrderCard;

ProductCard1.jsx

```

import React, { useState } from "react";
import { toTitleCase } from "../../GS-Libs/utils/toTitleCase";
import { getDiscountedPrice } from "../../GS-Libs/utils/productUtils";

export default function ProductCard1(props) {
 const [product, setProduct] = useState(props.product);

 return (
 <div className="w-full h-80 md:h-96 p-2 rounded-md shadow-md border border-dashed
border-Gray cursor-pointer hover:border-solid hover:border-Purple hover:shadow-lg">
 <div className="h-5/6 md:h-3/4">
 <img
 src={product.productImages[0]}
 alt=""
 className="w-full h-full object-fill rounded-md"
 />
 </div>
 <div className="pt-1 flex flex-col gap-1">
 <div className="text-sm sm:text-base md:text-xl font-semibold whitespace-nowrap
text-ellipsis overflow-hidden">
 {product.name}
 </div>
 <div>
 <p className="relative top-1 hidden md:block">Price</p>
 <div className="flex flex-col md:flex-row md:items-center md:justify-between">
 <div className="flex items-center gap-0.5 md:gap-1">
 <p className="text-sm sm:text-base md:text-xl font-semibold">
 ₹{getDiscountedPrice(product.price)}
 </p>
 <p className="text-sm line-through text-Gray">₹{product.price}</p>
 </div>

```



```

 <div className="bg-Purple text-White text-base px-2 py-1 rounded items-center
justify-center hidden md:flex">
 {toTitleCase(product.brand)}
 </div>
 </div>
</div>
</div>
</div>
);
}

```

ProductPage.tsx

```

import React, { useEffect, useState } from "react";
import { Link, useParams } from "react-router-dom";
import swal from "sweetalert";
import { Sizes } from "../../config";
import Checkbox from "../../GS-Libs/Input/Checkbox";
import { getDeliveryDate } from "../../GS-Libs/Utils/getDeliveryDate";
import ColorFilterCard from "../../Filters/ColorFilterCard";
import LabelValue from "../../GS-Libs/MultiUse/LabelValue";
import { apiCaller } from "../../GS-Libs/Utils/apiCaller";
import Button from "../../GS-Libs/Buttons/Button";
import {
 getDeliveryCharges,
 getDiscountedPrice,
 limitText,
} from "../../GS-Libs/Utils/productUtils";
import ProductImages from "../../GS-Libs/MultiUse/ProductImages";
import FullScreenLoader from "../../GS-Libs/MultiUse/FullScreenLoader";
import { useTheme } from "../../context/themeContext";

```

```

export default function ProductPage({ handleOpenCart }) {

```

```

 const { theme } = useTheme();
 const params = useParams();
 const productId = params.productId;

```

```

 const authToken = localStorage.getItem("CF_authToken");
 const [showTextLimit, setShowTextLimit] = useState(16);
 const [sellerDetails, setSellerDetails] = useState({});
 const [product, setProduct] = useState({});
 const [isProductAlreadyInCart, setIsProductAlreadyInCart] = useState(false);
 const [isProductFetched, setIsProductFetched] = useState(false);

```

```

 const getProductById = async () => {

```

```

setIsProductFetched(false);
try {
 const data = await apiCaller(
 "/get-product-with-id",
 "post",
 {
 productID: productID,
 },
 {},
 false
);

 setProduct(data.foundProduct);
 setSellerDetails(data.sellerDetails);
 setIsProductAlreadyInCart(data.isProductAlreadyInCart);
 setIsProductFetched(true);
} catch (error) {
 console.log(error);
}
};

useEffect(() => {
 getProductById();
}, [productID]);

const addToCart = async () => {
 try {
 await apiCaller("/add-to-cart", "post", {
 productID: productID,
 });

 swal("Congrats!", "Item added into your cart", "success").then(() =>
 setIsProductAlreadyInCart(true)
);
 } catch (error) {
 console.log(error);
 swal("Oops!", error, "error");
 }
};

if (!isProductFetched) {
 return (
 <div className="relative w-full h-full">
 <FullScreenLoader message="Loading Product Details..." />
 </div>
);
}

```

```

 </div>
);
}

return (
 <div
 className={`p-4 lg:p-8 w-full lg:h-full ${
 theme === "light" ? "bg-White text-Black/80" : "bg-Black text-White/80"
 }`}
 >
 <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-12 gap-4 sm:gap-6 lg:gap-16
h-full">
 <ProductImages product={product} />
 <div className="lg:col-span-5">
 <div className="h-full flex flex-col justify-between">
 <div>
 <div className="flex flex-col gap-2 lg:gap-4 w-full max-w-[500px]">
 <div className="text-2xl sm:text-3xl lg:text-5xl font-medium">
 {product?.name}
 </div>
 <div className="text-sm sm:text-base lg:text-lg font-semibold lg:font-medium">
 <p className="hidden xs:block">{product?.description}</p>
 <p className="xs:hidden">
 {limitText(product?.description, showTextLimit)}
 </p>
 <span
 className="text-Purple/80"
 onClick={
 showTextLimit <= 16
 ? () => setShowTextLimit(1000)
 : () => setShowTextLimit(16)
 }
 >
 {showTextLimit <= 16 ? "...show more" : "...hide"}

 </div>
 </div>
 <div className="">
 <div className="bg-Red/70 text-White font-medium px-2 py-1 rounded
inline-block">
 Limited time deal
 </div>
 <div className="text-3xl font-semibold text-Purple mt-2">
 –28% ₹
 {getDiscountedPrice(product?.price)}
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
);
}

```

```

 </div>
 <div className="text-lg text-Gray">
 MRP{" "}

 ₹{product?.price}

 </div>
 </div>
</div>
<div className="mt-1 lg:mt-2">
 <div className="text-xl font-semibold">Sizes</div>
 <div className="flex gap-2">
 {Sizes.map((size) => {
 return (
 <Checkbox
 label={size.name}
 key={size.id}
 isSelected={
 product?.sizes?.includes(size.name.toLowerCase()) ||
 false
 }
 readOnly={true}
 />
);
 })}
 </div>
</div>
<div className="mt-1 lg:mt-2">
 <div className="text-xl font-semibold">Colors</div>
 <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-5 gap-1">
 {product?.colors.map((color) => {
 return (
 <ColorFilterCard
 color={color}
 key={color}
 readOnly={true}
 />
);
 })}
 </div>
</div>
</div>
</div>
</div>

```

```

<div className="h-full lg:h-3/5 border border-dashed border-Gray p-2 rounded-md flex
flex-col justify-between md:col-span-2 lg:col-span-3 shadow-md pb-4">
 <div className="flex flex-col gap-4">
 <div>
 <div>
 <div className="text-2xl font-semibold text-Purple">
 ₹{getDiscountedPrice(product?.price)}
 </div>
 </div>
 <div className="text-sm lg:text-lg">
 {product?.price > 1000
 ? "Free delivery "
 : `₹${getDeliveryChagres(product?.price)} delivery chagres`}
 {getDeliveryDate()}.
 </div>
 </div>
 <div className="">
 <div className="text-xl font-medium text-Green">In Order</div>
 <div className="flex flex-col gap-1">
 <LabelValue label="Ships from" value="Closet Fashion" />
 <LabelValue label="Sold by" value={sellerDetails.name} />
 <LabelValue label="Packaging by" value="CF Packaging" />
 </div>
 </div>
 </div>
 <div className="grid grid-cols-2 gap-2 w-full mt-4 lg:mt-0">
 {
 <>
 {authToken === undefined ||
 authToken === null ||
 authToken === "" ? (
 <div className="col-span-2">
 <Link to={"/login"}>
 <Button text="Login to Buy" />
 </Link>
 </div>
) : (
 <>
 {isProductAlreadyInCart ? (
 <Link to={"/home"}>
 <Button
 onClick={() => handleOpenCart()}
 text="Go to Cart"
 primaryColor={false}

```

```

 />
 </Link>
) : (
 <Button
 onClick={() => addToCart()}
 text="Add to Cart"
 primaryColor={false}
 />
)}
 {product.quantity === 0 ? (
 <div className="bg-Red/70 text-White rounded shadow font-medium flex
justify-center items-center">
 Out of Stock
 </div>
) : (
 <Link to={"/product/buynow/" + productID}>
 <Button text="Buy" />
 </Link>
)}
 </>
)}
</>
}
</div>
</div>
</div>
</div>
);
}

```

MyOrders.jsx

```

import React, { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import OrderCard from "../ProductCards/OrderCard";
import NotFoundImage from "../Assets/images/not found.jpg";
import { apiCaller } from "../GS-Libs/utlis/apiCaller";

```

```

const MyOrders = ({ orders, showProfileSlider }) => {
 const [userOrders, setUserOrders] = useState([]);

```

```

 const fetchProductData = async () => {
 try {
 const data = await apiCaller("/ordered-products", "post", {
 orders: orders,

```

```

});

 setUserOrders(data.products);
 } catch (error) {
 console.log(error);
 }
};

useEffect(() => {
 if (showProfileSlider) {
 fetchProductData();
 }
}, [orders, showProfileSlider]);

return (
 <div className="w-full h-full">
 <div className="text-2xl font-semibold pb-4 flex items-center justify-between">
 <div>My Orders</div>
 {userOrders.length > 0 && (
 <div className="bg-Purple text-White text-lg font-semibold p-2 rounded w-7 h-7 flex
justify-center items-center">
 {userOrders.length}
 </div>
)}
 </div>
 <div className="w-full h-[90%]">
 {userOrders.length === 0 ? (
 <div className="w-full h-full">
 <div className="text-lg font-medium text-Gray">
 You did not order anything yet!
 </div>
 <div className="w-full h-full flex items-center justify-center">

 </div>
 </div>
) : (
 <div className="grid grid-cols-1 xs:grid-cols-2 gap-4">
 {userOrders.reverse().map((product, productIndex) => {
 return (
 <div className="w-full" key={`_${product._id}-${productIndex}`}>
 <Link
 className="product-card-link"
 to={"/product/" + product._id}
 >

```

```

 <OrderCard product={product} />
 </Link>
 </div>
);
 }}}
</div>
)}
</div>
</div>
);
};

```

export default MyOrders;

Profile.jsx

```

import React, { useState } from "react";
import { MdEdit } from "react-icons/md";
import uploadImage from "../../GS-Libs/utills/uploadImage";
import Button from "../../GS-Libs/Buttons/Button";
import ManProfileImage from "../../Assets/images/man.png";
import ProfileCard from "../ProfileCard";

```

```

export default function Profile({
 user,
 setUser,
 errors,
 handleChange,
 handleSubmit,
 isEditing,
 setIsEditing,
}) {
 const [showImageEditOption, setShowImageEditOption] = useState(true);

 return (
 <div className="w-full h-full relative">
 <form onSubmit={handleSubmit} className="w-full h-full">
 <div className="w-full h-full max-w-60 max-h-60 flex justify-center items-center mx-auto rounded-md">
 {isEditing && (
 <div className="absolute w-full h-full max-w-60 max-h-60 mx-auto">
 <label htmlFor="profile-image-upload">
 <div
 className={` ${
 showImageEditOption &&

```



```

 "absolute top-0 bg-Gray/50 z-20 w-full h-full rounded-full"
 } ${
 !showImageEditOption && "hidden"
 } flex items-center justify-center cursor-pointer`
 onMouseEnter={() => setShowImageEditOption(true)}
 onMouseLeave={() => setShowImageEditOption(false)}
 >
 <MdEdit className="w-8 h-8 text-White" />
 <input
 type="file"
 className="hidden"
 id="profile-image-upload"
 multiple={false}
 onChange={(event) => {
 uploadImage(event).then((res) => {
 setUser((prev) => ({
 ...prev,
 profileImage: res[0],
 }));
 });
 }}
 />
 </div>
</label>
</div>
)}
<div className="relative w-full h-full">
 <img
 className="rounded-lg absolute top-0 w-full h-full max-w-60 max-h-60"
 src={
 user.profileImage !== "" ? user.profileImage : ManProfileImage
 }
 alt="Profile Image"
 onMouseEnter={() => isEditing && setShowImageEditOption(true)}
 onMouseLeave={() => isEditing && setShowImageEditOption(false)}
 />
</div>
</div>
<div className="flex flex-col gap-2 pt-10">
 <ProfileCard
 label="Name"
 value={user.name}
 type="text"
 placeholder="Enter Name"
 />
</div>

```

```
name="name"
isEditing={isEditing}
onChange={(e) => handleChange(e)}
errorMessage={errors.name}
/>
<ProfileCard
 label="Email"
 value={user.email}
 type="text"
 placeholder="Enter Email"
 name="email"
 isEditing={isEditing}
 onChange={(e) => handleChange(e)}
 errorMessage={errors.email}
/>
<ProfileCard
 label="Phone Number"
 value={user.phoneNumber}
 type="number"
 placeholder="Enter Phone Number"
 name="phoneNumber"
 isEditing={isEditing}
 onChange={(e) => handleChange(e)}
 errorMessage={errors.phoneNumber}
/>
<ProfileCard
 label="Address"
 value={user.address}
 type="text"
 placeholder="Enter Address"
 name="address"
 isEditing={isEditing}
 onChange={(e) => handleChange(e)}
 errorMessage={errors.address}
/>
<ProfileCard
 label="Website"
 value={user.website}
 type="text"
 placeholder="Enter Website URL"
 name="website"
 isEditing={isEditing}
 onChange={(e) => handleChange(e)}
 errorMessage={errors.website}
```

```

 isLink={true}
 />
 </div>

 {isEditing ? (
 <div className="absolute bottom-0 right-0 w-full flex gap-2 items-center justify-end">
 <div className="w-2/5 md:w-1/3">
 <Button
 text="Cancel"
 type="button"
 primaryColor={false}
 onClick={() => setIsEditing(false)}
 />
 </div>
 <div className="w-2/5 md:w-1/3">
 <Button text="Save" type="button" onClick={handleSubmit} />
 </div>
 </div>
) : (
 <div className="absolute bottom-0 right-0 w-1/3 md:w-1/5">
 <Button
 text="Edit"
 type="button"
 onClick={() => setIsEditing(true)}
 />
 </div>
)}
 </form>
</div>
);
}

```

ProfileCard.jsx

```

import React from "react";
import { Input } from "../GS-Libs";

```

```

const ProfileCard = ({
 label,
 value,
 type,
 placeholder,
 name,
 isEditing,
 onChange,

```

```

 errorMessage,
 isLink = false,
 }) => {
 return (
 <div className="flex flex-col md:flex-row md:items-center md:gap-2 w-full">
 <p className="text-Gray w-full md:w-1/3 max-w-32">{label}</p>
 <div className="w-full md:w-2/3">
 {isEditing ? (
 <>
 <Input
 className="p-2 border-2 border-Black/20 rounded w-full"
 type={type}
 placeholder={placeholder}
 value={value}
 name={name}
 errorMessage={errorMessage}
 onChange={onChange}
 />
 </>
) : (
 <div className="font-semibold">
 {value === "" ? (
 "Not available"
) : isLink ? (

 Live

) : (
 {value}
)}
 </div>
)}
 </div>
 </div>
);
 };
};

```

export default ProfileCard;

ProfileDelete.jsx

```

import React from "react";
import { MdDelete } from "react-icons/md";
import swal from "sweetalert";
import { apiCaller } from "../../GS-Libs/Utils/apiCaller";

```

```

import { useNavigate } from "react-router-dom";

const ProfileDelete = ({ user }) => {
 const navigate = useNavigate();

 const deleteAccount = () => {
 swal(
 "Delete Account!",
 "Your account will be peramantly deleted",
 "warning",
 {
 buttons: {
 cancel: true,
 confirm: {
 text: "Delete",
 value: true,
 className: "bg-Red/70 text-White !hover:bg-Red/80",
 },
 },
 }
).then(async (confirm) => {
 if (confirm) {
 if (
 user?.email !== "guestuser@closetfashion.com" &&
 user?.email !== "cfadmin@gmail.com"
) {
 const data = await apiCaller(
 "/delete-account",
 "post",
 {},
 {},
 true
).then(() => {
 localStorage.removeItem("CF_authToken");
 });
 navigate("/home");
 } else {
 swal("Sorry!", "Guest User can't be deleted", "error");
 }
 }
 });
 };

 return (

```

```

 <div
 className="border-dashed border border-Gray !p-2 rounded-md flex justify-between
items-center w-full hover:bg-Red/70 hover:text-White hover:border-solid hover:border-Red/70
transition-all duration-75 ease-in-out cursor-pointer"
 onClick={deleteAccount}
 >
 <div>Delete Account</div>
 <div>
 <MdDelete className="w-5 h-5" />
 </div>
 </div>
);
};

```

```
export default ProfileDelete;
```

ProfileSlider.jsx

```

import React, { useEffect, useState } from "react";
import { ImCross } from "react-icons/im";
import { FaUser, FaShopify } from "react-icons/fa";
import { HiShoppingBag } from "react-icons/hi2";
import { IoSettingsSharp } from "react-icons/io5";
import { FiLogOut } from "react-icons/fi";
import { MdSpaceDashboard } from "react-icons/md";
import Profile from "../Profile";
import MyOrders from "../MyOrders";
import SellerAccount from "../Seller/SellerAccount/SellerAccount";
import useForm from "../../hooks/useForm";
import {
 profileInitailValues,
 profileValidations,
} from "../../validations/profile-form";
import swal from "sweetalert";
import UploadProduct from "../UploadProduct/UploadProduct";
import { apiCaller } from "../../GS-Libs/utils/apiCaller";
import Settings from "../Settings";
import { useTheme } from "../../context/themeContext";
import { useNavigate } from "react-router-dom";

```

```

const TabOptions = [
 {
 id: "user",
 name: "User",
 type: "tab",

```

```

 icon: <FaUser className="w-5 h-5" />,
 },
 {
 id: "orders",
 name: "Orders",
 type: "tab",
 icon: <HiShoppingBag className="w-5 h-5" />,
 },
 {
 id: "seller",
 name: "Seller",
 type: "tab",
 icon: <FaShopify className="w-5 h-5" />,
 },
 {
 id: "seller-admin-panel",
 name: "Seller Admin Panel",
 type: "link",
 link: "/seller-admin-panel",
 icon: <MdSpaceDashboard className="w-5 h-5" />,
 },
 {
 id: "setting",
 name: "Setting",
 type: "tab",
 icon: <IoSettingsSharp className="w-5 h-5" />,
 },
],
];

```

```

const ProfileSlider = ({
 userDetails,
 setUserAuthToken,
 showProfileSlider,
 setShowProfileSlider,
}) => {
 const navigate = useNavigate();
 const { theme } = useTheme();
 const [activeTab, setActiveTab] = useState("user");
 const [isEditing, setIsEditing] = useState(false);

 const updateProfile = async () => {
 try {
 const data = await apiCaller("/update-profile", "post", { user });
 }
 }
};

```

```

 if (data.status === 200) {
 swal("Done!", "Your profile has been updated", "success");
 setIsEditing(false);
 } else {
 swal("Oops!", "Something went wrong, try again", "error");
 }
 } catch (error) {
 swal("Oops!", "Something went wrong, try again", "error");
 console.log(error);
 }
};

```

```

const {
 formData: user,
 setFormData: setUser,
 errors,
 handleChange,
 handleSubmit,
} = useForm(profileInitialValues, profileValidations, updateProfile);

```

```

useEffect(() => {
 if (userDetails) {
 setUser(userDetails);
 }
}, [userDetails]);

```

```

const logout = () => {
 swal("Warning!", "You will be logged out", "warning", {
 buttons: {
 cancel: true,
 confirm: true,
 },
 dangerMode: true,
 }).then((confirm) => {
 if (confirm) {
 localStorage.removeItem("CF_authToken");
 setUserAuthToken("");
 navigate("/");
 }
 });
};

```

```

return (
 <div

```



```

 className={`w-full xs:w-3/4 md:w-1/3 h-full shadow shadow-Gray flex text-Black fixed
overflow-hidden transition-all duration-300 ease-in-out z-50 top-0 ${
 showProfileSlider ? "right-0" : "-right-full"
} ${
 theme === "light" ? "bg-White text-Black/80" : "bg-Black text-White/80"
}}
 >
 { /* Slider Navigation */ }
 <div className="bg-Purple w-12 h-full flex flex-col items-center justify-between pt-12 pb-4
px-2">
 <div>
 {TabOptions.map((option) => {
 return (
 <div
 key={option.id}
 className={`rounded-full p-2 mb-2 hover:bg-White hover:text-Purple flex
justify-center items-center cursor-pointer ${
 activeTab === option.id
 ? "bg-White text-Purple"
 : "text-White"
 } ${
 !user.isSeller &&
 option.id === "seller-admin-panel" &&
 "hidden"
 }`}
 onClick={() => {
 if (option.type === "tab") {
 setActiveTab(option.id);
 } else if (option.type === "link") {
 navigate(option.link);
 }
 }}
 >
 {option.icon}
 </div>
);
 })}
 </div>
 <div
 className="rounded-full p-2 text-White hover:bg-White hover:text-Purple flex
justify-center items-center content-end cursor-pointer"
 onClick={logout}
 >
 <FiLogout className="w-5 h-5 rotate-180" />

```

```

 </div>
 </div>
 <div className="px-4 py-2 w-full">
 <div
 className="w-7 h-7 bg-White p-2 rounded-full flex justify-center items-center
 cursor-pointer relative -left-[30px] shadow-lg"
 onClick={() => setShowProfileSlider(false)}
 >
 <ImCross className="w-5 h-5 text-Purple" />
 </div>
 <div className="flex items-center h-[95%] relative">
 <div
 className={`w-full h-full absolute md:p-2 rounded transition-all duration-300 ease-in-out
 overflow-y-scroll ${
 activeTab === "user" ? "right-0" : "-right-[120%]"
 }`}
 >
 <Profile
 user={user}
 setUser={setUser}
 errors={errors}
 handleChange={handleChange}
 handleSubmit={handleSubmit}
 isEditing={isEditing}
 setIsEditing={setIsEditing}
 />
 </div>
 <div
 className={`w-full h-full absolute md:p-2 rounded transition-all duration-300 ease-in-out
 overflow-y-scroll ${
 activeTab === "orders" ? "right-0" : "-right-[120%]"
 }`}
 >
 <MyOrders
 orders={user.orders || []}
 showProfileSlider={showProfileSlider}
 />
 </div>
 <div
 className={`w-full h-full absolute md:p-2 rounded transition-all duration-300 ease-in-out
 overflow-y-scroll ${
 activeTab === "seller" ? "right-0" : "-right-[120%]"
 }`}
 >

```

```

 {user.isSeller ? <UploadProduct /> : <SellerAccount />}
 </div>
 <div
 className={`w-full h-full absolute md:p-2 rounded transition-all duration-300 ease-in-out
overflow-y-scroll ${
 activeTab === "setting" ? "right-0" : "-right-[120%]"
 }}
 >
 <Settings user={user} />
 </div>
 </div>
 </div>
);
};
export default ProfileSlider;

```

Settings.jsx

```

import React from "react";
import ManageTheme from "../../GS-Libs/MultiUse/ManageTheme";
import ProfileDelete from "../ProfileDelete";
const Settings = ({ user }) => {
 return (
 <div className="flex flex-col gap-2 h-full w-full">
 <ManageTheme />
 <ProfileDelete user={user} />
 </div>
);
}
export default Settings;

```

SellerAccount.jsx

```

import React from "react";
import swal from "sweetalert";
import { useNavigate } from "react-router-dom";
import { Input } from "../../GS-Libs/Input/input";
import useForm from "../../hooks/useForm";
import {
 sellerInitailValues,
 sellerValidation,
} from "../../validations/seller-form";
import Button from "../../GS-Libs/Buttons/Button";
import { apiCaller } from "../../GS-Libs/utils/apiCaller";

export default function SellerAccount() {

```

```

const navigate = useNavigate();
const becomeSeller = async () => {
 try {
 await apiCaller("/become-seller", "post", {
 sellerEmail: sellerDetails.email,
 sellerPhoneNumber: sellerDetails.phoneNumber,
 sellerPANCardNumber: sellerDetails.panCardNumber,
 sellerGSTNumber: sellerDetails.gstNumber,
 });

 setSellerDetails({
 email: "",
 phoneNumber: "",
 panCardNumber: "",
 gstNumber: "",
 terms: false,
 });
 swal("Thanks!", "You become a seller now", "success").then(() => {
 navigate("/home");
 });
 } catch (error) {
 console.log(error);
 swal("Oops!", "Something is wrong", "error");
 }
};

const {
 formData: sellerDetails,
 setFormData: setSellerDetails,
 errors,
 setErrors,
 handleChange,
 handleSubmit,
} = useForm(sellerInitailValues, sellerValidation, becomeSeller);

return (
 <div className="h-full">
 <div className="text-2xl font-semibold pb-4">Become Seller</div>
 <div className="flex flex-col justify-between">
 <form onSubmit={handleSubmit}>
 <div className="flex flex-col gap-8">
 <div>
 <div className="text-Gray font-semibold">Contact Details</div>
 <div className="flex flex-col gap-2 w-full">

```

```
<Input
 type="text"
 name="email"
 placeholder="Enter Email"
 value={sellerDetails.email}
 onChange={handleChange}
 errorMessage={errors.email}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
/>
<Input
 type="number"
 name="phoneNumber"
 placeholder="Enter Mobile Number"
 value={sellerDetails.phoneNumber}
 onChange={handleChange}
 errorMessage={errors.phoneNumber}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
/>
</div>
</div>
<div>
 <div className="text-Gray font-semibold">Account Details</div>
 <div className="flex flex-col gap-2 w-full">
 <Input
 type="text"
 name="panCardNumber"
 placeholder="Enter PAN Card Number"
 value={sellerDetails.panCardNumber}
 onChange={handleChange}
 errorMessage={errors.panCardNumber}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 <Input
 type="text"
 name="gstNumber"
 placeholder="Enter GST Number"
 value={sellerDetails.gstNumber}
 onChange={handleChange}
 errorMessage={errors.gstNumber}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 </div>
</div>
<div className="flex gap-2 items-center">
```

```

 <input
 type="checkbox"
 name="terms"
 value={sellerDetails.terms}
 checked={sellerDetails.terms}
 onChange={(e) => {
 setSellerDetails((prev) => ({
 ...prev,
 terms: e.target.checked,
 }));

 if (e.target.checked) {
 setErrors((prev) => ({
 ...prev,
 terms: "",
 }));
 }
 }}
 />
 {errors.terms !== "" && (
 <div className="text-Red text-sm">{errors.terms}</div>
)}
 <div>Accept T&C to continue</div>
 </div>
 <div className="w-full md:w-1/3 absolute bottom-0 right-0">
 <Button text="Become Seller" type="submit" />
 </div>
</form>
</div>
</div>
);
}

```

Dashboard.jsx

import React from "react";

```

const DashboardCard = ({ heading }) => {
 return (
 <div className="h-40 p-4 shadow-md rounded-lg border border-Gray border-dashed
 hover:border-solid hover:border-Purple">
 <div className="text-xl font-medium border-b-2 border-Purple/70 pb-2 text-Purple">
 {heading}
 </div>
 </div>
)
}

```

```

 <div className="rounded-md bg-Yellow/80 flex items-center justify-center text-xl
font-semibold w-1/3 mt-2">
 ₹58243
 </div>
 </div>
);
};

```

```

);
};

const Dashboard = () => {
 return (
 <div className="grid gap-4 grid-cols-2 md:grid-cols-4">
 <DashboardCard heading="Total Earnings" />
 <DashboardCard heading="Ordered Products" />
 <DashboardCard heading="Total Uploaded Products" />
 <DashboardCard heading="Return Requests" />
 </div>
);
};

```

```
export default Dashboard;
```

EditProduct.jsx

```

import React, { useState } from "react";
import swal from "sweetalert";
import {
 Brands,
 BrandValues,
 Categories,
 CategoryValues,
 Colors,
 ColorValues,
 Genders,
 Materials,
 Sizes,
 SizeValues,
} from "../../config";
import { uploadProductValidation } from "../../validations/upload-product-validation";
import uploadImage from "../../GS-Libs/utlis/uploadImage";
import { apiCaller } from "../../GS-Libs/utlis/apiCaller";
import useForm from "../../hooks/useForm";
import FullScreenLoader from "../../GS-Libs/MultiUse/FullScreenLoader";
import Button from "../../GS-Libs/Buttons/Button";
import { Input } from "../../GS-Libs";
import Select from "../../GS-Libs/Input/Select";

```

```
import Checkbox from "../../GS-Libs/Input/Checkbox";
import ColorFilterCard from "../../Filters/ColorFilterCard";
import { Textarea } from "../../GS-Libs/Input/Textarea";
```

```
const EditProduct = ({ product, closeModal, showUpdatedProduct }) => {
 const [isProductUploading, setIsProductUploading] = useState(false);
 const [imageIndex, setImageIndex] = useState(0);
```

```
 const newInitailValues = {
 _id: product._id,
 name: product.name,
 price: product.price,
 brand: { name: product.brand, value: product.brand },
 category: { name: product.category, value: product.category },
 gender: product.gender,
 quantity: product.quantity,
 sizes: [...product.sizes.map((size) => SizeValues[size])],
 materials: product.materials,
 colors: [
 ...product.colors.map((color) => ColorValues[color.toLowerCase()]),
],
 description: product.description,
 productImages: [...product.productImages],
 };
};
```

```
const editProduct = async () => {
 setIsProductUploading(true);
 try {
 const { product } = await apiCaller(
 "/edit-product",
 "post",
 {
 productDetails: productDetails,
 },
 {},
 true
);
 }
};
```

```
 showUpdatedProduct(productDetails._id, product);
```

```
 swal("Thanks", "Your Product is edited!", "success").then(() => {
 closeModal();
 });
 setProductDetails(product);
```



```

 setImageIndex(0);
 setIsProductUploading(false);
 } catch (error) {
 console.log(error);
 swal("Oops", "Something went wrong", "error").then(() => {
 setProductDetails(product);
 setImageIndex(0);
 setIsProductUploading(false);
 });
 }
};

const {
 formData: productDetails,
 setFormData: setProductDetails,
 errors,
 setErrors,
 handleChange,
 handleSubmit,
} = useForm(newInitailValues, uploadProductValidation, editProduct);

if (isProductUploading) {
 return (
 <div className="pb-4 p-2 h-full relative flex items-center justify-center">
 <FullScreenLoader message="Editing Product..." />
 </div>
);
}

return (
 <>
 <div className="flex w-full gap-4">
 <div className="w-auto">
 <div className="pb-4 p-2 w-96 h-96 max-w-96 max-h-96">
 <label htmlFor="edit-product-image-upload">
 <div
 className={`bg-Black/40 w-full h-64 xs:h-80 sm:h-96 max-w-96 max-h-96 rounded-xl
absolute top-6 left-6 ${
 errors.productImages && "border border-Red"
 }`}
 >
 <input
 type="file"
 className="hidden"

```

```

id="edit-product-image-upload"
multiple={true}
onChange={(event) => {
 uploadImage(event).then((res) => {
 setProductDetails((prev) => ({
 ...prev,
 productImages: [...prev.productImages, ...res],
 }));
 });
});

if (errors.productImages) {
 setErrors((prev) => ({
 ...prev,
 productImages: "",
 }));
}
}}
/>
</div>
</label>

<div className="w-full h-[15rem] xs:h-[19rem] sm:h-[23rem] max-w-96 max-h-96">
 <img
 src={productDetails?.productImages?.[imageIndex]}
 alt=""
 className="w-full h-full rounded-lg block"
 />
</div>

<div className="pt-1">
 {errors.productImages && (

 {errors.productImages}

)}
</div>
</div>
{productDetails.productImages.length > 1 && (
 <div className="flex items-center gap-2 mt-2">
 <Button
 text="Prev"
 onClick={() => {
 if (imageIndex >= 1) {
 setImageIndex(imageIndex - 1);

```

```

 } else {
 setImageIndex(productDetails.productImages.length - 1);
 }
 }}
/>
<Button
 text="Next"
 onClick={() => {
 if (imageIndex < productDetails.productImages.length - 1) {
 setImageIndex(imageIndex + 1);
 } else {
 setImageIndex(0);
 }
 }}
/>
</div>
))
</div>

```

```

<div className="flex flex-col w-4/5 gap-4">
 <div className="w-full grid grid-cols-1 xs:grid-cols-2 gap-4">
 <div>
 <div className="text-lg font-semibold">Name</div>
 <Input
 type="text"
 name="name"
 placeholder="Product name"
 value={productDetails.name}
 onChange={handleChange}
 errorMessage={errors.name}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 </div>
 <div>
 <div className="text-lg font-semibold">Price</div>
 <Input
 type="number"
 name="price"
 placeholder="Product price"
 value={productDetails.price}
 onChange={handleChange}
 errorMessage={errors.price}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 </div>
 </div>
</div>

```

```

</div>
<div>
 <div className="text-lg font-semibold">Brand</div>
 <Select
 name="brand"
 onChange={(argument) => {
 handleChange({
 target: {
 name: "brand",
 value: {
 label: argument?.label?.toLowerCase(),
 value: argument?.label?.toLowerCase(),
 },
 type: "select",
 },
 });
 }}
 value={{
 label: BrandValues[productDetails.brand.name],
 value: productDetails.brand.name,
 }}
 options={Brands.map((brand) => ({
 label: brand.name,
 value: brand.id,
 })))
 placeholder="Select a brand"
 errorMessage={errors.brand}
 />
</div>
<div>
 <div className="text-lg font-semibold">Category</div>
 <Select
 name="category"
 onChange={(argument) => {
 handleChange({
 target: {
 name: "category",
 value: {
 label: argument?.label?.toLowerCase(),
 value: argument?.label?.toLowerCase(),
 },
 type: "select",
 },
 });
 }}
 >

```

```

 }}
 value={{
 label: CategoryValues[productDetails.category.name],
 value: productDetails.category.name,
 }}
 options={Categories.map((category) => ({
 label: category.name,
 value: category.id,
 })))}
 placeholder="Select a Category"
 errorMessage={errors.category}
 />
</div>
<div>
 <div className="text-lg font-semibold">Quantity</div>
 <Input
 type="number"
 name="quantity"
 placeholder="Product quantity"
 value={productDetails.quantity}
 onChange={handleChange}
 errorMessage={errors.quantity}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
</div>
</div>
<div className="w-full flex justify-between">
 <div>
 <div className="text-lg font-semibold">
 Genders{" "}
 {errors.gender && (

 ({errors.gender})

)}
 </div>
 <div className="flex gap-2">
 {Genders.map((option) => {
 const isGenderSelected =
 productDetails.gender.toLowerCase() ===
 option.name.toLowerCase();

 return (
 <div

```

```

 key={option.id}
 onClick={() => {
 handleChange({
 target: {
 name: "gender",
 value: isGenderSelected ? "" : option.name,
 type: "checkbox",
 },
 });
 }}
 >
 <Checkbox
 label={option.name}
 isSelected={isGenderSelected}
 />
 </div>
);
}}
</div>
</div>
<div>
 <div className="text-lg font-semibold">
 Sizes{" "}
 {errors.sizes && (

 ({errors.sizes})

)}
 </div>
 <div className="flex gap-2">
 {Sizes.map((option) => {
 const isSizeSelected = productDetails.sizes
 .map((size) => size.name?.toLowerCase())
 .includes(option.name?.toLowerCase());

 return (
 <div
 key={option.id}
 onClick={() => {
 handleChange({
 target: {
 name: "sizes",
 value: isSizeSelected
 ? [

```

```

 ...productDetails.sizes.filter(
 (size) => size.name !== option.name
),
]
 : [...productDetails.sizes, option].sort(
 (a, b) => a.id - b.id
),
 type: "checkbox",
 },
 });
}
}
>
<Checkbox
 label={option.name}
 isSelected={isSizeSelected}
/>
</div>
);
}}
</div>
</div>
<div>
 <div className="text-lg font-semibold">
 Materials{" "}
 {errors.materials && (

 {errors.materials}

)}
 </div>
 <div className="flex flex-wrap gap-2 w-full">
 {Materials.map((option) => {
 const isMaterialSelected =
 productDetails.materials.toLowerCase() ===
 option.name.toLowerCase();
 return (
 <div
 key={option.id}
 onClick={() => {
 handleChange({
 target: {
 name: "materials",
 value: isMaterialSelected ? "" : option.name,
 type: "checkbox",

```

```

 },
 });
 }}
 >
 <Checkbox
 label={option.name}
 isSelected={isMaterialSelected}
 />
 </div>
);
}}
</div>
</div>
</div>

<div className="w-full">
 <div className="text-lg font-semibold">
 Colors{" "}
 {errors.colors && (

 ({errors.colors})

)}
 </div>
 <div className="grid grid-cols-2 sm:grid-cols-4 md:grid-cols-8 gap-2 w-full">
 {Colors.map((color) => {
 const isColorSelected = productDetails?.colors
 .map((color) => color?.color?.toLowerCase())
 .includes(color?.color.toLowerCase());
 return (
 <ColorFilterCard
 key={color.id}
 color={color.color}
 onClick={() => {
 handleChange({
 target: {
 name: "colors",
 value: isColorSelected
 }
 ? [
 ...productDetails.colors.filter(
 (productColor) =>
 productColor.name.toLowerCase() !==
 color.name.toLowerCase()
),
],
)
 }}
 />
);
 })}
 </div>

```



```

]
 : [
 ...productDetails.colors,
 {
 name: color.name,
 color: color.color,
 id: color.id,
 },
],
 type: "checkbox",
 },
 });
}
isSelected={isColorSelected}
/>
);
}}}
</div>
</div>
</div>
</div>
<div className="flex flex-col gap-y-4 mt-4">
 <div className="w-full flex flex-col">
 <div className="text-lg font-semibold">Description</div>
 <Textarea
 rows="5"
 placeholder="As always great desgin and quality..."
 name="description"
 onChange={handleChange}
 value={productDetails.description}
 errorMessage={errors.description}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 </div>
</div>
<div className="w-full md:w-1/12 absolute right-6 bottom-6">
 <Button text="Edit Product" onClick={handleSubmit} type="button" />
</div>
</>
);
};

export default EditProduct;

```

Products.jsx

```
import React, { useEffect, useState } from "react";
import CompleteTable from "../../GS-Libs/Table/CompleteTable";
import { apiCaller } from "../../GS-Libs/utils/apiCaller";
```

```
const columns = [
 {
 id: "id",
 label: "ID",
 className: "w-8 text-center",
 },
 {
 id: "image",
 label: "Image",
 className: "w-10",
 },
 {
 id: "name",
 label: "Name",
 className: "w-64 max-w-64",
 },
 {
 id: "brand",
 label: "Brand",
 className: "w-40",
 },
 {
 id: "material",
 label: "Material",
 className: "w-40",
 },
 {
 id: "price",
 label: "Price",
 className: "w-20 text-center",
 },
 {
 id: "quantity",
 label: "Quantity",
 className: "w-20 text-center",
 },
 {
 id: "status",
 label: "Status",
 },
]
```

```

 className: "w-24 text-center",
 },
 // {
 // id: "edit",
 // label: "Edit",
 // },
 // {
 // id: "delete",
 // label: "Delete",
 // },
];

const Products = () => {
 const [tableData, setTableData] = useState([]);

 const getProducts = async () => {
 const data = await apiCaller(
 "/get-products-by-seller",
 "get",
 {},
 {},
 true
);

 setTableData(data.products);
 };

 useEffect(() => {
 getProducts();
 }, []);

 return (
 <div>
 <div className="text-2xl font-semibold">All Products</div>
 <CompleteTable columns={columns} data={tableData} />
 </div>
);
};

export default Products;

```

SellerDashboard.jsx

```

import React, { useState } from "react";
import SellerNavigation from "../SellerNavigation";

```

```

import Dashboard from "../Dashboard/Dashboard";
import Orders from "../Orders/Orders";
import Products from "../Products/Products";

const SellerDashboard = () => {
 const [selectedNavbarItem, setSelectedNavbarItem] = useState("products");

 const getComponent = (item) => {
 switch (item) {
 case "dashboard":
 return <Dashboard />;
 case "orders":
 return <Orders />;
 case "products":
 return <Products />;
 case "reviews":
 return <div>Reviews</div>;
 case "return-orders":
 return <div>Return Orders</div>;
 default:
 return <div>Not Found</div>;
 }
 };

 return (
 <div className="h-full flex">
 <div className="w-full xs:w-3/5 md:w-1/4 xl:w-1/5">
 { /* Left Section (Navbar) */ }
 <SellerNavigation
 selectedNavbarItem={selectedNavbarItem}
 setSelectedNavbarItem={setSelectedNavbarItem}
 />
 </div>

 { /* Right Section */ }
 <div className="w-full md:w-3/4 xl:w-4/5 p-4">
 {getComponent(selectedNavbarItem)}
 </div>
 </div>
);
};

export default SellerDashboard;

```

SellerNavigation.jsx

```
import React from "react";
import { MdSpaceDashboard, MdInsertComment } from "react-icons/md";
import { FaShoppingBag } from "react-icons/fa";
import { HiOutlineUpload } from "react-icons/hi";
import { TbTruckReturn } from "react-icons/tb";
import { useTheme } from "../../context/themeContext";

const SellerNavigationItemList = [
 // {
 // id: "dashboard",
 // name: "Dashboard",
 // Icon: <MdSpaceDashboard className="w-5 h-5" />,
 // },
 // {
 // id: "orders",
 // name: "Orders",
 // Icon: <FaShoppingBag className="w-5 h-5" />,
 // },
 {
 id: "products",
 name: "Products",
 Icon: <HiOutlineUpload className="w-5 h-5" />,
 },
 // {
 // id: "reviews",
 // name: "Reviews",
 // Icon: <MdInsertComment className="w-5 h-5" />,
 // },
 // {
 // id: "return-orders",
 // name: "Return orders",
 // Icon: <TbTruckReturn className="w-5 h-5" />,
 // },
];

const SellerNavigationItem = ({ Icon, name, isActive }) => {
 return (
 <div
 className={`flex items-center p-2 rounded-md cursor-pointer shadow-md transition-all
duration-150 ${
 isActive
 ? "bg-Purple text-White"
 }
 `}
 >
```

```
 : "hover:text-Purple border border-Gray border-dashed hover:border-solid
hover:border-Purple"
```

```
 }}
 >
 <div className="mr-2">{Icon}</div>
 <div className="font-medium uppercase">{name}</div>
 </div>
);
};
```

```
const SellerNavigation = ({ selectedNavbarItem, setSelectedNavbarItem }) => {
 const { theme } = useTheme();
```

```
 const handleSelectNavbarItem = (id) => {
 setSelectedNavbarItem(id);
 };
```

```
 return (
 <div
 className={`w-full h-full p-2 border-r ${
 theme === "light"
 ? "border-Black/40 bg-White text-Black"
 : "border-White/40 bg-Black text-White"
 }}
 >
 <div className="text-xl font-semibold mb-2 cursor-default border-b border-Gray pb-2">
 Admin Panel
 </div>
 <div className="flex flex-col gap-2">
 {SellerNavigationItemList.map((item) => {
 return (
 <div onClick={() => handleSelectNavbarItem(item.id)} key={item.id}>
 <SellerNavigationItem
 {...item}
 isActive={item.id === selectedNavbarItem}
 />
 </div>
);
 })}
 </div>
 </div>
);
};
```

```
export default SellerNavigation;
```

```
UploadProduct.jsx
```

```
import React, { useState } from "react";
import swal from "sweetalert";
import {
 Brands,
 Categories,
 Colors,
 Genders,
 Materials,
 Sizes,
} from ".././config";
import Checkbox from ".././GS-Libs/Input/Checkbox";
import Button from ".././GS-Libs/Buttons/Button";
import ColorFilterCard from "../Filters/ColorFilterCard";
import useForm from ".././hooks/useForm";
import {
 initailProductValues,
 uploadProductValidation,
} from ".././validations/upload-product-validation";
import { Input } from ".././GS-Libs/Input/input";
import Select from ".././GS-Libs/Input/Select";
import { Textarea } from ".././GS-Libs/Input/Textarea";
import { apiCaller } from ".././GS-Libs/utils/apiCaller";
import FullScreenLoader from ".././GS-Libs/MultiUse/FullScreenLoader";
import uploadImage from ".././GS-Libs/utils/uploadImage";
```

```
export default function UploadProduct() {
 const [isProductUploading, setIsProductUploading] = useState(false);
 const [imageIndex, setImageIndex] = useState(0);
```

```
 const uploadProduct = async () => {
 setIsProductUploading(true);
 try {
 await apiCaller("/upload-product", "post", {
 productDetails: productDetails,
 });

 swal("Thanks", "Your Product is uploaded!", "success");
 setProductDetails(initailProductValues);
 setImageIndex(0);
 setIsProductUploading(false);
 } catch (error) {
```

```

 swal("Oops", "Something went wrong", "error").then(() => {
 setProductDetails(initailProductValues);
 setImageIndex(0);
 setIsProductUploading(false);
 });
 }
 };

```

```

const {
 formData: productDetails,
 setFormData: setProductDetails,
 errors,
 setErrors,
 handleChange,
 handleSubmit,
} = useForm(initailProductValues, uploadProductValidation, uploadProduct);

```

```

if (isProductUploading) {
 return (
 <div className="pb-4 p-2 h-full relative flex items-center justify-center">
 <FullScreenLoader message="Uploading Product" />
 </div>
);
}

```

```

return (
 <>
 <div className="pb-4 p-2 px-0.5">
 <label htmlFor="product-image-upload">
 <div
 className={`bg-Black/40 w-full h-64 xs:h-80 sm:h-96 rounded-xl absolute top-2 left-0 ${
 errors.productImages && "border border-Red"
 }`}
 >
 <input
 type="file"
 className="hidden"
 id="product-image-upload"
 multiple={true}
 onChange={(event) => {
 uploadImage(event).then((res) => {
 setProductDetails((prev) => ({
 ...prev,
 productImages: res,

```



```

 });
 });
 if (errors.productImages) {
 setErrors((prev) => ({
 ...prev,
 productImages: "",
 }));
 }
}}
/>
</div>
</label>

<div className="w-full h-[15rem] xs:h-[19rem] sm:h-[23rem]">
 <img
 src={productDetails.productImages[imageIndex]}
 alt=""
 className="w-full h-full rounded-lg block"
 />
</div>
<div className="pt-1">
 {errors.productImages && (

 {errors.productImages}

)}
</div>
</div>
{productDetails.productImages.length > 1 && (
 <div className="flex items-center gap-2 mb-2">
 <Button
 text="Prev"
 onClick={() => {
 if (imageIndex >= 1) {
 setImageIndex(imageIndex - 1);
 } else {
 setImageIndex(productDetails.productImages.length - 1);
 }
 }}
 />
 <Button
 text="Next"
 onClick={() => {
 if (imageIndex < productDetails.productImages.length - 1) {

```

```

 setImageIndex(imageIndex + 1);
 } else {
 setImageIndex(0);
 }
 }}
/>
</div>
)}}
<>
<div className="grid grid-cols-1 xs:grid-cols-2 gap-x-2 gap-y-4 items-center">
 <div>
 <div className="text-lg font-semibold">Name</div>
 <Input
 type="text"
 name="name"
 placeholder="Product name"
 value={productDetails.name}
 onChange={handleChange}
 errorMessage={errors.name}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 </div>
 <div>
 <div className="text-lg font-semibold">Price</div>
 <Input
 type="number"
 name="price"
 placeholder="Product price"
 value={productDetails.price}
 onChange={handleChange}
 errorMessage={errors.price}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 </div>
 <div>
 <div className="text-lg font-semibold">Brand</div>
 <Select
 name="brand"
 onChange={(argument) => {
 handleChange({
 target: {
 name: "brand",
 value: {
 label: argument?.label?.toLowerCase(),

```

```

 value: argument?.label?.toLowerCase(),
 },
 type: "select",
 },
 });
}}
value={productDetails.brand.name}
options={Brands.map((brand) => ({
 label: brand.name,
 value: brand.id,
})))}
placeholder="Select a brand"
errorMessage={errors.brand}
/>
</div>
<div>
 <div className="text-lg font-semibold">Category</div>
 <Select
 name="category"
 onChange={(argument) => {
 handleChange({
 target: {
 name: "category",
 value: {
 label: argument?.label?.toLowerCase(),
 value: argument?.label?.toLowerCase(),
 },
 type: "select",
 },
 });
 }}
 value={productDetails.category.name}
 options={Categories.map((category) => ({
 label: category.name,
 value: category.id,
 })))}
 placeholder="Select a Category"
 errorMessage={errors.category}
 />
</div>
<div className="">
 <div className="text-lg font-semibold">
 Genders{" "}
 {errors.gender && (

```

```


 {errors.gender}

)}
 </div>
 <div className="flex gap-2">
 {Genders.map((option) => {
 const isGenderSelected = productDetails.gender === option.name;
 return (
 <div
 key={option.id}
 onClick={() => {
 handleChange({
 target: {
 name: "gender",
 value: isGenderSelected ? "" : option.name,
 type: "checkbox",
 },
 });
 }}
 >
 <Checkbox
 label={option.name}
 isSelected={isGenderSelected}
 />
 </div>
);
 })}
 </div>
 <div className="">
 <div className="text-lg font-semibold">
 Sizes{" "}
 {errors.sizes && (

 {errors.sizes}

)}
 </div>
 <div className="flex gap-2">
 {Sizes.map((option) => {
 const isSizeSelected = productDetails.sizes
 .map((size) => size.name)
 .includes(option.name);

```

```

return (
 <div key={option.id}>
 <Checkbox
 label={option.name}
 isSelected={isSizeSelected}
 onClick={() => {
 handleChange({
 target: {
 name: "sizes",
 value: isSizeSelected
 ? [
 ...productDetails.sizes.filter(
 (size) => size.name !== option.name
),
]
 : [...productDetails.sizes, { ...option }].sort(
 (a, b) => a.id - b.id
),
 type: "checkbox",
 },
 });
 }}
 />
 </div>
);
}}
</div>
</div>
</div>
<div className="flex flex-col gap-y-4">
 <div className="flex flex-col mt-4">
 <div className="text-lg font-semibold">
 Materials{" "}
 {errors.materials && (

 ({errors.materials})

)}
 </div>
 <div className="flex flex-wrap gap-2 w-full">
 {Materials.map((option) => {
 const isMaterialSelected =
 productDetails.materials === option.name;
 return (

```



```

 (productColor) =>
 productColor.name.toLowerCase() !==
 color.name.toLowerCase()
),
]
 : [
 ...productDetails.colors,
 {
 name: color.name.toLowerCase(),
 color: color.color,
 id: color.id,
 },
],
 type: "checkbox",
},
});
}}
isSelected={isColorSelected}
/>
);
}}
</div>
</div>
<div className="w-full flex flex-col">
 <div className="text-lg font-semibold">Description</div>
 <Textarea
 rows="5"
 placeholder="As always great desgin and quality..."
 name="description"
 onChange={handleChange}
 value={productDetails.description}
 errorMessage={errors.description}
 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
</div>
<div className="w-full flex flex-col">
 <div className="text-lg font-semibold">Quantity</div>
 <Input
 type="number"
 name="quantity"
 placeholder="Product quantity"
 value={productDetails.quantity}
 onChange={handleChange}
 errorMessage={errors.quantity}
 />
</div>

```

```

 className="p-2 border-2 border-Black/20 bg-Gray/10 rounded text-Black w-full"
 />
 </div>
 </div>
 <div className="flex justify-end w-full pt-5">
 <div className="w-full md:w-1/3">
 <Button
 text="Submit Product"
 onClick={handleSubmit}
 type="button"
 />
 </div>
 </div>
</>
</>
);
}

```

searchContext.jsx

```
import React, { createContext, useContext, useState } from "react";
```

```
const SearchContext = createContext();
```

```
export const useSearchContext = () => {
 return useContext(SearchContext);
};
```

```
export const SearchContextProvider = ({ children }) => {
 const [searchQuery, setSearchQuery] = useState("");

 return (
 <SearchContext.Provider value={{ searchQuery, setSearchQuery }}>
 {children}
 </SearchContext.Provider>
);
};
```

themeContext.jsx

```
import React, { createContext, useState, useEffect, useContext } from "react";
```

```
const ThemeContext = createContext();
```

```
export const ThemeProvider = ({ children }) => {
 // Check the initial theme preference from localStorage

```



```

const [theme, setTheme] = useState(() => {
 return localStorage.getItem("cf-theme") || "light";
});

useEffect(() => {
 // Save the theme preference to localStorage
 localStorage.setItem("cf-theme", theme);
}, [theme]);

const toggleTheme = () => {
 setTheme((prevTheme) => (prevTheme === "light" ? "dark" : "light"));
};

return (
 <ThemeContext.Provider value={{ theme, setTheme, toggleTheme }}>
 {children}
 </ThemeContext.Provider>
);
};

export const useTheme = () => useContext(ThemeContext);

```

#### [useAPI.js](#)

```

import { useState, useEffect, useCallback } from "react";
import axios from "axios";
import { SERVER_URL } from "../config";

```

```

const useAPI = (
 method,
 path,
 includeAuthToken = true,
 variables = {},
 headers = {},
 dependencies = []
) => {
 const [data, setData] = useState(null);
 const [loading, setLoading] = useState(false);
 const [error, setError] = useState(null);

 const fetchData = useCallback(async () => {
 setLoading(true);
 setError(null);

 try {

```

```

const response = await axios({
 method,
 url: `${SERVER_URL}${path}`,
 data: method === "GET" ? null : { ...variables },
 params: method === "GET" ? { ...variables } : null,
 headers: { ...headers },
});
setData(response.data);
} catch (err) {
 setError(err);
} finally {
 setLoading(false);
}
}, [method, path, JSON.stringify(variables), JSON.stringify(headers)]);

useEffect(() => {
 if (includeAuthToken) {
 fetchData();
 }
}, [fetchData, ...dependencies]);

return { data, loading, error };
};

export default useAPI;

```

[useDebounce.js](#)

```
import { useState, useEffect } from "react";
```

```

const useDebounce = (callback, delay, dependencies = []) => {
 const [data, setData] = useState(null);
 const [timerId, setTimerId] = useState(null);

 useEffect(() => {
 if (timerId) {
 clearTimeout(timerId);
 }

 const newTimerId = setTimeout(async () => {
 const data = await callback();
 setData(data);
 }, delay);

 setTimerId(newTimerId);
 }, dependencies);
}

```

```

 return () => {
 clearTimeout(newTimerId);
 };
 }, [...dependencies, delay]); // eslint-disable-line react-hooks/exhaustive-deps

 return { timerId, data };
};

export default useDebounce;

```

#### [useDebounceAPI.js](#)

```

import { useState, useEffect, useCallback, useRef } from "react";
import axios from "axios";
import { SERVER_URL } from "../config";

```

```

const useDebounceAPI = (
 method,
 path,
 variables = {},
 headers = {},
 dependencies = [],
 debounceDelay = 200
) => {
 const [data, setData] = useState(null);
 const [loading, setLoading] = useState(false);
 const [error, setError] = useState(null);
 const debounceTimeout = useRef(null);

 const fetchData = useCallback(async () => {
 setLoading(true);
 setError(null);

 try {
 const response = await axios({
 method,
 url: `${SERVER_URL}${path}`,
 data: method === "GET" ? null : { ...variables },
 params: method === "GET" ? { ...variables } : null,
 headers: { ...headers },
 });
 setData(response.data);
 } catch (err) {
 setError(err);
 }
 }, dependencies);

```

```

 } finally {
 setLoading(false);
 }
 }, [method, path, JSON.stringify(variables), JSON.stringify(headers)]);

 useEffect(() => {
 if (debounceTimeout.current) {
 clearTimeout(debounceTimeout.current);
 }

 debounceTimeout.current = setTimeout(fetchData, debounceDelay);

 return () => {
 if (debounceTimeout.current) {
 clearTimeout(debounceTimeout.current);
 }
 };
 }, [fetchData, debounceDelay, ...dependencies]);

 return { data, loading, error };
};

export default useDebouncedAPI;

```

### [useForm.js](#)

```

import { useState } from "react";

const useForm = (initialValues, validationRules, onSubmit) => {
 const [formData, setFormData] = useState(initialValues);
 const [errors, setErrors] = useState({});
 const [isSubmitting, setIsSubmitting] = useState(false);

 const handleChange = (e) => {
 const { name, value } = e.target;
 setFormData((prev) => ({ ...prev, [name]: value }));
 setErrors((prev) => ({ ...prev, [name]: undefined }));
 };

 const handleSubmit = (e) => {
 e.preventDefault();
 setIsSubmitting(true);

 if (validationRules) {
 const newErrors = {};

```

```

Object.keys(validationRules).forEach((key) => {
 const rule = validationRules[key];
 if (rule && typeof rule === "function") {
 const errorMessage = rule(formData[key], formData);
 if (errorMessage) {
 newErrors[key] = errorMessage;
 }
 }
});
setErrors(newErrors);
const hasErrors = Object.keys(newErrors).length > 0;
if (hasErrors) {
 setIsSubmitting(false);
 return;
}

if (onSubmit) {
 onSubmit(formData);
}

setIsSubmitting(false);
};

return {
 formData,
 errors,
 isSubmitting,
 setFormData,
 setErrors,
 handleChange,
 handleSubmit,
};
};

export default useForm;

```

[buy-product-page-validation.js](#)

```

export const buyProductPageUserValidations = {
 name: (value) => {
 if (!value) {
 return "Name is required";
 }
 },
};

```

```

email: (value) => {
 if (!value) {
 return "Email is required";
 }
 const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 if (!emailRegex.test(value)) {
 return "Invalid email format";
 }
},
phoneNumber: (value) => {
 if (!value) {
 return "Phone number is required";
 }
 if (value.length !== 10) {
 return "Phone number must be 10 digits long";
 }
 const phoneRegex = /^[0-9]{10,15}$/;
 if (!phoneRegex.test(value)) {
 return "Invalid phone number format";
 }
},
address: (value) => {
 console.log("Address", value);
 if (!value) {
 return "Address is required";
 }
},
upildValue: (value, formData) => {
 if (!value && !formData.cardNumber && formData.paymentMethod !== "cod") {
 return "UPI ID is required";
 }
 if (value) {
 const upiRegex = /^[a-zA-Z0-9.\-_{2,256}@[a-zA-Z]{2,64}$/;
 if (!upiRegex.test(value)) {
 return "Invalid UPI ID format";
 }
 }
},
cardNumber: (value, formData) => {
 if (!value && !formData.upildValue && formData.paymentMethod !== "cod") {
 return "Card Number is required";
 }
 if (value) {
 const cardNumberRegex = /^[0-9]{13,19}$/;

```

```

 if (!cardNumberRegex.test(value)) {
 return "Invalid card number format";
 }
 },
 validateDate: (value, formData) => {
 if (!value && !formData.upildValue && formData.paymentMethod !== "cod") {
 return "Card valid date is required";
 }
 if (value) {
 const validateDateRegex = /^(0[1-9]|1[0-2])\/?([0-9]{4}|[0-9]{2})$/;
 if (!validateDateRegex.test(value)) {
 return "Invalid date format";
 }
 }
 },
 cvc: (value, formData) => {
 if (!value && !formData.upildValue && formData.paymentMethod !== "cod") {
 return "CVC is required";
 }
 if (value) {
 const cvcRegex = /^[0-9]{3,4}$/;
 if (!cvcRegex.test(value)) {
 return "Invalid CVC format";
 }
 }
 },
};

```

```

export const buyProductPageUserInitailValues = {
 name: "",
 email: "",
 phoneNumber: "",
 address: "",
 paymentMethod: "",
 upildValue: "",
 cardNumber: "",
 validateDate: "",
 cvc: "",
};

```

[login-validation.js](#)

```

export const loginValidations = {
 email: (value) => {

```

```

 if (!value) {
 return "Email is required";
 }
 const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 if (!emailRegex.test(value)) {
 return "Invalid email format";
 }
 },
 password: (value) => {
 if (!value) {
 return "Password is required";
 }
 if (value.length < 6) {
 return "Password must be at least 6 characters";
 }
 },
};

```

```

export const loginInitailValues = {
 email: "",
 password: "",
};

```

#### [profile-form.js](#)

```

export const profileValidations = {
 name: (value) => {
 if (!value) {
 return "User name is required";
 }
 },
 email: (value) => {
 if (!value) {
 return "Email is required";
 }
 const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 if (!emailRegex.test(value)) {
 return "Invalid email format";
 }
 },
 phoneNumber: (value) => {
 if (value !== "" && value.length !== 10) {
 return "Phone number must be 10 digits long";
 }
 const phoneRegex = /^[6-9]\d{9}$/;

```



```

 if (value !== "" && !phoneRegex.test(value)) {
 return "Invalid phone number format";
 }
 },
 website: (value) => {
 const urlRegex =
 /^((https?:\/\/)?([\da-z.-]+)\.([a-z.]{2,6})([/\w .-]*)*\/?$)/;
 if (value !== "" && !urlRegex.test(value)) {
 return "Invalid URL format";
 }
 },
};

```

```

export const profileInitailValues = {
 name: "",
 email: "",
 phoneNumber: "",
 address: "",
 profileImage: "",
 website: "",
 // cart: [],
 orders: [],
 isSeller: false,
};

```

#### [seller-form.js](#)

```

export const sellerValidation = {
 email: (value) => {
 if (!value) {
 return "Email is required";
 }
 if (value.length < 3) {
 return "Email must be at least 3 characters long";
 }
 const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 if (!emailRegex.test(value)) {
 return "Invalid email format";
 }
 },

 phoneNumber: (value) => {
 if (!value) {
 return "Phone number is required";
 }
 }
}

```

```

 if (value.length !== 10) {
 return "Phone number must be 10 digits long";
 }
 const phoneRegex = /^[6-9]\d{9}$/;
 if (!phoneRegex.test(value)) {
 return "Invalid phone number format";
 }
 },

 panCardNumber: (value) => {
 if (!value) {
 return "PAN card number is required";
 }
 const panRegex = /^[A-Z]{5}[0-9]{4}[A-Z]{1}$/;
 if (!panRegex.test(value)) {
 return "Invalid PAN card number format";
 }
 },

 gstNumber: (value) => {
 if (!value) {
 return "GST number is required";
 }
 const gstRegex =
 /^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}[Z]{1}[0-9A-Z]{1}$/;
 if (!gstRegex.test(value)) {
 return "Invalid GST number format";
 }
 },

 terms: (value) => {
 if (!value) {
 return "Accept to continue";
 }
 },
};

export const sellerInitailValues = {
 email: "",
 phoneNumber: "",
 panCardNumber: "",
 gstNumber: "",
 terms: false,
};

```

### [signup-validation.js](#)

```
export const signupValidations = {
 name: (value) => {
 if (!value) {
 return "Username is required";
 }
 },
 email: (value) => {
 if (!value) {
 return "Email is required";
 }
 const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 if (!emailRegex.test(value)) {
 return "Invalid email format";
 }
 },
 password: (value) => {
 if (!value) {
 return "Password is required";
 }
 if (value.length < 6) {
 return "Password must be at least 6 characters";
 }
 },
};
```

```
export const signupInitailValues = {
 name: "",
 email: "",
 password: "",
};
```

### [upload-product-validation.js](#)

```
import NotFoundImage from "../Assets/images/not found.jpg";
```

```
export const uploadProductValidation = {
 name: (value) => {
 if (!value) {
 return "Name is required";
 }
 },
 price: (value) => {
 if (!value) {
```

```

 return "Price is required";
 }
 if (isNaN(value)) {
 return "Price must be a number";
 }
 if (value <= 0) {
 return "Price must be greater than zero";
 }
 if (value > 1000000) {
 return "Price must be less than 1 million";
 }
},
brand: (value) => {
 if (!value.value) {
 return "Brand is required";
 }
},
gender: (value) => {
 if (!value) {
 return "Gender is required";
 }
},
category: (value) => {
 if (!value.value) {
 return "Category is required";
 }
},
materials: (value) => {
 if (!value) {
 return "Material is required";
 }
},
description: (value) => {
 if (!value) {
 return "Description is required";
 }
 if (value.length < 10) {
 return "Description must be at least 10 characters long";
 }
 if (value.length > 500) {
 return "Description must be less 500 characters";
 }
},
quantity: (value) => {

```

```

 if (!value) {
 return "Quantity is required";
 }
 if (value < 0) {
 return "Quantity must be greater than zero";
 }
 },
 sizes: (value) => {
 if (!Array.isArray(value) || value.length === 0) {
 return "Size is required";
 }
 },
 colors: (value) => {
 if (!Array.isArray(value) || value.length === 0) {
 return "At least one color is required";
 }
 },
 productImages: (value) => {
 if (!Array.isArray(value) || value.length === 0) {
 return "At least one product image is required";
 }
 if (value.length === 1 && value.includes(NotFoundImage)) {
 return "At least one product image is required";
 }
 },
};

```

```

export const initailProductValues = {
 name: "",
 price: 0,
 brand: {
 name: "",
 value: "",
 },
 gender: "",
 category: {
 name: "",
 value: "",
 },
 materials: "",
 description: "",
 quantity: 0,
 sizes: [],
 colors: [],
};

```

```
 productImages: [NotFoundImage],
 };
}
```

#### [user-profile-validation.js](#)

```
export const profileValidations = {
 name: (value) => {
 if (!value) {
 return "Name is required";
 }
 },

 email: (value) => {
 if (!value) {
 return "Email is required";
 }
 if (value.length < 3) {
 return "Email must be at least 3 characters long";
 }
 const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 if (!emailRegex.test(value)) {
 return "Invalid email format";
 }
 },

 phoneNumber: (value) => {
 if (!value) {
 return "Phone number is required";
 }
 if (value.length !== 10) {
 return "Phone number must be 10 digits long";
 }
 const phoneRegex = /^[6-9]\d{9}$/;
 if (!phoneRegex.test(value)) {
 return "Invalid phone number format";
 }
 },

 address: (value) => {
 if (!value) {
 return "Address is required";
 }
 },
};
```

```
export const profileInitailValues = {
 name: "",
 email: "",
 phoneNumber: "",
 address: "",
 profileImage: "images/man.png",
 website: "",
};
```

[config.js](#)

```
import LinkedInImage from "../Assets/images/linkedin.png";
import GithubImage from "../Assets/images/github.png";
```

```
export const BrandValues = {
 adidas: "Adidas",
 nike: "Nike",
 puma: "Puma",
 gucci: "Gucci",
 dior: "Dior",
 zara: "Zara",
 "h&b": "H&B",
 "calvin klein": "Calvin Klein",
 "louis vuitton": "Louis Vuitton",
};
```

```
export const CategoryValues = {
 "t-shirt": "T-Shirt",
 shirt: "Shirt",
 sweatshirt: "Sweatshirt",
 jacket: "Jacket",
 hoodie: "Hoodie",
 shoes: "Shoes",
 trousers: "Trousers",
};
```

```
export const MaterialValues = {
 cotton: "Cotton",
 polyester: "Polyester",
 leather: "Leather",
 denim: "Denim",
 wool: "Wool",
 silk: "Silk",
 linen: "Linen",
};
```

```
export const SizeValues = {
 s: { id: 1, name: "S", },
 m: { id: 2, name: "M", },
 l: { id: 3, name: "L", },
 xl: { id: 4, name: "XL", },
 xxl: { id: 5, name: "XXL", },
};
```

```
export const GenderValues = {
 men: "Men",
 women: "Women",
 unisex: "Unisex",
};
```

```
export const ColorValues = {
 white: {
 id: "white",
 name: "White",
 color: "white",
 },
 black: {
 id: "black",
 name: "Black",
 color: "black",
 },
 navy: {
 id: "navy",
 name: "Navy",
 color: "navy",
 },
 skyblue: {
 id: "skyBlue",
 name: "Sky Blue",
 color: "skyBlue",
 },
 green: {
 id: "green",
 name: "Green",
 color: "green",
 },
 red: {
 id: "red",
 name: "Red",
 },
};
```



```
 color: "red",
 },
 mustard: {
 id: "mustard",
 name: "Mustard",
 color: "mustard",
 },
 coral: {
 id: "coral",
 name: "Coral",
 color: "coral",
 },
 lavender: {
 id: "lavender",
 name: "Lavender",
 color: "lavender",
 },
 teal: {
 id: "teal",
 name: "Teal",
 color: "teal",
 },
 olive: {
 id: "olive",
 name: "Olive",
 color: "olive",
 },
 charcoal: {
 id: "charcoal",
 name: "Charcoal",
 color: "charcoal",
 },
};
```

```
export const Brands = [
 { id: 1, name: "Adidas", },
 { id: 2, name: "Nike", },
 { id: 3, name: "Puma", },
 { id: 4, name: "Gucci", },
 { id: 5, name: "Dior", },
 { id: 6, name: "Zara", },
 { id: 7, name: "H&B", },
 { id: 8, name: "Calvin Klein", },
 { id: 9, name: "Louis Vuitton", },
```

```
];
```

```
export const Colors = [
```

```
{
 id: "white",
 name: "White",
 color: "white",
},
{
 id: "black",
 name: "Black",
 color: "black",
},
{
 id: "navy",
 name: "Navy",
 color: "navy",
},
{
 id: "skyBlue",
 name: "Sky Blue",
 color: "skyBlue",
},
{
 id: "green",
 name: "Green",
 color: "green",
},
{
 id: "red",
 name: "Red",
 color: "red",
},
{
 id: "mustard",
 name: "Mustard",
 color: "mustard",
},
{
 id: "coral",
 name: "Coral",
 color: "coral",
},
{
```

```
 id: "lavender",
 name: "Lavender",
 color: "lavender",
 },
 {
 id: "teal",
 name: "Teal",
 color: "teal",
 },
 {
 id: "olive",
 name: "Olive",
 color: "olive",
 },
 {
 id: "charcoal",
 name: "Charcoal",
 color: "charcoal",
 },
];
```

```
export const Sizes = [
 { id: 1, name: "S", },
 { id: 2, name: "M", },
 { id: 3, name: "L", },
 { id: 4, name: "XL", },
 { id: 5, name: "XXL", },
];
```

```
export const Discounts = [
 { id: 1, name: "10% Off" },
 { id: 2, name: "20% Off" },
 { id: 3, name: "30% Off" },
 { id: 4, name: "50% Off" },
];
```

```
export const Genders = [
 { id: 1, name: "Men" },
 { id: 2, name: "Women" },
 { id: 3, name: "Unisex" },
];
```

```
export const Materials = [
 { id: 1, name: "Cotton" },
```

```
{ id: 2, name: "Polyester" },
{ id: 3, name: "Leather" },
{ id: 4, name: "Denim" },
{ id: 5, name: "Wool" },
{ id: 6, name: "Silk" },
{ id: 7, name: "Linen" },
];
```

```
export const Ratings = [
 { id: 1, name: "1 Star & Up" },
 { id: 2, name: "2 Stars & Up" },
 { id: 3, name: "3 Stars & Up" },
 { id: 4, name: "4 Stars & Up" },
 { id: 5, name: "5 Stars" },
];
```

```
export const Categories = [
 { id: "1", name: "T-Shirt" },
 { id: "2", name: "Shirt" },
 { id: "3", name: "Sweatshirt" },
 { id: "4", name: "Jacket" },
 { id: "5", name: "Hoodie" },
 { id: "6", name: "Shoes" },
 { id: "7", name: "Trousers" },
 { id: "8", name: "Other" },
];
```

```
export const Quantity = [
 { value: 1, label: "1" },
 { value: 2, label: "2" },
 { value: 3, label: "3" },
 { value: 4, label: "4" },
 { value: 5, label: "5" },
];
```

```
export const DaysOfWeek = [
 "Sunday",
 "Monday",
 "Tuesday",
 "Wednesday",
 "Thursday",
 "Friday",
 "Saturday",
];
```

```
export const Months = [
 "January", "February", "March", "April", "May", "June",
 "July", "August", "September", "October", "November", "December",
];
```

```
export const PaymentMethods = [
 {
 id: "cod",
 name: "Cash On Delivery",
 value: "cod",
 },
 {
 id: "upi",
 name: "UPI",
 value: "upi",
 },
 {
 id: "credit-card",
 name: "Credit Card",
 value: "creditcard",
 },
 {
 id: "debit-card",
 name: "Debit Card",
 value: "debitcard",
 },
];
```

```
export const SocialMedia = [
 {
 id: "linkedin",
 name: "Linkedin",
 imgSrc: LinkedInImage,
 link: "https://www.linkedin.com/in/gagan-saini-gs/",
 },
 {
 id: "github",
 name: "Github",
 imgSrc: GithubImage,
 link: "https://github.com/Gagan-Saini-GS",
 },
];
```

```
export const AllFilters = [
 {
 id: "brands",
 name: "Brands",
 options: Brands,
 },
 {
 id: "sizes",
 name: "Sizes",
 options: Sizes,
 },
 {
 id: "genders",
 name: "Genders",
 options: Genders,
 },
 {
 id: "materials",
 name: "Materials",
 options: Materials,
 },
];
```

```
export const MIN_PRICE = 100;
export const MAX_PRICE = 100000;
```

Index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

```
body {
 margin: 0;
 font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", "Roboto", "Oxygen",
 "Ubuntu", "Cantarell", "Fira Sans", "Droid Sans", "Helvetica Neue",
 sans-serif;
 -webkit-font-smoothing: antialiased;
 -moz-osx-font-smoothing: grayscale;
 background-color: var(--White);
}
```

```
code {
 font-family: source-code-pro, Menlo, Monaco, Consolas, "Courier New",
 monospace;
```

```
}
```

```
:root {
 --White: #f0f3f6;
 --Light: #dfe7f1;
 --Gray: #999999;
 --Black: #1e1e1e;
 --Blue: #0f4392;
 --Purple: #7b68ee;
 --Green: #48d198;
 --Yellow: #efc863;
}
```

```
::-webkit-scrollbar {
 display: none;
}
```

```
input {
 outline: none;
}
```

```
.background-effect {
 position: relative;
 z-index: 1;
 color: var(--White);
 transition: all 100ms ease-in-out;
 padding: 5px;
 margin: 5px 0px;
}
```

```
.background-effect::before {
 content: "";
 position: absolute;
 top: 0px;
 right: 0px;
 bottom: 0px;
 left: 0px;
 transform: scaleY(0);
 transform-origin: 50%;
 background-color: var(--White);
 z-index: -1;
 transition: all 300ms ease-in-out;
 border-radius: 4px;
}
```

```
.background-effect:hover {
 color: var(--Black);
}
```

```
.background-effect:hover::before {
 transform: scaleY(1);
}
```

```
.hero-heading {
 background-image: linear-gradient(
 108deg,
 var(--Blue) 2%,
 var(--Green) 40%,
 var(--Black) 77%,
 var(--Yellow) 91%
);
 background-clip: text;
 -webkit-background-clip: text;
 -webkit-text-fill-color: transparent;
}
```

```
input::-webkit-outer-spin-button,
input::-webkit-inner-spin-button {
 -webkit-appearance: none;
 margin: 0;
}
```

```
img {
 object-fit: fill;
}
```

```
.validation-error-msg {
 font-size: 1rem;
 font-weight: 400;
 color: red;
}
```

```
.validation-error-input {
 border: 2px solid red;
}
```

```
input[type="checkbox"] {
 width: 20px;
```



```
height: 20px;
accent-color: var(--Purple); /* Changes the color when checked */
}
```

```
.slider .thumb {
background-color: var(--White);
border: 2px solid var(--Purple);
border-radius: 50%;
width: 20px;
height: 20px;
position: relative;
top: -9px;
cursor: grab;
outline: var(--Purple);
}
```

```
.track {
top: 0;
bottom: 0;
background: var(--Light);
border-radius: 999px;
}
```

```
.track-1 {
background: var(--Purple);
}
```

```
.css-13cymwt-control {
border: 2px solid;
color: var(--Black);
background-color: var(--Gray);
}
```

```
.fullscreen-loader {
position: absolute;
top: 0;
left: 0;
width: 100%;
height: 100%;
background-color: rgba(153, 153, 153, 0.3);
border-radius: 8px;
display: flex;
justify-content: center;
align-items: center;
```

```
/* z-index: 9999; */
}
```

```
.spinner {
 border: 5px solid var(--Gray);
 border-top: 5px solid var(--Purple);
 border-radius: 50%;
 width: 40px;
 height: 40px;
 animation: spin 1s linear infinite;
}
```

```
@keyframes spin {
 0% {
 transform: rotate(0deg);
 }
 100% {
 transform: rotate(360deg);
 }
}
```

```
/* Safer Side Color Classes*/
.white {
 background-color: #f0f3f6;
}
```

```
.black {
 background-color: #1e1e1e;
}
```

```
.navy {
 background-color: #000080;
}
```

```
.skyBlue {
 background-color: #87ceeb;
}
```

```
.green {
 background-color: #55eca8;
}
```

```
.red {
 background-color: #800020;
```

```
}

.mustard {
 background-color: #ffdb58;
}

.coral {
 background-color: #ff7f50;
}

.lavender {
 background-color: #e6e6fa;
}

.teal {
 background-color: #008080;
}

.olive {
 background-color: #808000;
}

.charcoal {
 background-color: #36454f;
}
```

[index.js](#)

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import reportWebVitals from "./reportWebVitals";
import App from "./Components/App/App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);

reportWebVitals();
```

```
package.json
{
 "name": "frontend",
 "version": "0.1.0",
 "private": true,
 "dependencies": {
```

```
"@testing-library/jest-dom": "^5.16.5",
"@testing-library/react": "^13.4.0",
"@testing-library/user-event": "^13.5.0",
"axios": "^1.2.4",
"react": "^18.2.0",
"react-dom": "^18.2.0",
"react-icons": "^5.1.0",
"react-router-dom": "^6.7.0",
"react-scripts": "5.0.1",
"react-select": "^5.8.0",
"react-slider": "^2.0.6",
"sweetalert": "^2.1.2",
"web-vitals": "^2.1.4"
},
"scripts": {
 "start": "react-scripts start",
 "build": "react-scripts build",
 "test": "react-scripts test",
 "eject": "react-scripts eject"
},
"eslintConfig": {
 "extends": [
 "react-app",
 "react-app/jest"
]
},
"browserslist": {
 "production": [
 ">0.2%",
 "not dead",
 "not op_mini all"
],
 "development": [
 "last 1 chrome version",
 "last 1 firefox version",
 "last 1 safari version"
]
},
"devDependencies": {
 "tailwindcss": "^3.4.4"
}
}
```

## Backend

```
server.js
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const { connectDB } = require("./config/db-config");
const { createUser } = require("./controllers/userControls/signup");
const { loginUser } = require("./controllers/userControls/login");
const { getUserDetails } = require("./controllers/userControls/user-details");
const {
 updateUserProfile,
} = require("./controllers/userControls/update-profile");
const { becomeSeller } = require("./controllers/userControls/become-seller");
const {
 getAllProducts,
} = require("./controllers/productControls/get-all-products");
const {
 getProductById,
} = require("./controllers/productControls/get-product-by-id");
const {
 uploadProduct,
} = require("./controllers/productControls/upload-product");
const {
 askProductQuestion,
} = require("./controllers/productControls/ask-product-question");
const {
 setProductReview,
} = require("./controllers/productControls/set-product-review");
const { addToCart } = require("./controllers/productControls/add-to-cart");
const {
 accessCartItems,
} = require("./controllers/productControls/access-cart-items");
const { removeItem } = require("./controllers/productControls/remove-item");
const { buyProduct } = require("./controllers/productControls/buy-product");
const {
 checkoutProduct,
} = require("./controllers/productControls/checkout-product");
const {
 searchProduct,
} = require("./controllers/productControls/search-product");
const authenticateJWT = require("./middleware/authenticateUser");
const {
 getOrderedProducts,
```

```
} = require("./controllers/productControls/get-ordered-products");
const { updateAddress } = require("./controllers/userControls/update-address");
const {
 updatePhoneNumber,
} = require("./controllers/userControls/update-phonenummer");
const port = 5000;
const app = express();
app.use(
 cors({
 origin: ["https://closet-fashion.vercel.app", "http://localhost:3000"],
 credentials: true,
 })
);
```

```
app.use(express.json({ limit: "50mb" }));
app.use(
 express.urlencoded({
 limit: "50mb",
 extended: true,
 parameterLimit: 50000,
 })
);
```

```
// Connecting Database by calling connectDB function
connectDB();
```

```
// User Related Routes
app.post("/signup", createUser);
app.post("/login", loginUser);
```

```
// Product Related Routes
app.post("/search-product", searchProduct); // Not Working
app.post("/get-all-products", getAllProducts);
```

```
// Authenticate JWT Routes
app.post("/user-details", authenticateJWT, getUserDetails);
app.post("/update-profile", authenticateJWT, updateUserProfile);
app.post("/update-address", authenticateJWT, updateAddress);
app.post("/update-phonenummer", authenticateJWT, updatePhoneNumber);
app.post("/become-seller", authenticateJWT, becomeSeller);
app.post("/get-product-with-id", authenticateJWT, getProductById);
app.post("/add-to-cart", authenticateJWT, addToCart);
app.post("/access-cart-items", authenticateJWT, accessCartItems);
app.post("/upload-product", authenticateJWT, uploadProduct);
```

```

app.post("/buy-product", authenticateJWT, buyProduct);
app.post("/checkout-product", authenticateJWT, checkoutProduct);
app.post("/ordered-products", authenticateJWT, getOrderedProducts);
app.post("/remove-from-cart", authenticateJWT, removeItem);
app.post("/ask-product-question", authenticateJWT, askProductQuestion);
app.post("/set-product-review", authenticateJWT, setProductReview);
app.listen(port, () => {
 console.log("Server is running at port 5000");
});

```

```

config\db-config.js
const mongoose = require("mongoose");
const DB_URL = process.env.DBURL;
const connectDB = async () => {
 try {
 await mongoose.connect(DB_URL);
 console.log("Database Connected Successfully");
 } catch (error) {
 console.error("MongoDB connection error:", error);
 process.exit(1);
 }
};

module.exports = { connectDB };
controllers\productControls\access-cart-items.js;
const Product = require("../models/Products");
const accessCartItems = async (req, res) => {
 try {
 const products = [];
 const userCart = await req.user.cart;
 for (let i = 0; i < userCart.length; i++) {
 const product = await Product.findById({ _id: userCart[i] });
 products.push({
 _id: product._id,
 name: product.name,
 price: product.price,
 productImages: product.productImages,
 brand: product.brand,
 category: product.category,
 quantity: 1,
 });
 }
 await res.json({ products });
 } catch (error) {
 res.status(500).json({ error: error.message });
 }
}

```

```

 console.log(error);
 }
};
module.exports = { accessCartItems };

controllers\productControls\add-to-cart.js
const mongoose = require("mongoose");
const addToCart = async (req, res) => {
 try {
 const userCart = await req.user.cart;
 if (!mongoose.Types.ObjectId.isValid(req.body.productID)) {
 return res.status(400).json({ error: "Invalid product ID" });
 }
 await userCart.push(req.body.productID);
 await req.user.save();
 res.status(200).json({ message: "Product added to cart" });
 } catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
 }
};
module.exports = { addToCart };

controllers\productControls\ask-product-question.js
const Product = require("../models/Products");
const askProductQuestion = async (req, res) => {
 try {
 const { productID, question } = req.body;
 const foundProduct = await Product.findById(productID);
 if (!foundProduct) {
 return res.status(404).json({ message: "Product not found." });
 }
 const arr = await foundProduct.questions;
 await arr.push({
 question: question,
 name: req.user.name,
 });
 await foundProduct.save();
 res.json(foundProduct.questions);
 } catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
 }
};
module.exports = { askProductQuestion };

```



```

controllers\productControls\buy-product.js
const Product = require("../models/Products");
const buyProduct = async (req, res) => {
 try {
 const productID = req.body.productID;
 const foundUser = req.user;
 const userDetails = {
 name: await foundUser.name,
 email: await foundUser.email,
 phoneNumber: await foundUser.phoneNumber,
 address: await foundUser.address,
 };
 const foundProduct = await Product.findById(productID);
 if (!foundProduct) {
 res.status(404).json({ error: "Product not found" });
 return;
 }
 const productDetails = {
 productImages: await foundProduct.productImages,
 name: await foundProduct.name,
 price: await foundProduct.price,
 description: await foundProduct.description,
 sizes: await foundProduct.sizes,
 colors: await foundProduct.colors,
 category: await foundProduct.category,
 brand: await foundProduct.brand,
 quantity: 1,
 };
 res.json({ userDetails, productDetails });
 } catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
 }
};
module.exports = { buyProduct };

```

```

controllers\productControls\checkout-product.js
const checkoutProduct = async (req, res) => {
 const user = req.user;
 const products = req.body.products;
 user.orders = [...user.orders, ...products];
 await user.save();
 res.status(200).json({ message: "Checkout successful" });
};

```

```

};
module.exports = { checkoutProduct };

controllers\productControls\get-all-products.js
const Product = require("../models/Products");
const getAllProducts = async (req, res) => {
 try {
 const { selectedFilters, searchQuery, page, limit } = req.body;
 let filterExist = false;
 const filters = {
 price: {
 minimum: selectedFilters.price[0].minimum,
 maximum: selectedFilters.price[0].maximum,
 },
 brands: [...selectedFilters.brands],
 sizes: [...selectedFilters.sizes],
 genders: [...selectedFilters.genders],
 materials: [...selectedFilters.materials],
 colors: [...selectedFilters.colors],
 };
 let query = {};
 if (
 filters.price.minimum !== undefined ||
 filters.price.maximum !== undefined
) {
 query.price = {};
 if (filters.price.minimum !== undefined) {
 query.price.$gte = parseFloat(filters.price.minimum);
 }
 if (filters.price.maximum !== undefined) {
 query.price.$lte = parseFloat(filters.price.maximum);
 }
 }
 if (filters.brands.length > 0) {
 query.brand = { $in: filters.brands };
 filterExist = true;
 }
 if (filters.sizes.length > 0) {
 query.sizes = { $in: filters.sizes };
 filterExist = true;
 }
 if (filters.genders.length > 0) {
 query.gender = { $in: filters.genders };
 filterExist = true;
 }
 }
};

```

```

 }
 if (filters.materials.length > 0) {
 query.materials = { $in: filters.materials };
 filterExist = true;
 }
 if (filters.colors.length > 0) {
 query.colors = { $in: filters.colors };
 filterExist = true;
 }
 if (searchQuery && searchQuery.trim()) {
 query.$or = [
 { name: { $regex: searchQuery, $options: "i" } },
 { description: { $regex: searchQuery, $options: "i" } },
 { brand: { $regex: searchQuery, $options: "i" } },
];
 filterExist = true;
 }
 const offset = filterExist ? 0 : (page - 1) * limit;
 const allProducts = await Product.find(query).skip(offset).limit(limit);
 const totalProducts = await Product.countDocuments(query);
 const products = await allProducts.map((product) => ({
 _id: product._id,
 name: product.name,
 price: product.price,
 brand: product.brand,
 category: product.category,
 productImages: product.productImages,
 }));
 res.status(200).json({
 totalProducts,
 totalPages: Math.ceil(totalProducts / limit),
 currentPage: page,
 products,
 });
 } catch (error) {
 res.status(500).json({ error: error.message });
 }
};
module.exports = { getAllProducts };

```

```

controllers\productControls\get-ordered-products.js
const Product = require("../models/Products");
const getOrderedProducts = async (req, res) => {
 try {

```

```

const orders = req.body.orders;
const products = await Promise.all(
 orders.map(async (order) => {
 const product = await Product.findById(order._id);
 if (product) {
 return {
 _id: product._id,
 name: product.name,
 price: product.price,
 productImages: product.productImages,
 brand: product.brand,
 category: product.category,
 quantity: order.quantity,
 };
 } else {
 throw new Error(`Product with id ${order._id} not found`);
 }
 })
);
await res.json({ products });
} catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
}
};
module.exports = { getOrderedProducts };

```

```

controllers\productControls\get-product-by-id.js
const Product = require("../models/Products");
const User = require("../models/Users");

```

```

const getProductById = async (req, res) => {
 try {
 const includeAuthToken = req.body.includeAuthToken;
 const productID = req.body.productID;
 const foundProduct = await Product.findById(productID);
 if (!foundProduct) {
 return res.status(404).json({ message: "Product not found." });
 }
 const seller = await User.findById(foundProduct.seller);
 if (!seller) {
 return res.status(404).json({ message: "Seller not found." });
 }
 let isProductAlreadyInCart = false;

```

```

 if (includeAuthToken !== undefined && includeAuthToken) {
 const userCart = await req.user.cart;
 if (userCart.includes(productID)) {
 isProductAlreadyInCart = true;
 }
 }
 const sellerDetails = {
 name: seller.name,
 email: seller.email,
 phoneNumber: seller.phoneNumber,
 };
 res.json({ foundProduct, sellerDetails, isProductAlreadyInCart });
 } catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
 }
};
module.exports = { getProductById };

```

controllers\productControls\remove-item.js

```

const removeItem = async (req, res) => {
 try {
 const productID = req.body.productId;
 const foundUser = req.user;
 const userCart = await foundUser.cart;
 const products = await userCart.filter((product) => product !== productID);
 foundUser.cart = products;
 await foundUser.save();
 await res.json({ message: "Item Removed" });
 } catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
 }
};
module.exports = { removeItem };

```

controllers\productControls\search-product.js

```

const Product = require("../models/Products");
const searchProduct = async (req, res) => {
 try {
 const searchQuery = req.body.searchQuery.toLowerCase();
 const products = await Product.find();
 const finalProducts = await products.filter((product) => {
 const { name, company, description } = product;

```

```

 const index1 = name.toString().toLowerCase().indexOf(searchQuery);
 const index2 = company.toString().toLowerCase().indexOf(searchQuery);
 const index3 = description.toString().toLowerCase().indexOf(searchQuery);
 if (index1 !== -1 || index2 !== -1 || index3 !== -1) {
 return product;
 }
});
res.json(finalProducts);
} catch (error) {
 res.status(500).json({ error: error.message });
}
};
module.exports = { searchProduct };

```

```

controllers\productControls\set-product-review.js
const Product = require("../models/Products");
const setProductReview = async (req, res) => {
 try {
 const { reviewContent, starCount, productID } = req.body;
 const foundProduct = await Product.findById(productID);
 if (!foundProduct) {
 return res.status(404).json({ message: "Product not found." });
 }
 const arr = await foundProduct.reviews;
 await arr.push({
 reviewContent: reviewContent,
 starCount: starCount,
 name: req.user.name,
 });
 await foundProduct.save();
 res.json(foundProduct.reviews);
 } catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
 }
};
module.exports = { setProductReview };

```

```

controllers\productControls\upload-product.js
const Product = require("../models/Products");
const User = require("../models/Users");
const uploadProduct = async (req, res) => {
 try {
 const productDetails = req.body.productDetails;

```

```

const sellerId = await req.user._id;
const seller = await User.findById(sellerId);
if (!seller) {
 return res.status(404).json({ error: "Seller not found" });
}
const product = new Product({
 name: productDetails.name,
 price: Number(productDetails.price),
 brand: productDetails.brand.toLowerCase(),
 gender: productDetails.gender.toLowerCase(),
 category: productDetails.category.toLowerCase(),
 materials: productDetails.materials.toLowerCase(),
 description: productDetails.description,
 sizes: productDetails.sizes.map((size) => size.name.toLowerCase()),
 colors: productDetails.colors.map((color) => color.color.toLowerCase()),
 productImages: productDetails.productImages,
 reviews: [],
 questions: [],
 seller: sellerId,
});
await product.save();
res.json("OK");
} catch (error) {
 res.status(500).json({ error: error.message });
 console.log(error);
}
};
module.exports = { uploadProduct };

```

controllers\userControls\become-seller.js

```

const becomeSeller = async (req, res) => {
 try {
 const foundUser = req.user;
 foundUser.isSeller = true;
 foundUser.PANCardNumber = req.body.sellerPANCardNumber;
 foundUser.GSTNumber = req.body.sellerGSTNumber;
 foundUser.TandC = true;
 await foundUser.save();
 const data = "OK";
 res.json({ data });
 } catch (error) {
 console.error("Error:", error);
 res.status(500).json({ error: "An internal server error occurred." });
 }
}

```

```

}
module.exports = { becomeSeller };

controllers\userControls\login.js
const User = require("../models/Users");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");

const loginUser = async (req, res) => {
 try {
 const { email, password } = req.body.user;
 const foundUser = await User.findOne({ email: email });
 if (!foundUser) {
 return res.status(401).json({ error: "Invalid email or password." });
 }
 const passwordsMatch = await bcrypt.compare(password, foundUser.password);
 if (passwordsMatch) {
 const authToken = jwt.sign(
 {
 _id: foundUser._id,
 name: foundUser.name,
 email: email,
 },
 process.env.AUTH_TOKEN
);
 return res.json({ authToken: authToken });
 } else {
 // If passwords don't match, return error
 return res.status(401).json({ error: "Invalid email or password." });
 }
 } catch (error) {
 console.error("Error in login:", error);
 return res
 .status(500)
 .json({ error: "An internal server error occurred." });
 }
};

module.exports = { loginUser };

controllers\userControls\signup.js
const User = require("../models/Users");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const saltRounds = 8;

```



```

const createUser = async (req, res) => {
 try {
 const { name, email, password } = req.body.user;
 const hash = await bcrypt.hash(password, saltRounds);
 const newUser = new User({
 name: name,
 email: email,
 profileImage: "",
 password: hash,
 phoneNumber: "",
 address: "",
 website: "",
 cart: [],
 orders: [],
 recentsProducts: [],
 isSeller: false,
 PANCardNumber: "",
 GSTNumber: "",
 TandC: true,
 });
 await newUser.save();
 const authToken = jwt.sign(
 {
 _id: newUser._id,
 name: name,
 email: email,
 },
 process.env.AUTH_TOKEN
);
 res.json({ authToken: authToken });
 } catch (error) {
 console.log("Signup Issue: Can't create new user. ", error);
 return res
 .status(500)
 .json({ error: "An internal server error occurred." });
 }
};

module.exports = { createUser };

```

```

controllers\userControls\update-address.js
const updateAddress = async (req, res) => {
 try {
 const foundUser = req.user;
 const address = req.body.address;

```

```

 foundUser.address = address || foundUser.address;
 await foundUser.save();
 res.json({
 status: 200,
 message: "Address updated successfully.",
 data: foundUser,
 });
 } catch (error) {
 console.error("Error in updating profile:", error);
 res.status(500).json({ error: "An internal server error occurred." });
 }
};
module.exports = { updateAddress };

```

```

controllers\userControls\update-phonenumbers.js
const updatePhoneNumber = async (req, res) => {
 try {
 const foundUser = req.user;
 const phoneNumber = req.body.phoneNumber;
 foundUser.phoneNumber = phoneNumber || foundUser.phoneNumber;
 await foundUser.save();
 res.json({
 status: 200,
 message: "Phone number updated successfully.",
 data: foundUser,
 });
 } catch (error) {
 console.error("Error in updating profile:", error);
 res.status(500).json({ error: "An internal server error occurred." });
 }
};
module.exports = { updatePhoneNumber };

```

```

controllers\userControls\update-profile.js
const updateUserProfile = async (req, res) => {
 try {
 const foundUser = req.user;
 const user = req.body.user;
 foundUser.name = user.name || foundUser.name;
 foundUser.email = user.email || foundUser.email;
 foundUser.profileImage = user.profileImage || foundUser.profileImage;
 foundUser.phoneNumber = user.phoneNumber || foundUser.phoneNumber;
 foundUser.address = user.address || foundUser.address;
 foundUser.website = user.website || foundUser.website;
 }
};

```

```

 await foundUser.save();
 res.json({
 status: 200,
 message: "Profile updated successfully.",
 data: foundUser,
 });
 } catch (error) {
 console.error("Error in updating profile:", error);
 res.status(500).json({ error: "An internal server error occurred." });
 }
};
module.exports = { updateUserProfile };

```

```

controllers\userControls\user-details.js
const getUserDetails = async (req, res) => {
 try {
 const foundUser = req.user;
 const userDetails = {
 name: foundUser.name,
 email: foundUser.email,
 phoneNumber: foundUser.phoneNumber,
 address: foundUser.address,
 profileImage: foundUser.profileImage,
 website: foundUser.website,
 orders: foundUser.orders,
 isSeller: foundUser.isSeller,
 };
 res.json({ userDetails });
 } catch (error) {
 console.log("Issue: Can't get user details. ", error);
 return res
 .status(500)
 .json({ error: "An internal server error occurred." });
 }
};
module.exports = { getUserDetails };

```

```

middleware\authenticateUser.js
const jwt = require("jsonwebtoken");
const User = require("../models/Users");
const authenticateJWT = async (req, res, next) => {
 const includeAuthToken = req.body.includeAuthToken;
 const authHeader = req.headers.authorization;
 if (!authHeader && includeAuthToken !== undefined && includeAuthToken) {

```

```

 return res.status(401).json({ message: "No auth token provided" });
 }
 if (includeAuthToken !== undefined && !includeAuthToken) {
 next();
 } else {
 const token = authHeader.split(" ")[1];
 try {
 const decoded = jwt.verify(token, process.env.AUTH_TOKEN);
 const user = await User.findById(decoded._id);
 if (!user)
 return res.status(401).json({ message: "Invalid auth token" });

 req.user = user;
 next();
 } catch (err) {
 return res
 .status(403)
 .json({ message: "Token is not valid", error: err.message });
 }
 }
};
module.exports = authenticateJWT;

```

```

models\Products.js
const mongoose = require("mongoose");
const productSchema = new mongoose.Schema({
 name: String,
 price: Number,
 brand: String,
 gender: String,
 category: String,
 materials: String,
 description: String,
 sizes: [],
 colors: [],
 productImages: [],
 reviews: [],
 questions: [],
 seller: {
 type: mongoose.Schema.Types.ObjectId,
 ref: "User",
 },
});
const Product = new mongoose.model("product", productSchema);

```

```
module.exports = Product;
```

```
models\Users.js
```

```
const mongoose = require("mongoose");
const userSchema = new mongoose.Schema({
 userID: String,
 name: String,
 email: String,
 profileImage: String,
 password: String,
 phoneNumber: String,
 address: String,
 website: String,
 cart: [],
 orders: [],
 recentsProducts: [],
 isSeller: Boolean,
 sellerEmail: String,
 PANCardNumber: String,
 GSTNumber: String,
 TandC: Boolean,
});
```

```
const User = new mongoose.model("user", userSchema);
module.exports = User;
```

```
utils\categoryList.js
```

```
module.exports = {
 categories: ["all-cloths", "tshirts", "shirts", "shoes", "hoodies", "others"],
};
```

```
utils\companyList.js
```

```
module.exports = {
 prices: ["799", "1299", "1899", "2399", "3099"],
};
```

```
package.json
```

```
{
 "name": "backend",
 "version": "1.0.0",
 "description": "This is backend of Closet Fashion",
 "main": "index.js",
 "scripts": {
 "start": "node server.js",
```

```
 "test": "echo \\\"Error: no test specified\\\" && exit 1"
 },
 "author": "Gagan Saini",
 "license": "ISC",
 "dependencies": {
 "bcrypt": "^5.1.0",
 "cors": "^2.8.5",
 "dotenv": "^16.0.3",
 "express": "^4.18.2",
 "jsonwebtoken": "^9.0.0",
 "mongoose": "^6.8.3"
 }
}
```

—

**END OF CODE FOR CLOSET FASION (AN E-COMMERCE WEB SOLUTION)**